Starling

Animated renderings of data structures for teaching

Manual for v0.2.0

Contents

1. Introduction

Starling renders animated data structures in Typst, built on top of cetz, typsy, and (optionally) touying. The package is designed for a programming course: it ships a binary search tree today and is structured to grow to heaps, hash tables, and graphs without rewriting the animation kernel.

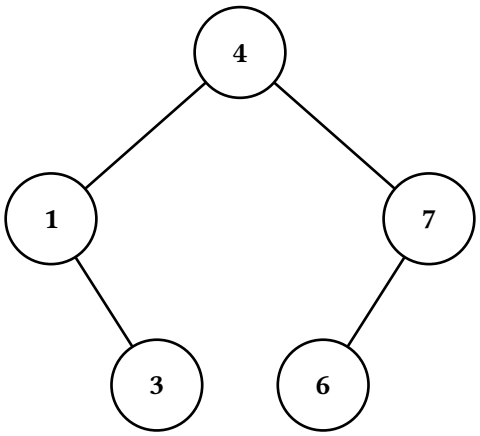
Every animation in starling is built from the same primitive — an ordered array of `Frame` records. Each `Frame` carries a rendered cetz canvas, an optional textual caption, and free-form step metadata. The package supplies a few helpers to collapse the array into a final image, a vertical stack, or an array of subslide figures, but you are free to ignore them and lay frames out however your document needs.

1.1. Quick start

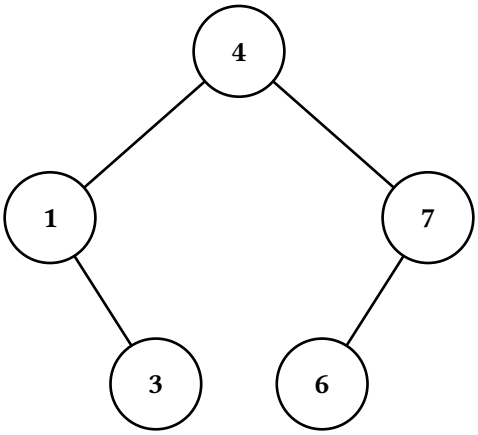
```
#import "@preview/starling:0.2.0" as starling
#import starling: bst

#let t = bst(4, 1, 7, 3, 6)
```

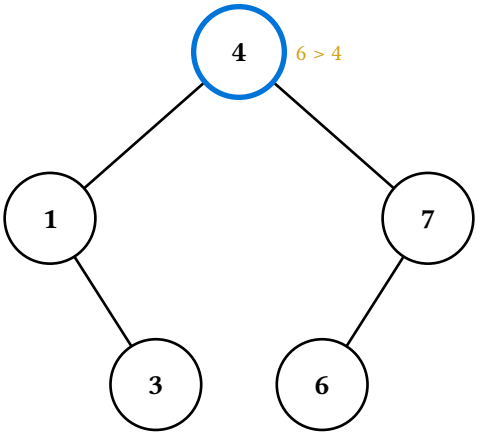
The first argument becomes the root; the rest are inserted in order. For custom node labels, pass a `(value, label)` 2-tuple in place of a bare value — e.g. `bst((4, [FOUR]), 1, (7, [SEVEN]))`. The lower-level constructor `(BST.new)(value:, label:, left:, right:)` and the `(t.insert-many)(..vals)` method are still available for finer control.

A static render of the tree, taking the final frame from `(t.display)()`:

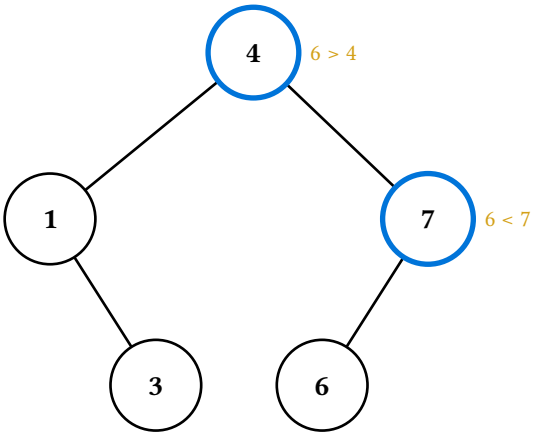


A full search animation, stacked vertically with per-step captions:

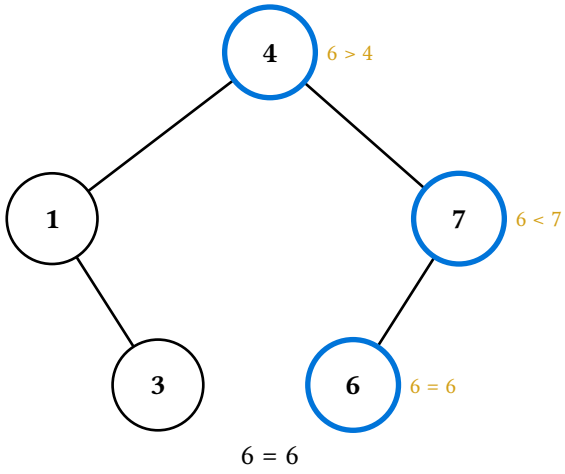




6 > 4



6 < 7



2. Architecture

Starling is layered into three source files, each with a single responsibility:

File	Role
`src/anim-core.typ`	The structure-agnostic animation core — defines `Snapshot`, `Frame`, the `Renderer` (with a pluggable draw backend), `make-renderer`, `Op`, and `apply-ops`. Knows nothing about any particular structure.
`src/tree-anim.typ`	The tree backend on the core — `draw-tree`, the cetz-tree builders, `PathId`, `path-anchor`, and the tree-bound `make-renderer` wrapper. Re-exports the core.
`src/bst.typ`	The BST class — implements pure operations (`insert`, `delete`, `rotate`) and the `*-display` methods that build animations using the kernel.
`src/graph-draw.typ`	The graph renderer on the core — `draw-graph`, `edge-key`, `node-anchor`, and the graph-bound `make-graph-renderer` wrapper. The graph analog of `tree-anim.typ`; knows drawing, not algorithms.
`src/graph.typ`	The Graph class — data plus the MST / Dijkstra / BFS / DFS `*-display` algorithms. Split from its renderer, like `bst.typ` is from `draw-tree`.
`src/graph-layout.typ`	Optional graphviz auto-layout (`auto-layout`) via `diagraph-layout`. The only file referencing that dependency, and it does so with a lazy import inside `auto-layout`'s body — so the dependency stays optional even though `lib.typ` re-exports the function.
`src/git-graph.typ`	The git-graph DSL — a **stateful** cetz builder (`commit`, `branch`, `merge`, `tag`, `branch-pointer`, `head-pointer`, `detached-commit`) for drawing git commit graphs. Deliberately off the `Frame` stack; surfaced under the `starling.git.*` namespace. Carries its own per-DS theme (`default-git-theme` / `set-git-theme`).
`src/lib.typ`	The public surface — re-exports everything users need, plus the render helpers (`last`, `stacked`, `figures`, `canvases-only`).

This split lets us add a new data structure (e.g. a heap) by writing just a new file at the `bst.typ` layer; the kernel does not need to change. Conversely, a power user who wants to assemble custom animations can talk to the kernel directly without touching the BST class at all.

2.1. Snapshots, frames, and the renderer

Four types do most of the work. Their roles are intentionally distinct:

Type	Role
`Snapshot`	A *sparse style overlay* for one moment in time — which nodes are filled what colour, which edges are dashed or hidden, which notes are attached to which nodes. Built up by chaining `style-node` / `style-edge` / `note-node` calls. Pure data; no theme awareness, no content.
`TreeRenderer`	A tree plus an ordered list of snapshots plus optional caption / step-metadata for each snapshot. The thing you build up while describing an animation.
`Frame`	The *output* record: `(render, caption, step, alt)`. `render` is a builder function `(op-theme, render-theme) -> content` — not pre-baked content — so callers can resolve theme state at layout time and reuse the same animation across documents with different themes. `TreeRenderer.render` produces an array of these, one per snapshot. This is what `*-display` methods return.
Theme dict	One of three layers, in merge order: `render-theme` (structural defaults like node fill, edge stroke, note colour), `op-theme` (operation strokes/fills shared across data structures, like `search-stroke`, `success-fill`, `traversal-palette`), and an optional per-DS theme (e.g. `rbt-theme`'s red/black palette). Frames are theme-agnostic until a render helper feeds resolved themes into each frame's `render` builder.

The data flow, end to end:

`Snapshot` array → `TreeRenderer.render`
→ `Array(Frame)` *(builders, theme-agnostic)*
→ helper resolves theme state once
→ helper calls each `(f.render)(op, rt)`
→ document content

The shape matters: snapshots and frames are both pure data until the final step. That makes them inspectable, slicable, and composable — you can pull a frame out, look at its `step` metadata, swap its caption, or thread it into a custom layout without rendering. The theming-aware materialisation is concentrated at the helper boundary.

2.2. Path identity

Every node is identified by its position in the tree, encoded as a string of `"L"` and `"R"` characters from the root. The root is `""`, the right child of the root is `"R"`, the left child of `"R"` is `"RL"`, and so on. Edges are identified by the path of their *child* node — each child has exactly one parent, so this is unambiguous.

Path identity is the linchpin that keeps node and edge styling independent of the tree's value or layout. It also means a node-highlighting animation built for one tree can be reused on a tree of the same shape with different values just by passing a different `tree` to `make-renderer`.

The only places that interpret path characters are `PathId` (the refined string type), `_build-cetz-tree` (which walks `.left` and `.right`), and the BST's own `by-value` / `path-to` / `resolve` methods. Everywhere else treats paths as opaque keys, which leaves room to generalise — n-ary trees would use slash-separated child indices like `"0/1/2"` or arrays of ints, and only those four places need to change.

2.3. Sticky animation accumulation

`make-renderer(tree, sticky: true)` is the common case: each new frame pushed via `r.push-frame()` starts from the previous frame's snapshot. That means a sequence like

```
#let r = starling.make-renderer(t, sticky: true)
#let r = (r.push-with-node)("",  stroke: blue + 2pt)   // frame 1
#let r = (r.push-with-node)("L", stroke: blue + 2pt)   // frame 2
#let r = (r.push-with-node)("LR", stroke: blue + 2pt)  // frame 3
```

produces three frames where each highlights one more node than the last — frame 3 has all three highlights, not just `LR`. This matches the natural mental model for teaching ("we visited these nodes in order"). Set `sticky: false` if you want each frame to start fresh.

Captions and `step` metadata do *not* sticky-accumulate; each frame's caption is whatever was last set on that frame, with no carry-over.

2.4. The cetz integration

The animation kernel renders each snapshot via `draw-tree`, which emits cetz draw commands using `cetz.tree.tree(...)` for layout. The key choice here is *not* to wrap `draw-tree`'s output in `cetz.canvas` inside the kernel — that wrap happens one layer up, in `_render-canvas`. The two functions are split so a power user can write their own `cetz.canvas` block, call `draw-tree` inside it, and add their own cetz annotations alongside the tree (see Section 12).

Anchors for individual nodes are exposed by `path-anchor`, which translates an L/R path string into the cetz anchor name produced by `draw-tree`. The fact that this needs a translation function at all is a quirk worth flagging — see Section 3.7.

3. Design decisions

This section explains the major calls that shape the API. None of them are obvious from reading the code, but each was a conscious choice and the reasoning is load-bearing for anyone considering a refactor.

3.1. Why frames, not a wrap callback

The original (v0.1) API let users configure `starling.configure(wrap: alternatives)` to bake touying's `alternatives` into the BST class via closure capture. Each `*-display` method internally called `wrap(..figs)` on the array of figures it generated.

That approach was rigid in two ways. First, the rendering decision was made *inside* the animation method, so the same animation could not appear as a static figure in handouts and as a subslide sequence in slides — you'd configure differently per context. Second, and worse, the animation was always one opaque blob: there was no way to weave the caption "6 > 4" alongside other slide content while keeping the visual animation synchronised, because everything lived inside a single `alternatives` mark.

The current API solves both: `*-display` returns the raw frame array, and the helpers (`last`, `stacked`, `figures`) collapse it however the caller wants. Wanting to lay captions in a sidebar while the animation plays? Pull them out yourself:

```
#let frames = (t.search-display)(6)
#grid(columns: 2,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

A historical note worth preserving: closure-captured `wrap` was itself chosen over a state-based approach (`std.state`, typsy's `safe-state`) because touying's `alternatives` is a layout-time "mark" that touying rejects inside `context { ... }` blocks. With frames as plain data, that constraint no longer applies to the animation API — and state did become viable later for the theming layer (see *State for ergonomics, per-call for perf* below).

3.2. Three theme layers, one per concern

Theming is split into three dicts that each own a different concern:

- **Render theme** (`default-render-theme`, in `anim-core.typ`) holds structural defaults — node fill, node stroke, node text fill, edge stroke, note fill, edge-tag fill (persistent edge labels like AVL heights and graph weights), and note-bg (the fill drawn behind a node's in-canvas annotations so they stay legible over edges; defaults to `white`, set it to the page color on a dark background). Anything any data structure would need to render at all.
- **Op theme** (`default-op-theme`, in `op-theme.typ`) holds operation- semantic strokes/fills/palettes that apply to *any* data structure with those operations — `search-stroke`, `attention-stroke`, `success-stroke`, `settled-stroke`, `success-fill`, `danger-stroke`, `reset-stroke`, `traversal-palette`. BST search and a hypothetical hash-table search share the same `search-stroke` from this layer.
- **Per-DS theme** (e.g. `default-rbt-theme`, in `rbt.typ`) holds styling intrinsic to one data structure — the red/black palette for RBTs is the canonical example. BST itself has no per-DS theme, because BST nodes have no intrinsic differentiation.

The split lets each layer own its own concerns. A new data structure (heap, trie, B-tree) brings its own per-DS theme dict if needed and reuses the render theme and op theme as-is. If operation strokes lived in a BST theme, every new data structure would either duplicate them or grow a coupling to BST.

The render theme also has a clear "lowest in the chain" role: `draw-tree` falls back to it when neither a snapshot override nor a `make-renderer(default-node-style:, default-edge-style:)` argument specifies a property. So the merge order, lowest precedence first, is: `default-render-theme` → user render-theme override → op-theme / per-DS-theme overlays applied per snapshot → renderer defaults → per-snapshot per-path overrides. Each layer adds more specificity.

Strokes throughout the op theme are full stroke dictionaries (`(paint:, thickness:, dash:)`), not bare colours. That lets users change any aspect of a stroke (dash pattern, cap, width) without us adding more theme keys for each possibility.

3.3. Frames carry builders, not pre-rendered content

`Frame.render` is a function `(op-theme, render-theme) -> content`, not a piece of pre-baked content. This is so render helpers (`last`, `stacked`, `figures`) can resolve theme state *once per call* and feed those resolved themes into every frame's builder, rather than each frame independently resolving theme inside its own context block.

The frame-as-builder shape also has a non-perf benefit: an `Array(Frame)` is now a portable, theme-agnostic description of an animation. The same frames can render against different themes in different contexts without re-running the `*-display` method. That's a cleaner separation than the older `(canvas, caption, step)` shape, where `canvas` baked in whatever theme was active at construction time.

A typsy quirk worth noting: typsy auto-injects `self` into any function-typed field on a class. To keep `(frame.render)(op, rt)` from getting a spurious third argument, the actual closure is stored as `_builder: (fn: ...)` — a singleton dict around the function — and exposed through a `render` method that dereferences it. Users only see `(frame.render)(op, rt)`; the dict wrap is private.

RBT and any future per-DS theme are read inside the frame's builder itself, not by the lib.typ helpers. The helpers only resolve the two universal layers — op-theme and render-theme — and pass them in; per-DS themes are state-read on demand. That keeps the helper signature stable as more data structures land.

3.4. State for ergonomics, per-call for perf

Theme overrides come in two flavours:

Form	Behaviour
`set-op-theme((..))` / `set-render-theme((..))` / `set-rbt-theme((..))`	State-based. One declaration at the top of the document propagates through every subsequent `*-display` call. Ergonomic and intent-matching, but participates in Typst's state-convergence machinery — see Section 10.3.
`(t.search-display)(v, theme: (..))`	Per-call. Overrides the theme for one call; ignores state for that call. Verbose if you have many calls, but skips the state-cost path entirely. Run at the no-state baseline.

Both paths exist because there's no single answer. State is right for a document that uses one theme throughout — write `set-op-theme(my-palette)` once, forget about it. Per-call is right when compile speed dominates and the user is happy threading a `let palette = (..)` value through their calls. The library does not pick a winner.

The cost being structural to Typst — not something starling can optimise away — meant choosing whether to expose state at all was a real call. Skipping state would have removed the perf footgun but also removed the ergonomic win. Exposing both, with the cost documented up-front, lets users decide.

3.5. Two-channel captions

For `search-display` and `insert-display`, the comparison string ("6 > 4") appears in *two* places:

- **Inline**, as a small note drawn beside the highlighted node in the cetz canvas. This labels which comparison happened at which node spatially.
- **Caption**, as a textual track on the `Frame` record. This is what `stacked` and `figures` render below the canvas, and what callers pulling captions out for custom layouts read.

The redundancy is deliberate. The inline note is part of the visual: when looking at a single frame, you should be able to see at a glance what the algorithm just did. The caption is a parallel *textual* track for layouts that want narration apart from the animation.

This is why `last(frames)` defaults to `caption: false` — the inline note already labels the node, so adding a caption block below the static figure would be redundant. `stacked` and `figures` default to `caption: true` because animated rendering invites the textual narrative.

3.6. The six-frame rotation

`rotate-display` walks through rotation as six discrete frames: `init`, `pivots`, `break`, `restructure`, `connect`, `settle`. The sequence is calibrated for teaching: each frame answers exactly one question.

`step.kind`	Question answered
`init`	What's the starting tree?
`pivots`	Which two nodes are rotating?
`break`	Which edges are about to disappear?
`restructure`	What does the new shape look like, edges aside?
`connect`	Where do the new edges go?
`settle`	What does the final, clean tree look like?

An earlier draft had a "dashed red" intermediate between `pivots` and `break` (edges going dashed before disappearing). That was dropped because the orange pivot highlight already cues the eye to those edges — the extra frame added length without information.

The `restructure` frame is the load-bearing one: it shows the new tree shape with the rotated edges still hidden, so the student can see the new positions form *before* the edges connect. Without it, the rotation would collapse into one cut from "broken" to "rotated + highlighted edges", which is too much information at once.

3.7. Path-based anchors and the cetz tree quirk

`cetz.tree.tree` numbers its nodes internally as `0`, `0-0`, `0-1`, `0-0-1`, … — sibling indices joined by hyphens, with the root being `0`. Starling configures cetz-tree's `group-name-prefix` to `"node-"`, so the resulting anchor names look like `node-0-0-1`. The whole tree also sits inside an outer named group (`"tree"` by default), so the fully qualified anchor for path `"LR"` is `tree.node-0-0-1`.

Two things made me reach for `path-anchor` rather than letting users type that out:

1. The user thinks in `"LR"`, not `"0-0-1"`. The translation is mechanical (`L → 0`, `R → 1`, prepend the root `0`) but it's error-prone to do by hand.
2. The naming scheme is a cetz-tree implementation detail. Wrapping it in `path-anchor(path, tree-name:, prefix:)` lets us swap to a different naming scheme later (e.g. if we drop cetz-tree for a custom layout) without churn at every annotation call site.

Phantom siblings (used to off-centre lone children — see `_build-cetz-tree`) also get cetz-tree-internal anchor names like `node-0-0-0`, but they are hidden at draw time and you generally should not annotate them.

4. BST animations tour

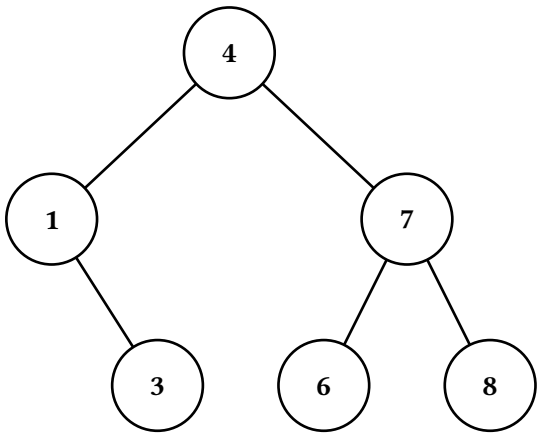
The BST class provides five `*-display` methods. Each returns `Array(Frame)`; the examples below pass the result to `starling.stacked` to render every frame vertically with its caption.

We'll use the same sample tree throughout this section, seeded via the `bst(..vals)` factory (first arg = root, rest are inserted):

```
#let t = bst(4, 1, 7, 3, 6, 8)
```

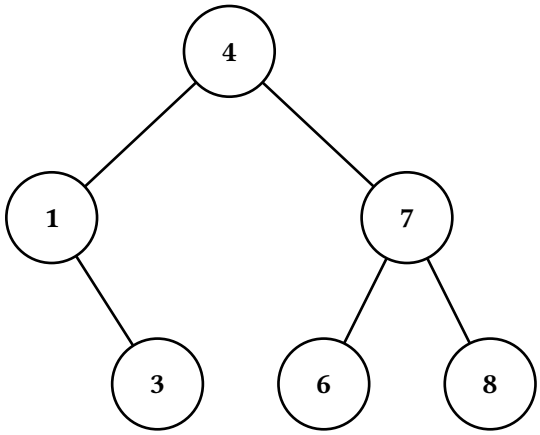
4.1. Static display

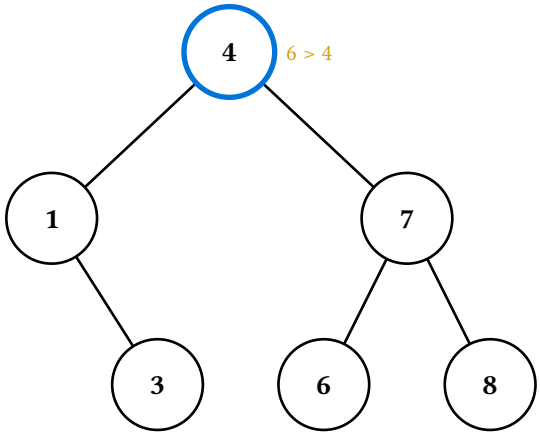
`(t.display)()` returns a one-element frame array — the unmodified tree, with no step metadata.



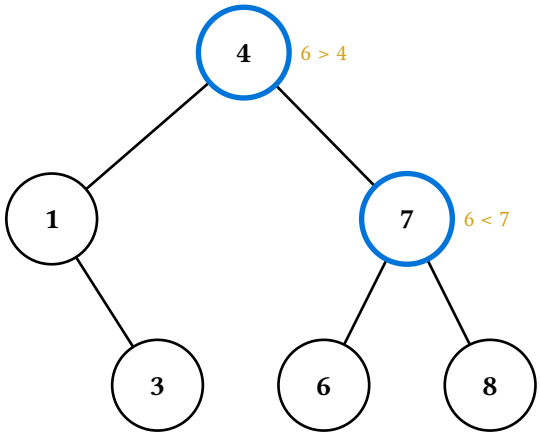
4.2. Search

`(t.search-display)(v)` walks the search path, highlighting each node visited and labelling it with the comparison made. `step.kind` is `"init"` for the initial frame and `"compare"` for each step; `step.found` records whether the value was reached.

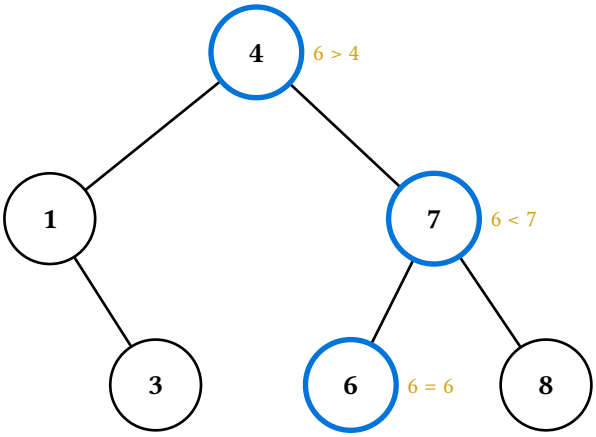




6 > 4



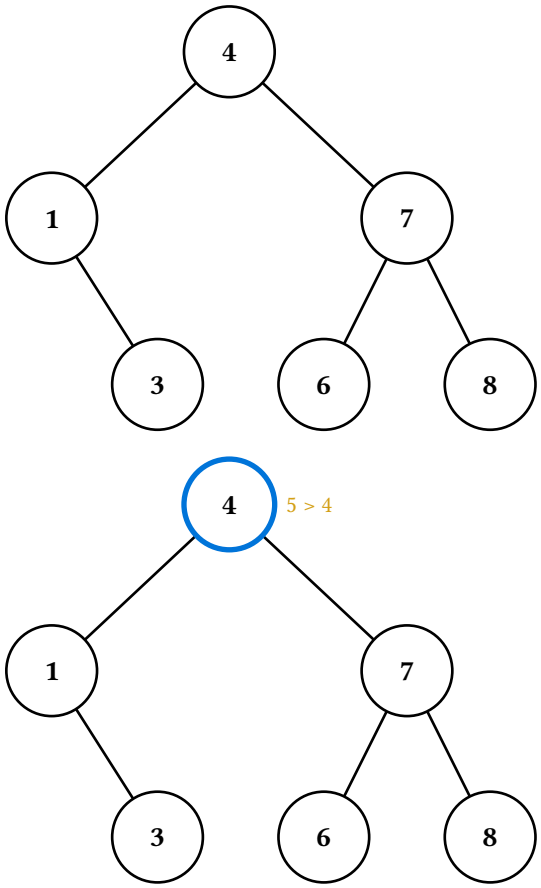
6 < 7



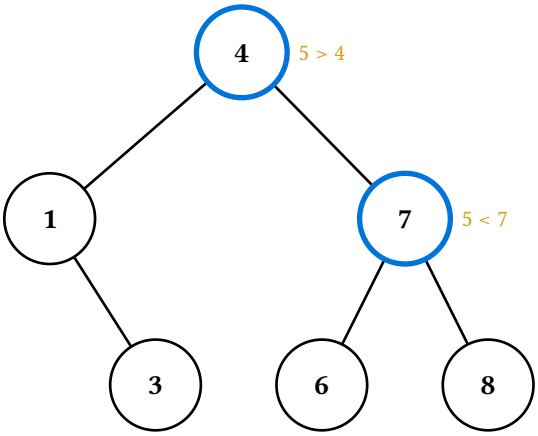
6 = 6

4.3. Insert

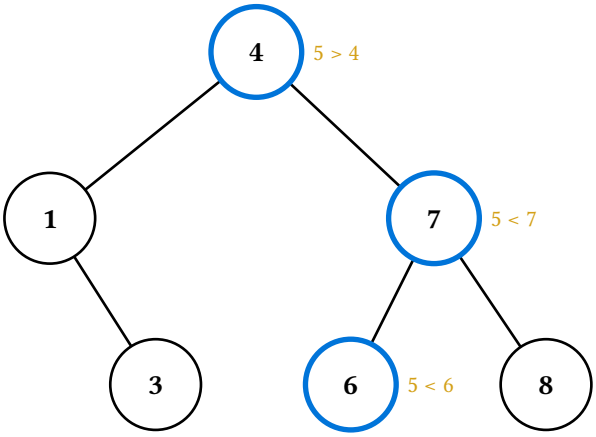
`(t.insert-display)(v)` walks the search path as for `search-display`, then transitions to the after-tree with the new node highlighted via `success-stroke` / `success-fill`. `step.kind` is `"init"`, `"compare"` (per search step), and finally `"inserted"`.



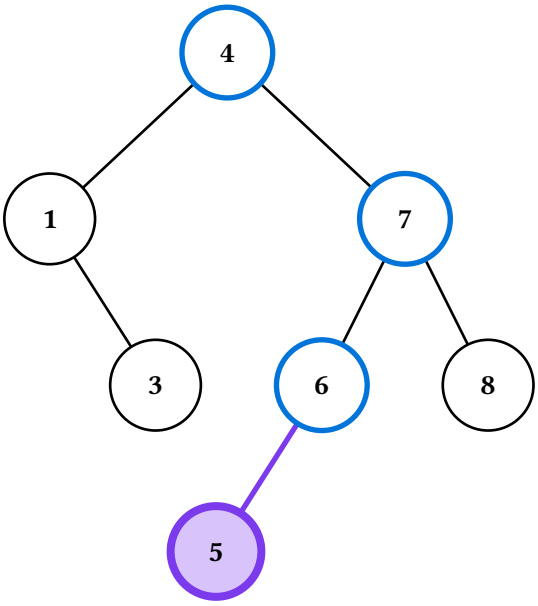
5 > 4



5 < 7



5 < 6

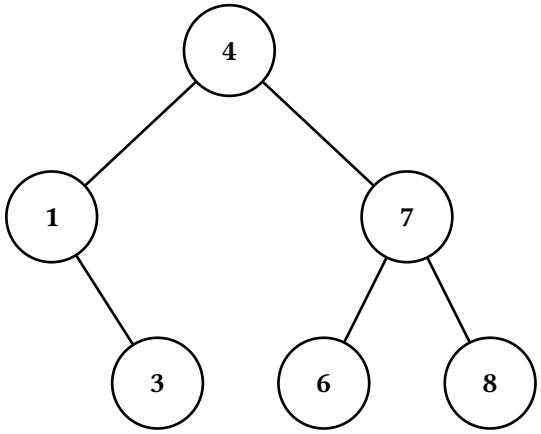


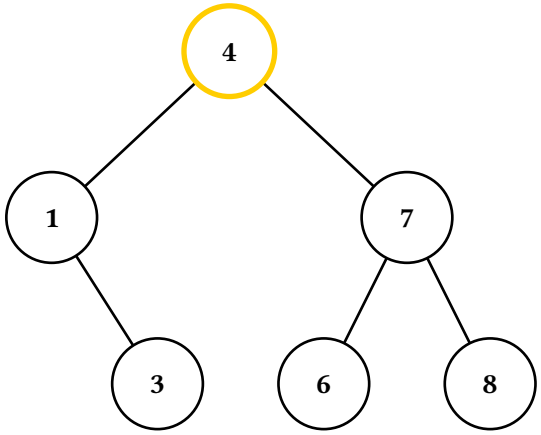
Inserted 5

4.4. Delete

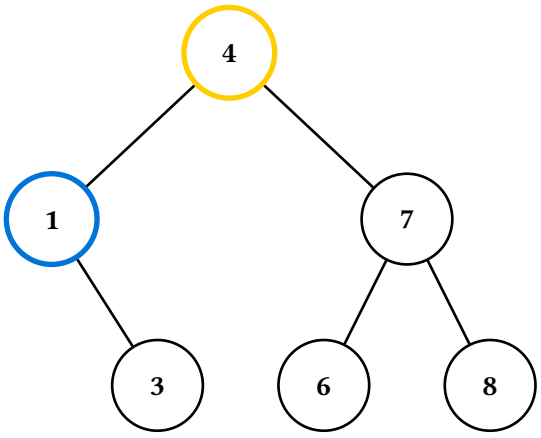
`(t.delete-display)(v)` has three internal cases — leaf, one-child, and two-children — and dispatches on the target's children. `step.kind` ranges over `"init"`, `"highlight"`, `"break"`, `"descend"`, `"transfer"`, and `"settle"` depending on case.

The two-children deletion is the most complex: it highlights the target, descends into the left subtree to find the in-order predecessor, annotates the value transfer, then jumps to the after-tree with the new root of the affected subtree highlighted via `success-stroke`.

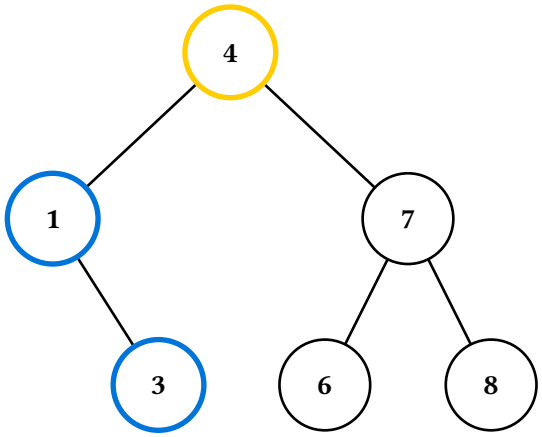




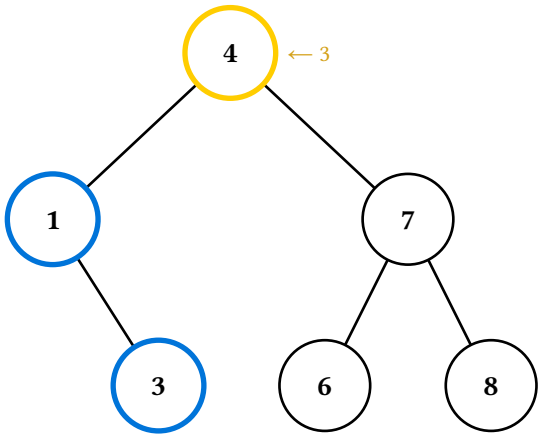
Delete 4



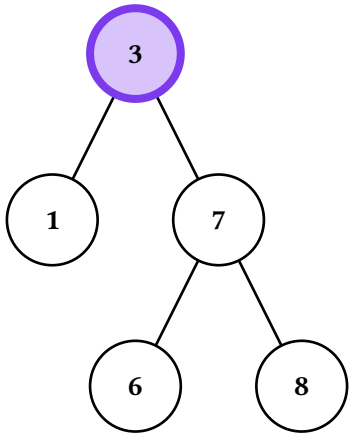
Find predecessor



Find predecessor



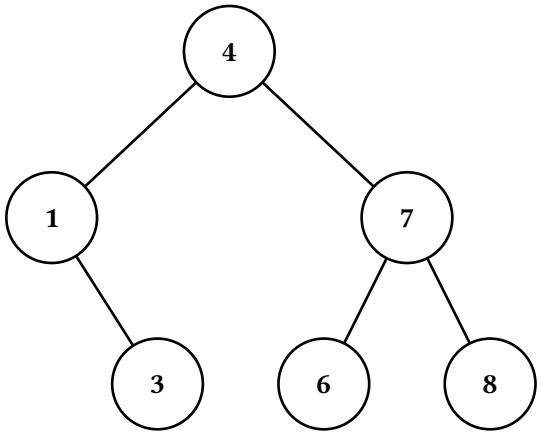
Transfer 3

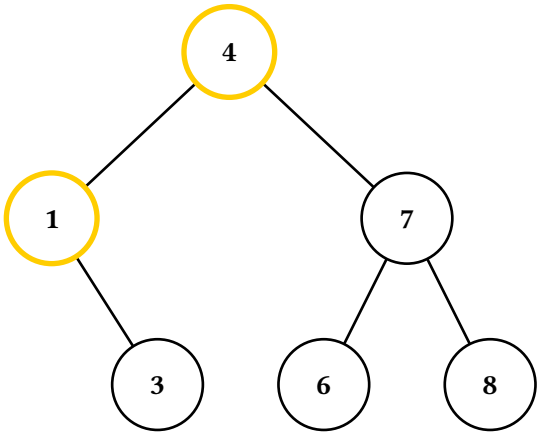


Done

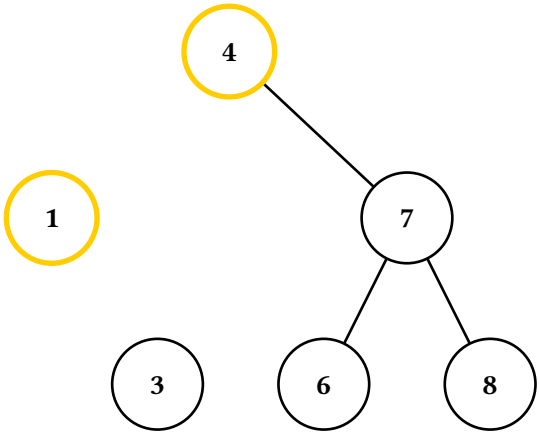
4.5. Rotate

`(t.rotate-display)(c)` rotates around node `c` — any node in the tree with a parent, not just a direct child of the root. The direction (left/right) is inferred from `c`'s position in the BST. The six-frame sequence is described in Section 3.6.

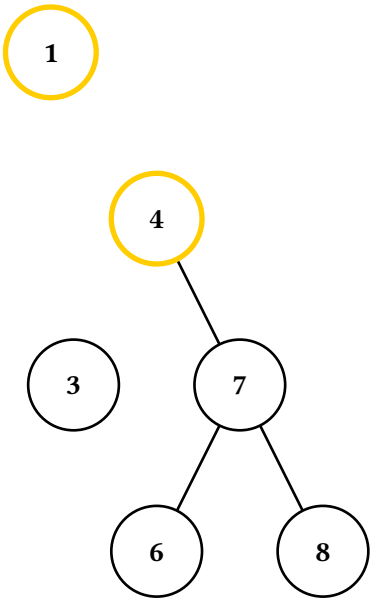




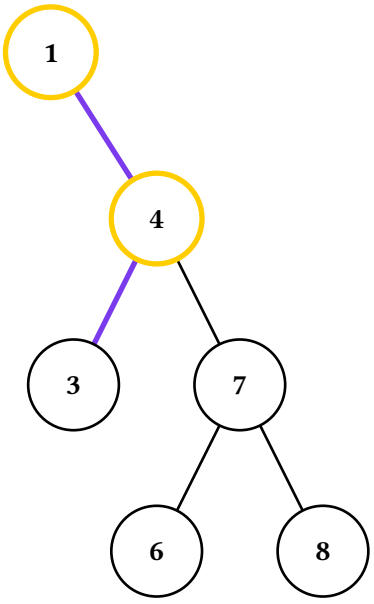
Rotate around 1



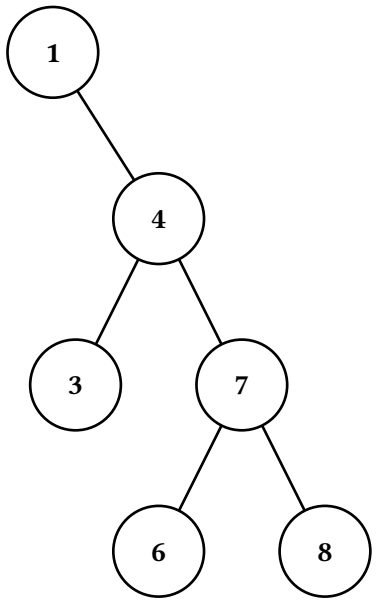
Break edges



Restructure tree



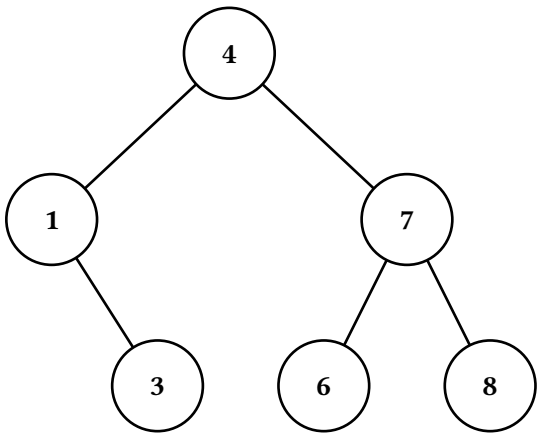
Reconnect edges

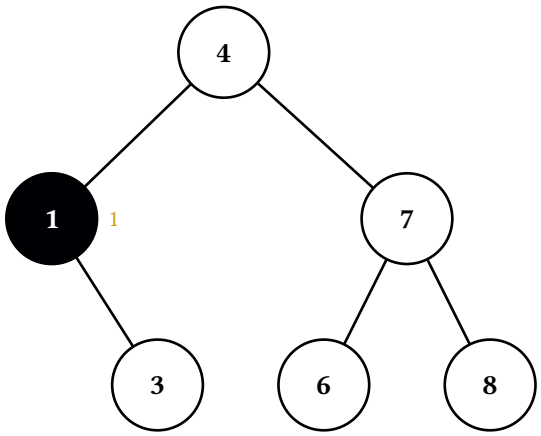


4.6. Traversals

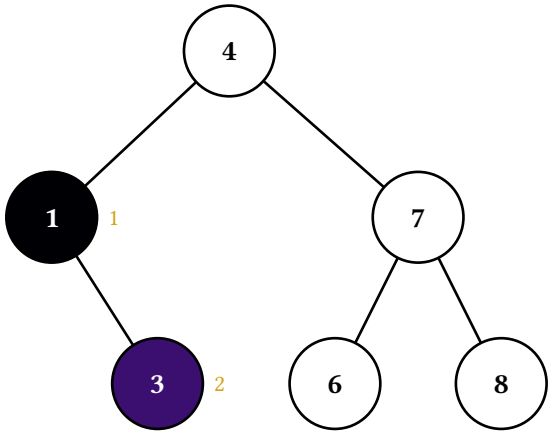
`(t.in-order-display)()`, `(t.pre-order-display)()`, `(t.post-order-display)()`, and `(t.level-order-display)()` animate the four standard traversals. Each returns one frame per visit (plus the initial frame): the visited node is filled with the next color in a perceptually-uniform palette (magma by default) and tagged with a numbered badge marking its visit order, and the caption accumulates the running output sequence. `step.kind` is `"init"` for the initial frame and `"visit"` thereafter, with `step.value` and `step.index` (1-indexed) recording each visit. Switch palettes via the theme system (see Theming) — either doc-wide with `set-op-theme((traversal-palette: color.map.viridis))` or per-call with `(t.in-order-display)(theme: (traversal-palette: ...))`.

Pure-data variants `(t.in-order)()`, `(t.pre-order)()`, `(t.post-order)()`, and `(t.level-order)()` return the visit sequence as an array of path strings without producing any animation, in case you want to drive a custom layout.

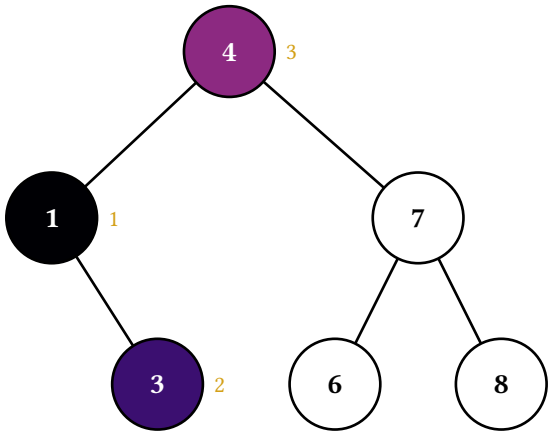




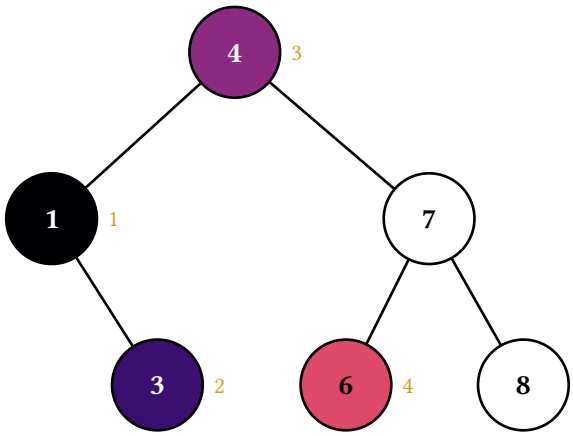
Output: `[1]`



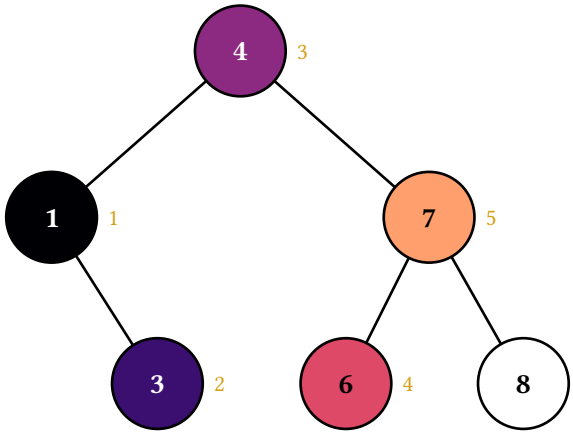
Output: `[1, 3]`



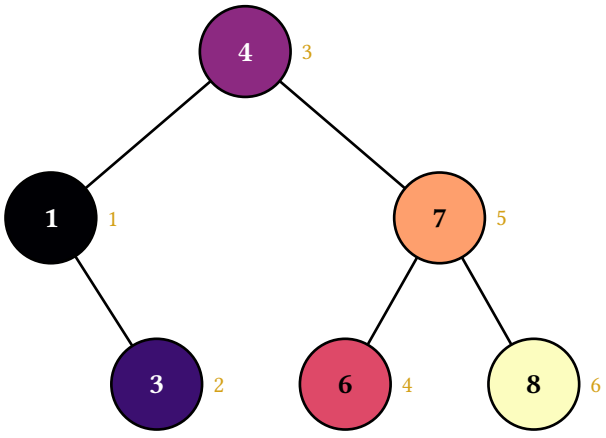
Output: `[1, 3, 4]`



Output: `[1, 3, 4, 6]`

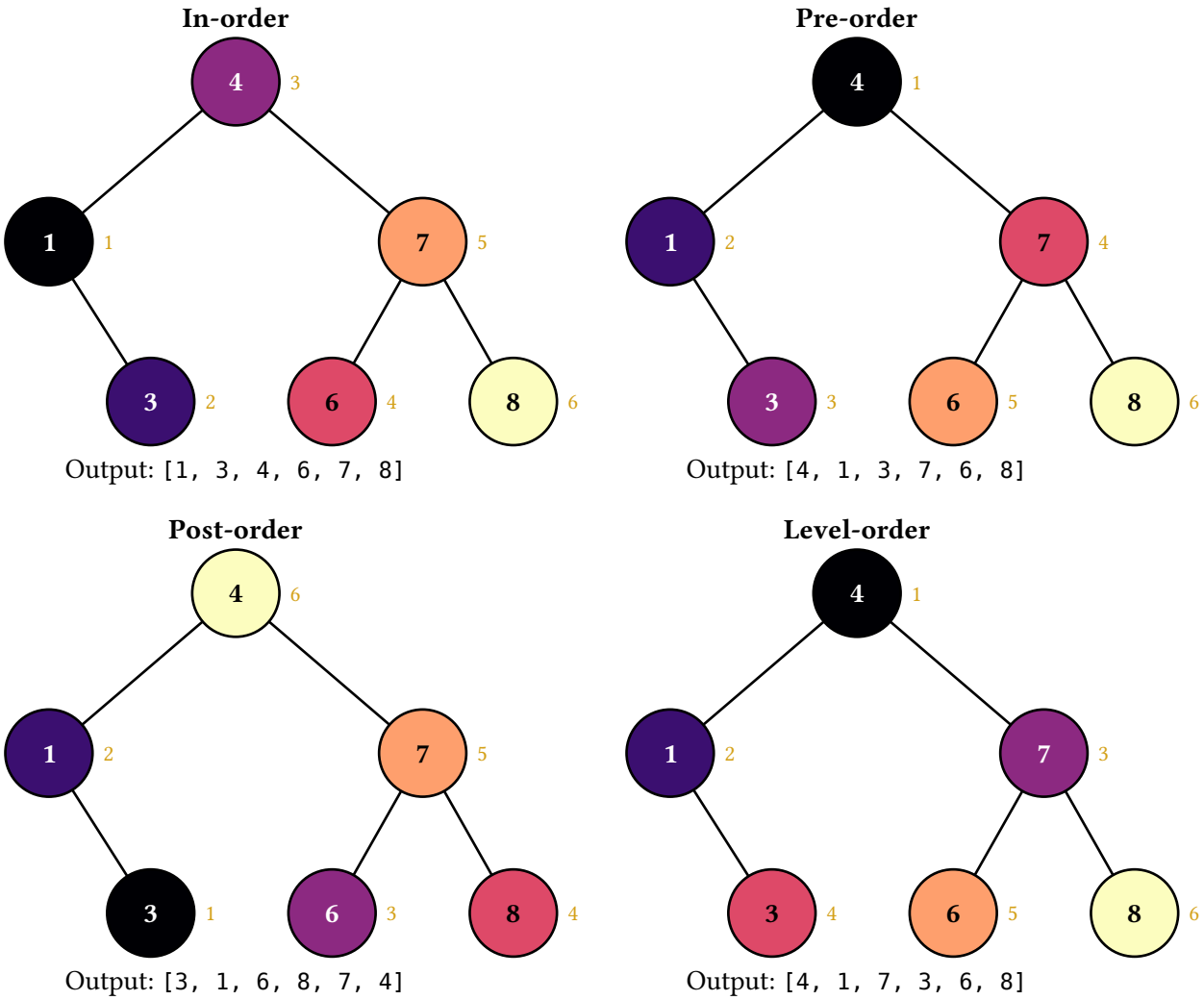


Output: `[1, 3, 4, 6, 7]`



Output: `[1, 3, 4, 6, 7, 8]`

The final frame of each traversal carries the algorithm's full color signature. Laying all four out side-by-side gives a fast visual comparison — especially useful as a recall cue in review material where students already understand the algorithms and the gradient pattern reads as "oh right, post-order goes leaves-cool to root-warm":



5. RBT animations tour

The `RBT` class ships three `*-display` methods today: `display`, `insert-display`, and `delete-display`. Each returns `Array(Frame)` in the same shape as the BST methods, so the render helpers (`last`, `stacked`, `figures`) and the caption / `step` / `alt` conventions carry over unchanged. The rbt-theme palette (red and black fill, stroke, and text-fill) is layered on top of the render theme so every frame's nodes wear their semantic colors automatically; operation-specific highlights from the op-theme (search-stroke on descent, attention-stroke on fix-up pivots, settled / success strokes on resolution) compose on top.

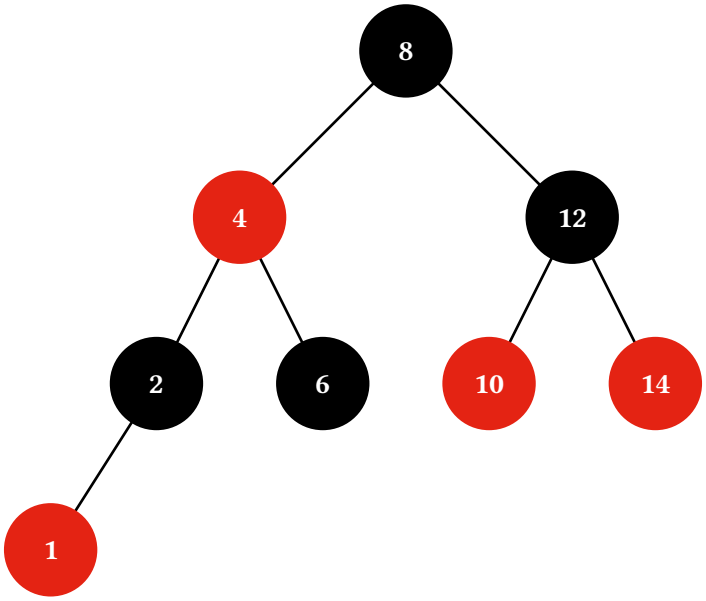
The sample tree throughout this section is seeded via the `rbt(..vals)` factory — first arg becomes the (black) root, the rest are inserted in order so the CLRS fix-ups produce a clean balanced shape with a mix of red and black nodes:

```
#let t = rbt(8, 4, 12, 2, 6, 10, 14, 1)
```

For finer control, the lower-level `(RBT.new)(value:, label:, red:, left:, right:)` constructor plus `(t.insert-many)(..vals)` method are still available — the root takes a `red:` argument because a single-node tree also carries a colour (it must be black after every public operation).

5.1. Static display

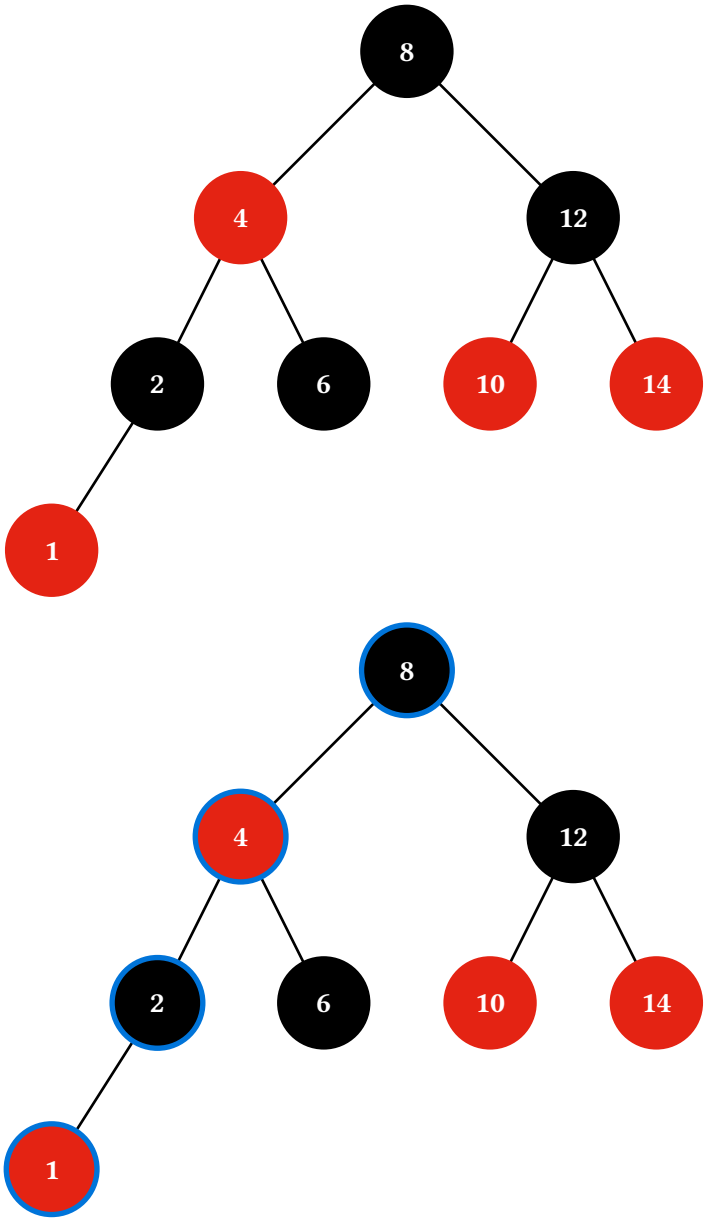
`(t.display)()` returns a one-element frame array — the unmodified tree, with every node coloured by its `red` field from the active rbt-theme palette.



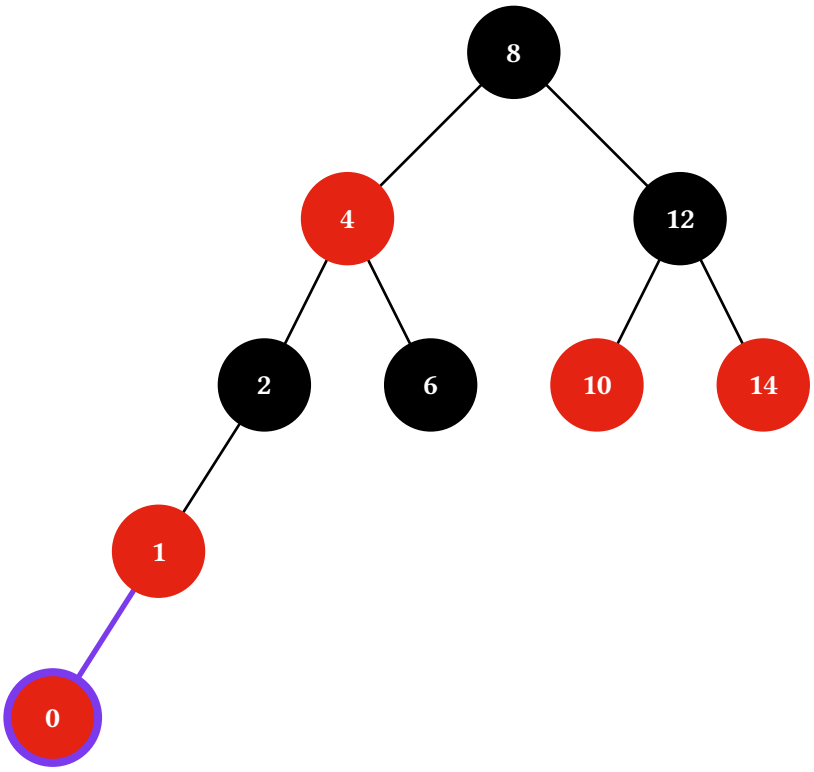
5.2. Insert

`(t.insert-display)(v)` traces the CLRS insertion algorithm: a BST descent records each visited node, the new value splices in as a red leaf, then a fix-up loop walks back up the tree applying Case 1 (uncle red — recolour parent and uncle black, grandparent red, continue at the grandparent), Case 2 (zigzag — rotate around the parent to straighten the red-red pair), and Case 3 (straight line — rotate around the grandparent and swap its colour with the new subtree root). The root is blackened at the end if it ended up red. `step.kind` ranges over `"init"`, `"descend"`, `"insert"`, `"check"`, `"recolor"`, `"rotate-zigzag"`, `"rotate-recolor"`, and `"blacken-root"`.

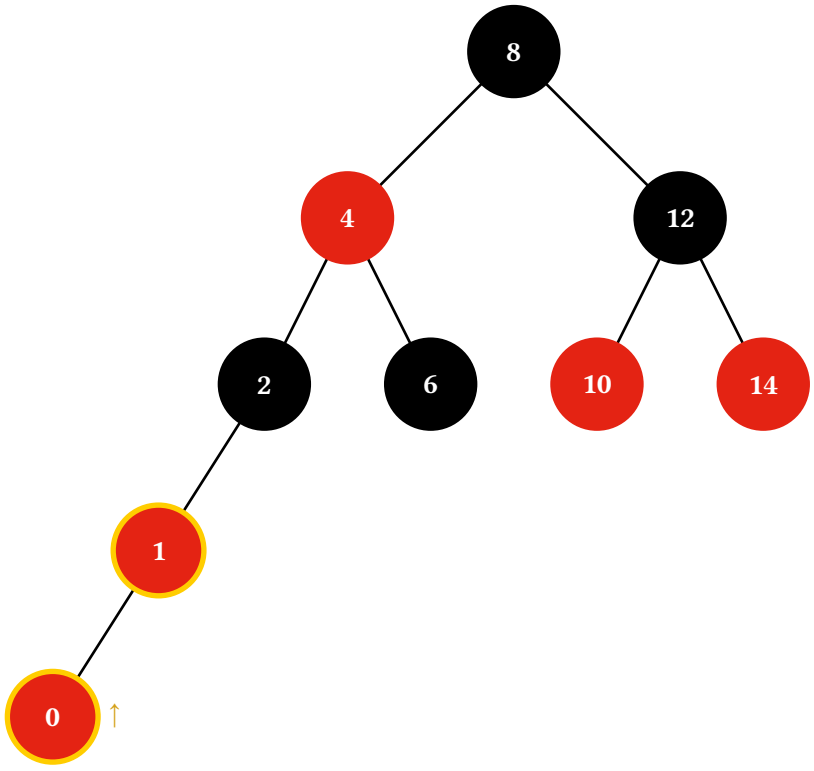
Inserting 0 walks down to the red leaf 1, splices a red 0 beneath it, then resolves the red-red pair via a straight-line Case 3 (rotate around the grandparent 2, swap its colour with the new subtree root 1):



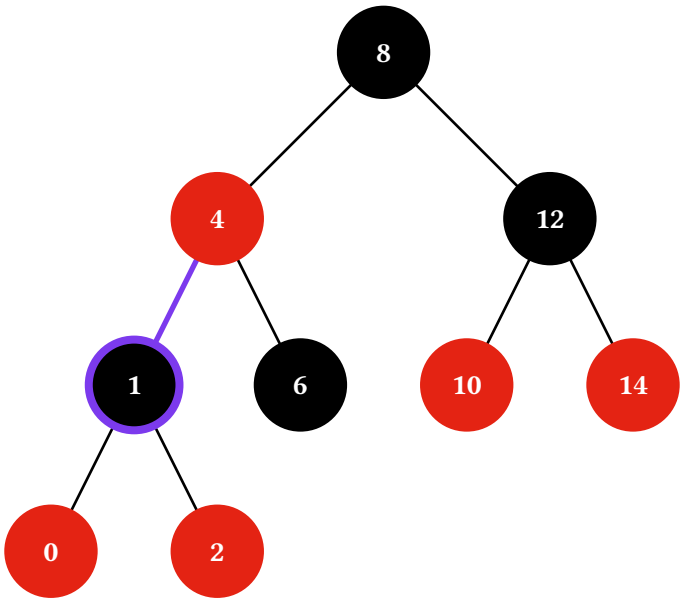
Search for 0



Insert 0 as red leaf



Red-red violation

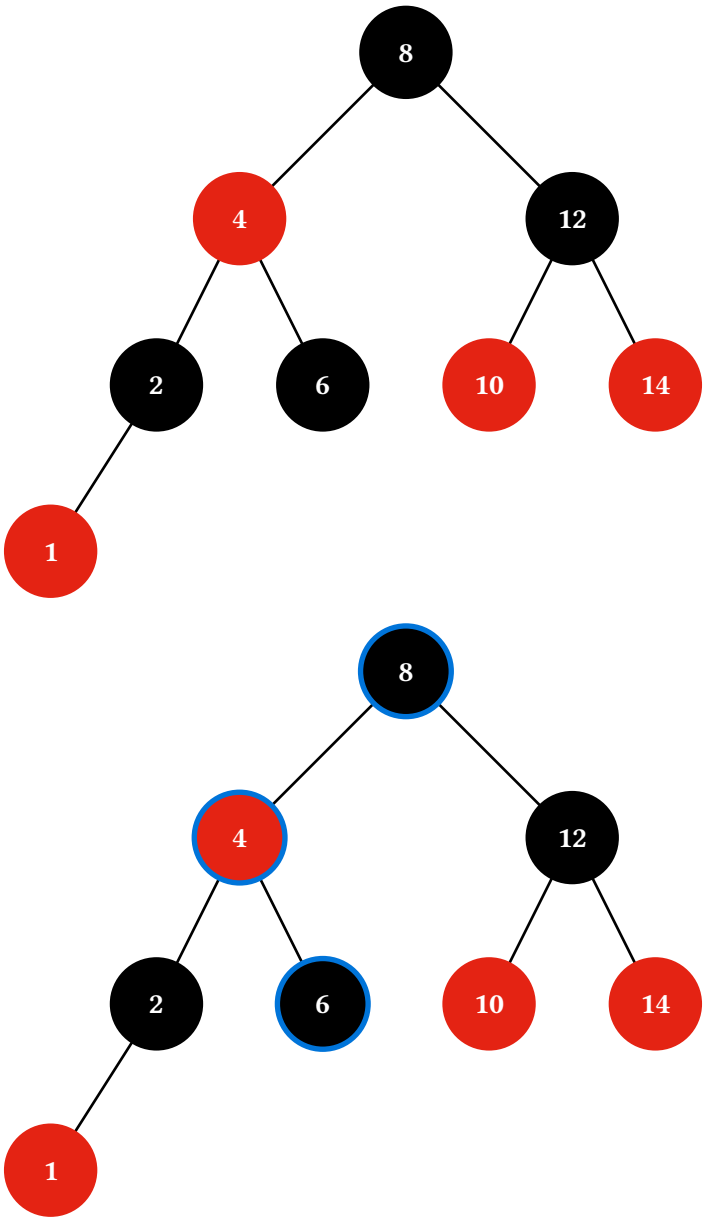


Rotate and color swap

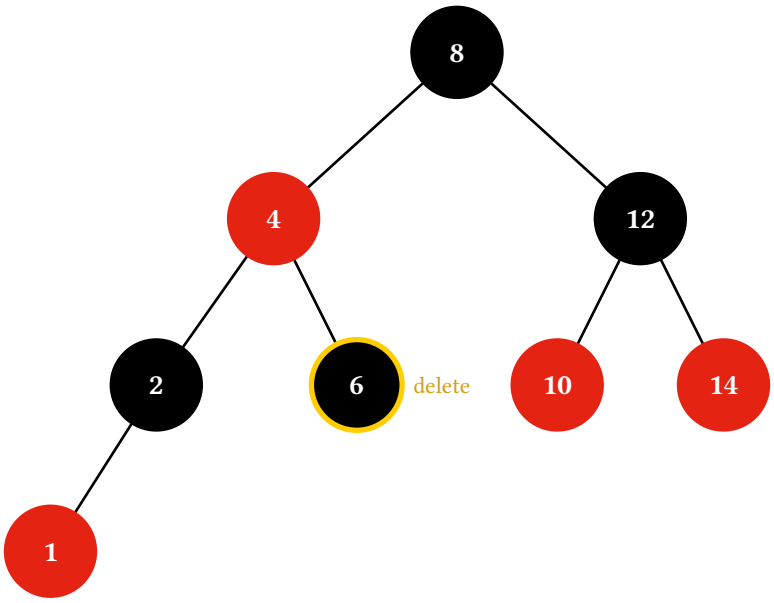
5.3. Delete

`(t.delete-display)(v)` traces a BST search for the target, the in-order predecessor walk if the target has two children, the value transfer, the structural excise, and the four-case rebalancing loop. `step.kind` covers `"init"`, `"descend"` (a single frame highlighting the full search path; pass `search: true` for one `"compare"` frame per step instead), `"not-found"`, `"mark-target"`, `"find-predecessor"`, `"transfer"`, `"excise"`, `"paint-black-promoted"`, `"paint-black-db"`, and the fix-up cases. Cases 1, 3, and 4 each emit two frames — a rotation-only intermediate (`"case-1-rotate"`, `"case-3-rotate"`, `"case-4-rotate"`) followed by the recolor (`"case-1"`, `"case-3"`, `"case-4"`) — so the structural pivot lands separately from the color swap. Case 2 is a pure recolor and emits a single `"case-2"` frame.

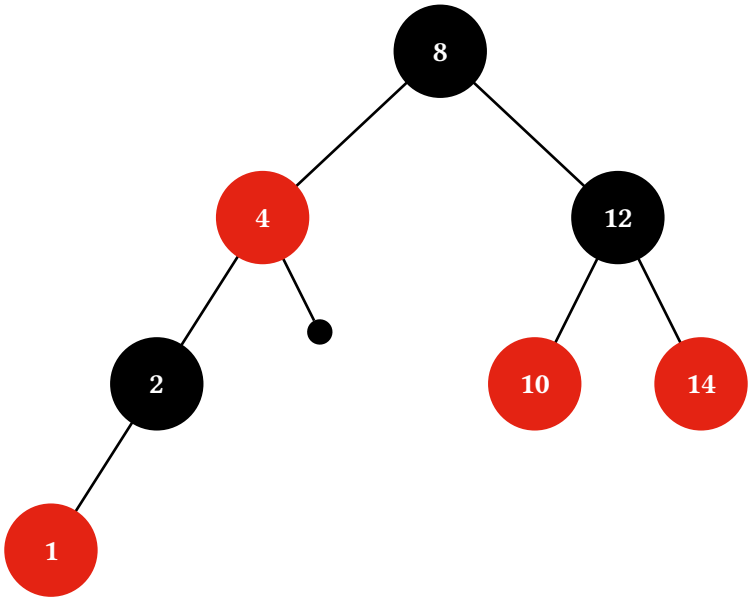
Excising a black node leaves the subtree one black short of the rest of the tree. The animation marks that imbalance with a small filled circle at the child end of the affected edge — the textbook convention. The circle stays in place across each fix-up step that doesn't resolve the missing black; it disappears once a Case 4 rotation drains it or a red ancestor absorbs it. Deleting the black leaf 6 takes the Case 4 branch directly (sibling 2 is black, far nephew 1 is red):



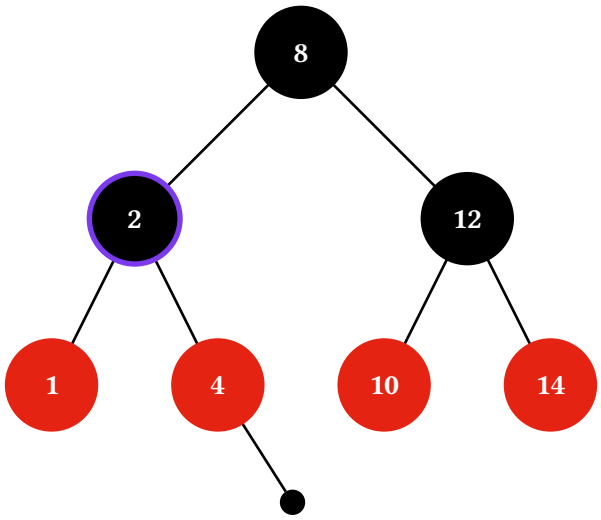
Search for 6



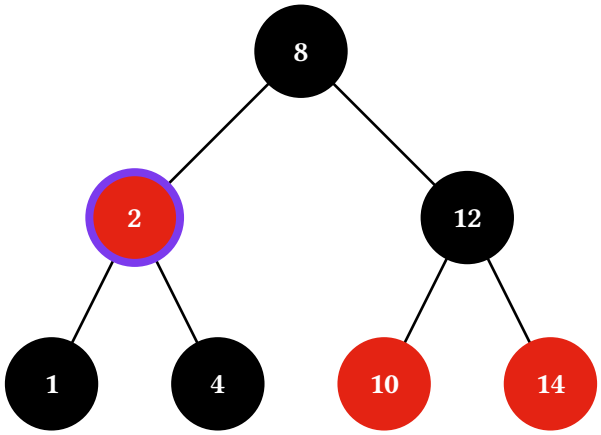
Delete 6



Remove node



Rotate parent



Recolor

5.4. Theming the red/black palette

The rbt-theme palette lives in its own state, separate from the render theme and op theme, with `default-rbt-theme` as the seed and `set-rbt-theme` / a per-call `theme:` argument as the two override paths. Recognised keys are `red-fill`, `red-stroke`, `red-text-fill`, `black-fill`, `black-stroke`, and `black-text-fill`; unknown keys panic so typos surface immediately.

Set the palette document-wide via `set-rbt-theme` (state-based, participates in the layout-pass cost — see Section 10.3):

```
#import "@preview/starling:0.2.0": RBT, set-rbt-theme

#set-rbt-theme((
  red-fill: orange,
  red-stroke: orange.darken(20%),
  black-fill: navy,
```

```
  black-stroke: navy,
))
```

Or per-call to skip state and run at the no-state baseline:

```
(t.display)(theme: (red-fill: red.darken(20%)))
```

6. AVL animations tour

The `AVL` class is a height-balanced BST. Each node carries an extra `height` field; after every public operation the height is up-to-date and `|h(left) - h(right)|` (the **balance factor**) is at most 1. Imbalances are repaired with single or double rotations, dispatched on four cases (LL, RR, LR, RL) — the same ones every textbook walks through.

`AVL` ships the standard suite of `*-display` methods (`display`, `search-display`, `insert-display`, `delete-display`, `rotate-display`, `fixup-display`, and the four `*-order-display` traversals), all returning `Array(Frame)` so the render helpers and caption / `step` / `alt` conventions carry across unchanged. AVL has no per-DS theme — animation strokes come from the op-theme, structural defaults from the render theme.

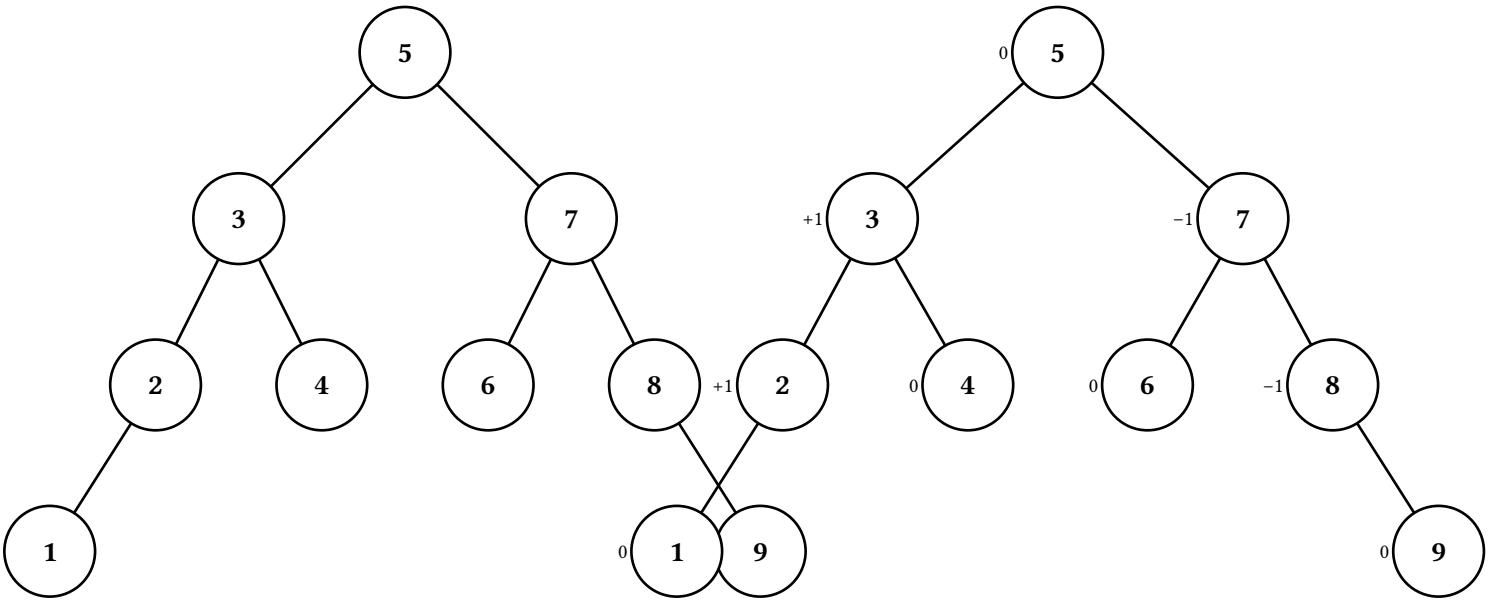
The sample tree throughout this section is seeded via the `avl(..vals)` factory — first arg becomes the root, the rest are inserted in order so the AVL rebalancing produces a clean balanced shape:

```
#let t = avl(5, 3, 7, 2, 4, 6, 8, 1, 9)
```

For finer control, the lower-level `(AVL.new)(value:, label:, height:, left:, right:)` constructor plus `(t.insert-many)(..vals)` method are still available — the root takes a `height:` argument because a single-node tree already has height 1.

6.1. Static display

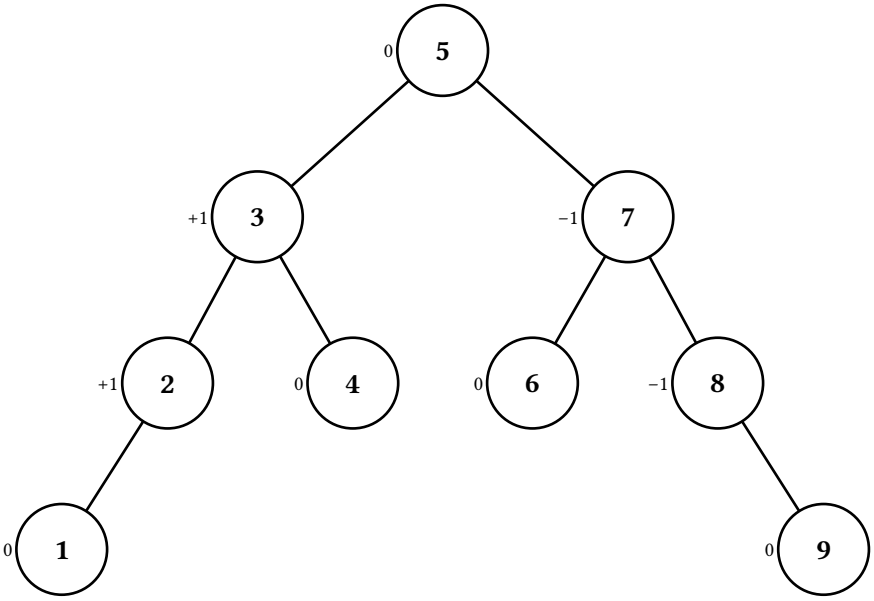
`(t.display)()` returns a one-element frame array — the unmodified tree. Pass `factors: true` to tag every node with its signed balance factor (e.g. `"+1"`, `"0"`, `"-1"`), drawn just west of the node so it doesn't compete with the label or the operation `note` slot. The tag layer is reused by every `*-display` method as a base layer (off by default; opt in per call).

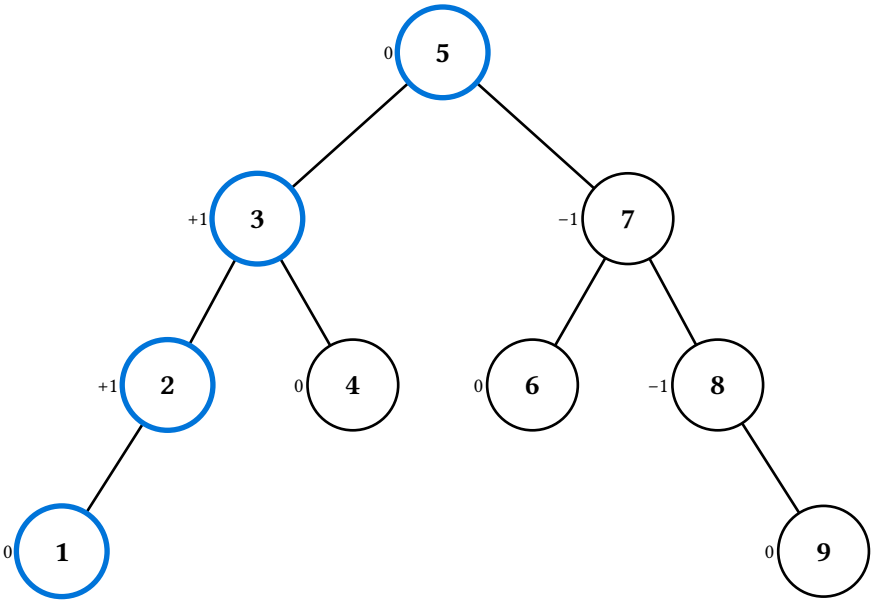


6.2. Insert

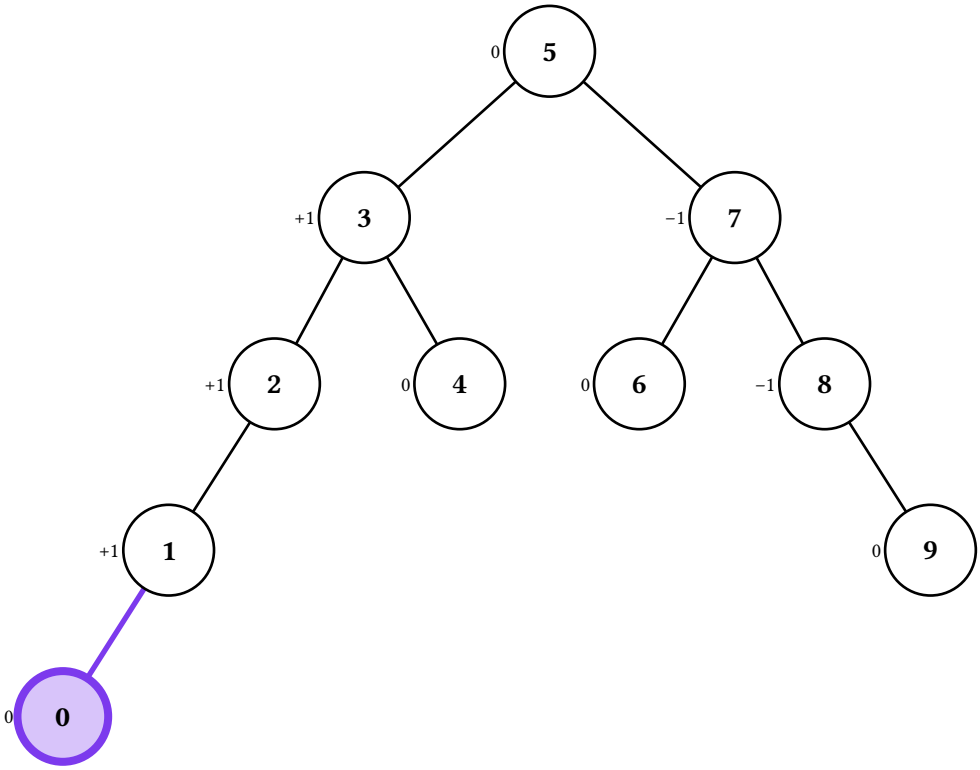
`(t.insert-display)(v)` traces the AVL insertion algorithm. A BST descent records each visited node, the new leaf appears with height 1, then the climb back to the root recomputes each ancestor's height in turn. On the first ancestor whose balance factor reaches ±2, the animation labels the imbalance case (LL/LR/RR/RL), applies a child rotation if the case is a zigzag (LR or RL), and finishes with the single rotation at the imbalanced node. After a single insertion the subtree's height returns to its pre-insert value, so the climb stops there. `step.kind` ranges over `"init"`, `"descend"`, `"insert"`, `"recompute"`, `"check"`, `"rotate-zigzag"`, and `"rotate-finish"`.

Inserting 0 into the sample tree triggers an LL fix-up at node 2 (after the climb sees node 1′s height grow). The fix-up is a single right rotation:

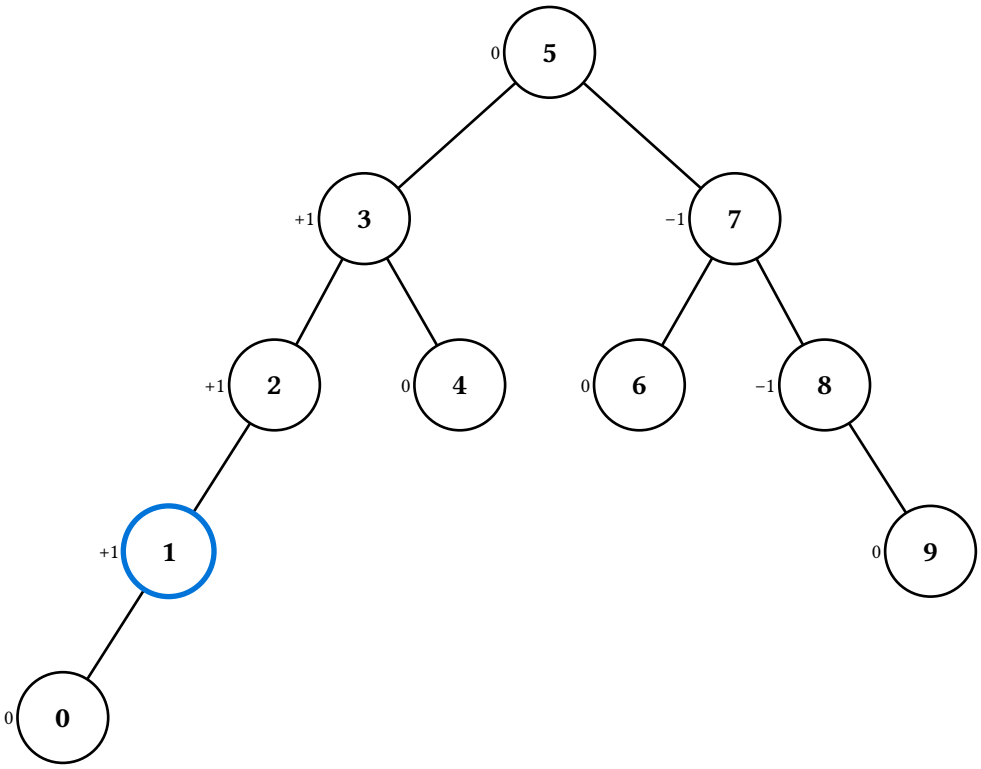




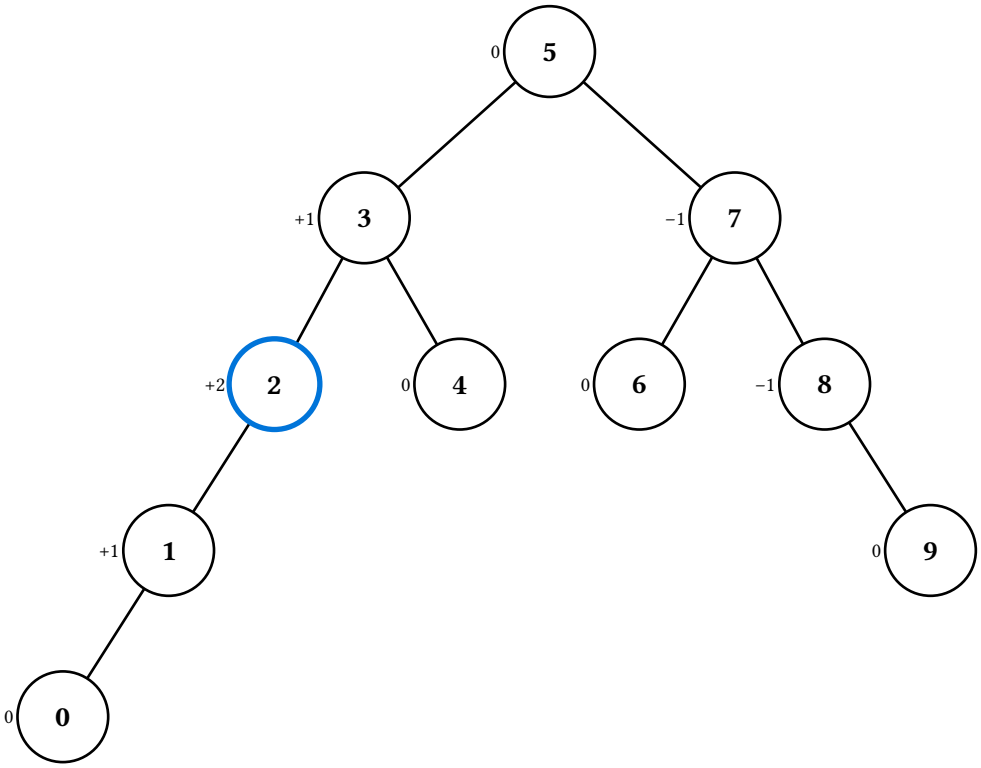
Search for 0



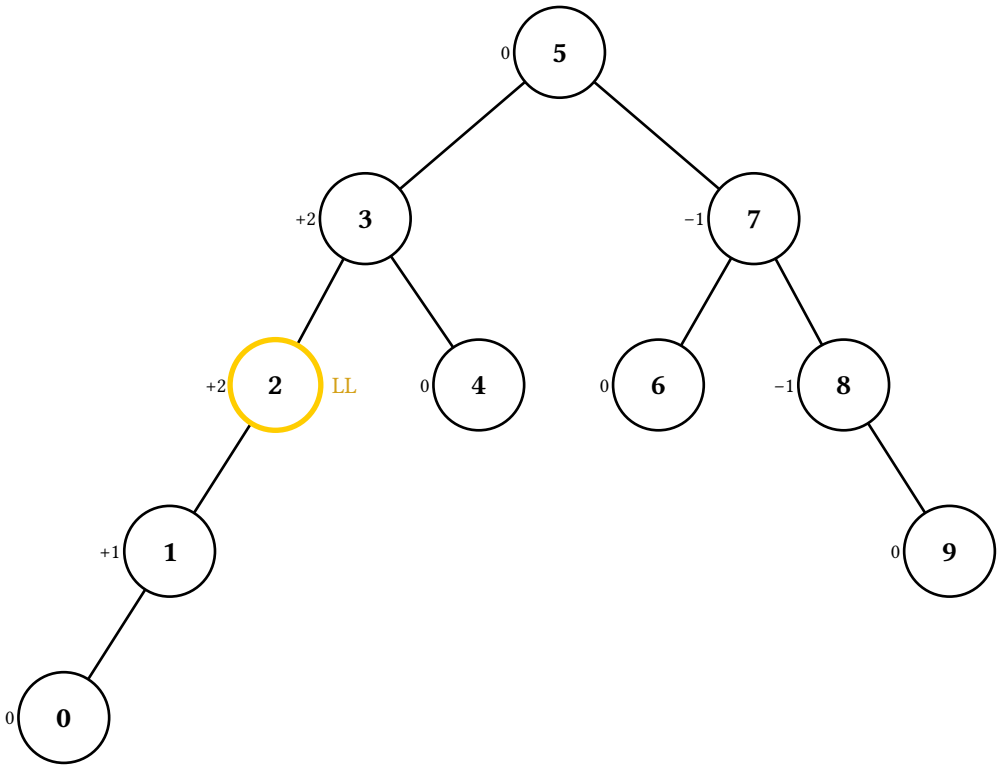
Insert 0 as new leaf



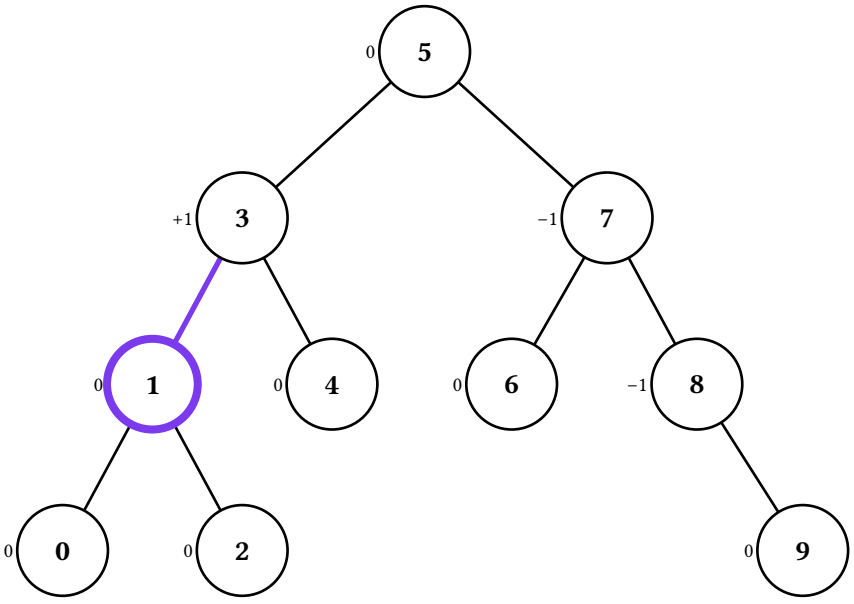
Recompute height



Recompute height



Imbalance: case LL



Rotate (case LL)

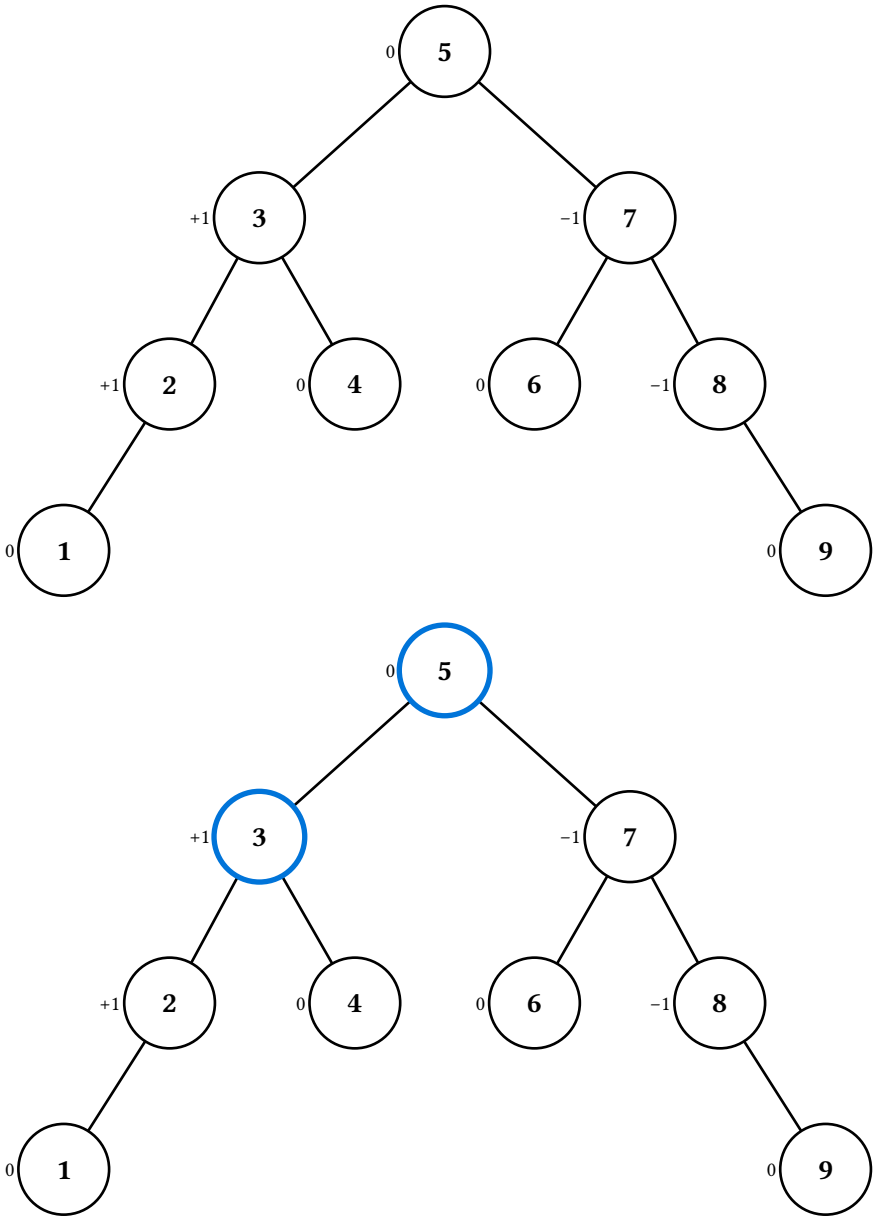
The zigzag cases (LR/RL) emit an extra `rotate-zigzag` frame between `check` and `rotate-finish` for the child rotation that straightens the configuration before the final rotation at the imbalanced node.

6.3. Delete

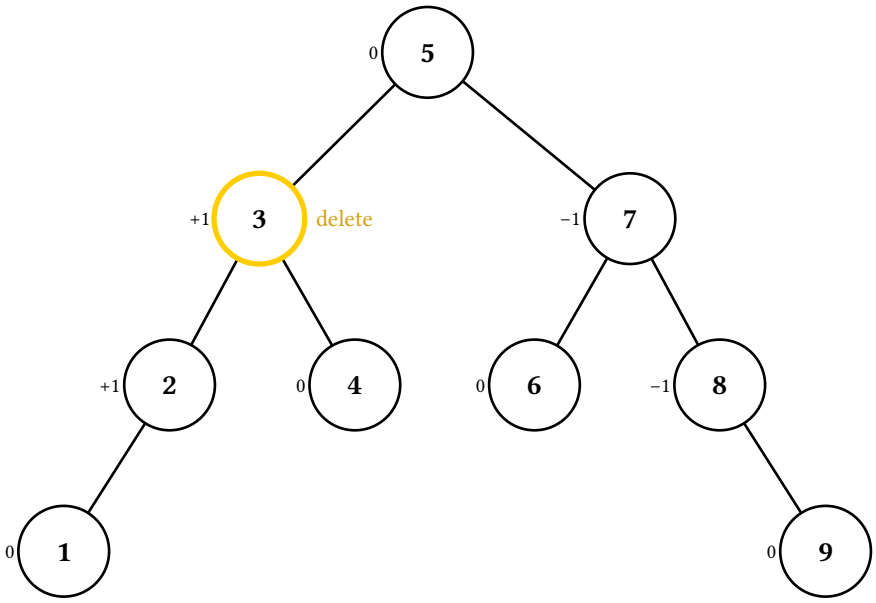
`(t.delete-display)(v)` traces a BST search for the target, the in-order predecessor walk if the target has two children, the value transfer, the structural excise, and then a climb that recomputes heights and rotates at every imbalanced ancestor. Unlike insert, a single deletion can trigger several rotations on the way up

— the loop runs to the root. `step.kind` covers `"init"`, `"descend"` (single frame; pass `search: true` for one `"compare"` per step instead), `"not-found"`, `"mark-target"`, `"find-predecessor"`, `"transfer"`, `"excise"`, and the same `"recompute"` / `"check"` / `"rotate-zigzag"` / `"rotate-finish"` events insert emits.

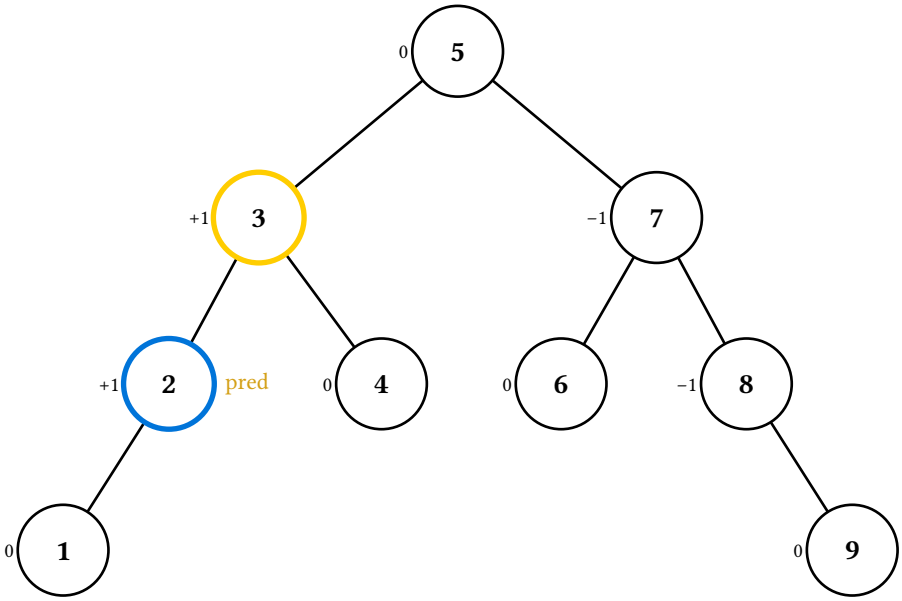
Deleting 3 from the sample tree takes the two-child branch (predecessor transfer from 2) and stays balanced — the climb recomputes heights without triggering any rotation:



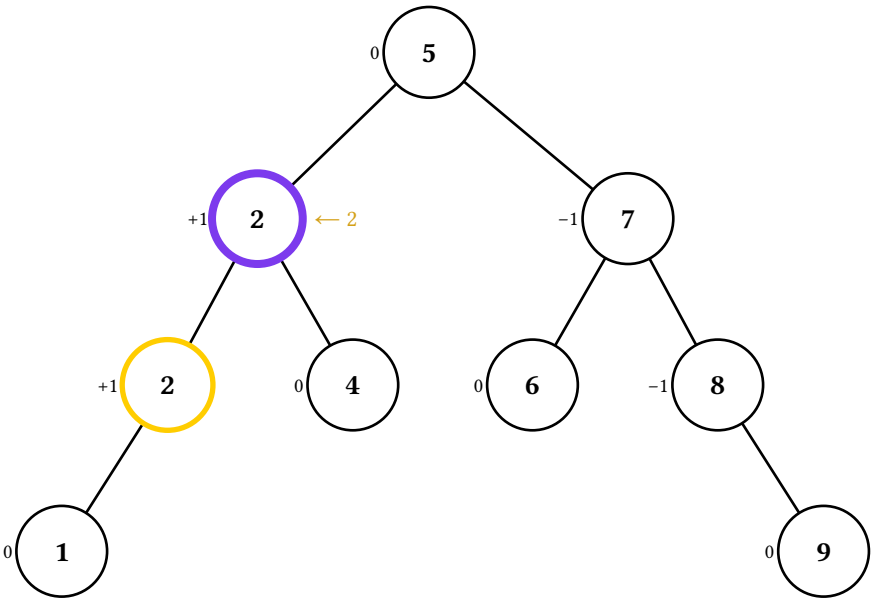
Search for 3



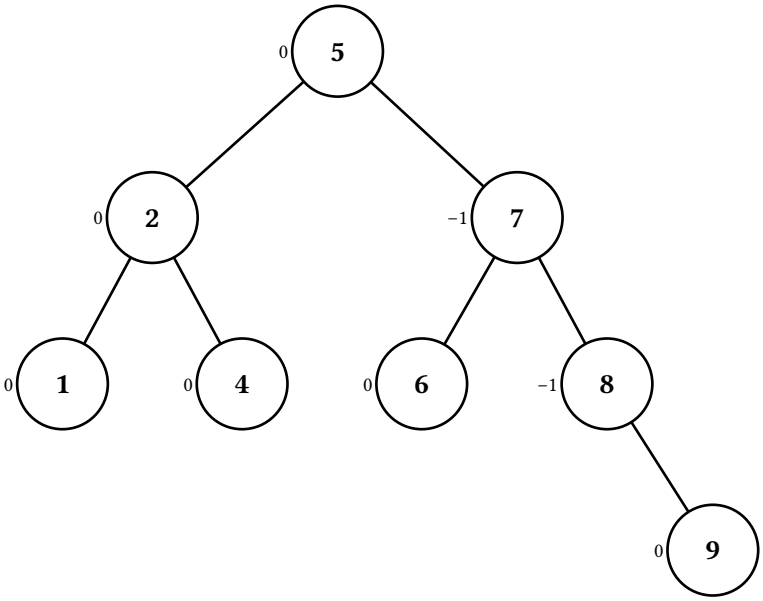
Delete 3



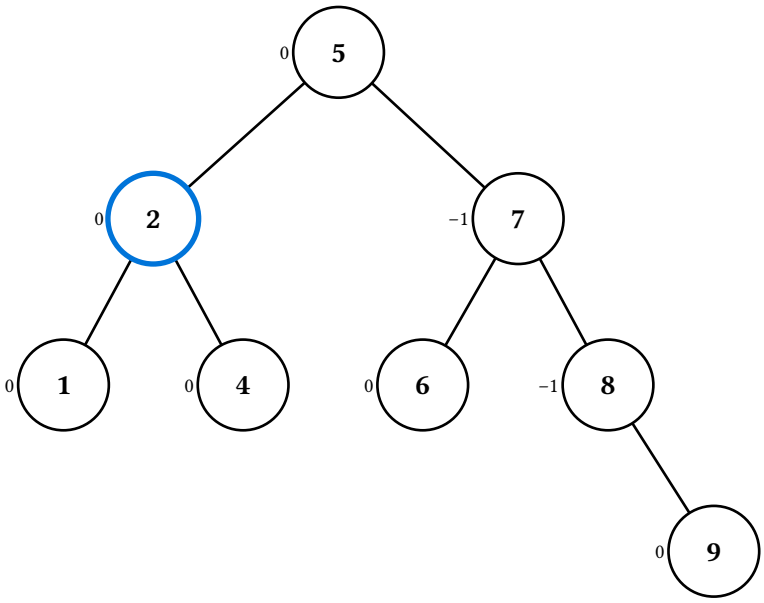
Find predecessor



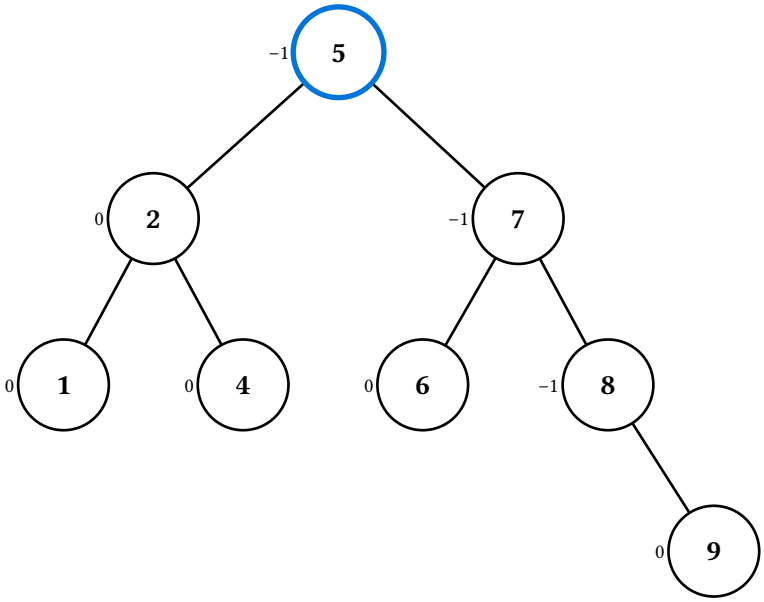
Transfer 2



Remove node



Recompute height



Recompute height

6.4. Rotate

`(t.rotate-display)(c)` is the same six-frame structural rotation animation BST exposes, with heights refreshed in the after-tree. AVL rebalancing in `insert` / `delete` does **not** go through this method — the case-dispatch rotations are applied directly so the animation can narrate the recompute-then-rotate logic.

6.5. Fix-up

`(t.fixup-display)(violation-path)` runs the AVL fix-up climb from a hand-constructed (possibly invalid) tree. The tree is **not** validated: this is intended for teaching configurations that can't arise from a single insert or delete — for example, multiple imbalances on one spine where the inner rotation propagates the imbalance up. The climb walks every proper prefix of `violation-path` deepest-first, emitting the same `recompute` / `check` / `rotate-zigzag` / `rotate-finish` events that insert and delete use.

6.6. Traversals

`(t.in-order-display)()`, `(t.pre-order-display)()`, `(t.post-order-display)()`, and `(t.level-order-display)()` work exactly like their BST counterparts — one frame per visit, gradient fill from the op-theme's `traversal-palette`, running output sequence in the caption. Pass `factors: true` on any traversal to layer the balance-factor tags under the per-visit highlights.

7. B24 animations tour

The `B24` class is a 2-3-4 tree (a B-tree of order 4) — every internal node holds 1, 2, or 3 keys (so 2, 3, or 4 children) and every leaf sits at the same depth. The node renderer for this data structure is the subdivided rectangle shape `"btree-node"`, which scales its width with the number of keys; per-key compartments are individually addressable via the `"<path>#<i>"` path syntax and the `key-styles` slot on `NodeStyle`.

`B24` exposes the standard suite of `*-display` methods (`display`, `search-display`, `insert-display`, `delete-display`, and the four `*-order-display` traversals). Insert and delete accept a `strategy:` argument that switches between two textbook algorithms:

- **Top-down** (the default): preventive — split any 3-key node on the way down for insert; refill any 1-key node on the way down for delete. Single-pass recursion.
- **Bottom-up**: reactive — walk to the leaf first, then propagate splits or merges back up through the descent path.

Both produce valid 2-3-4 trees containing the same keys but they can produce structurally different **shapes** — bottom-up promotes a key from the **post-overflow** 4-key state (so the freshly inserted key may itself be promoted), while top-down promotes the middle of the **pre-insert** 3-key state. There is no per-DS theme — animation strokes come from the op-theme; structural defaults from the render theme.

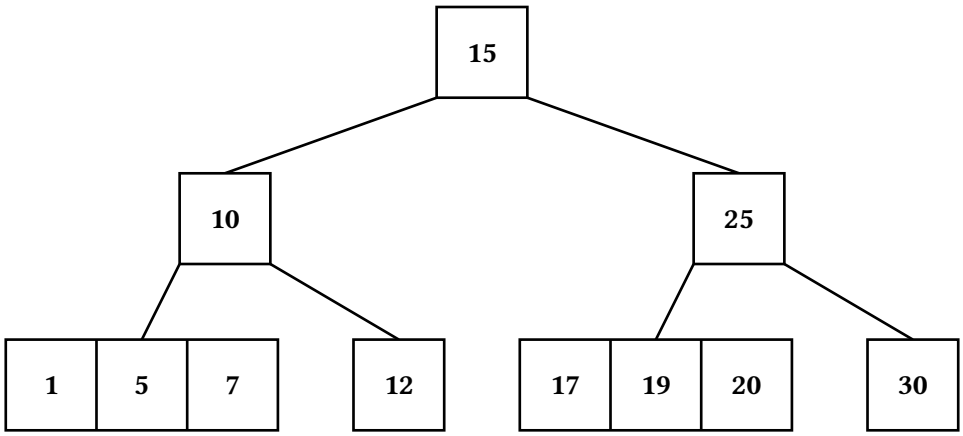
The sample tree throughout this section is seeded via the `b24(..vals)` factory:

```
#let t = b24(10, 5, 15, 1, 7, 12, 20, 25, 30, 17, 19)
```

For finer control, the lower-level `(B24.new)(keys:, labels:, children:)` constructor lets you hand-build a tree with any compartment count at every depth — useful for fixtures that demonstrate a specific fix-up case.

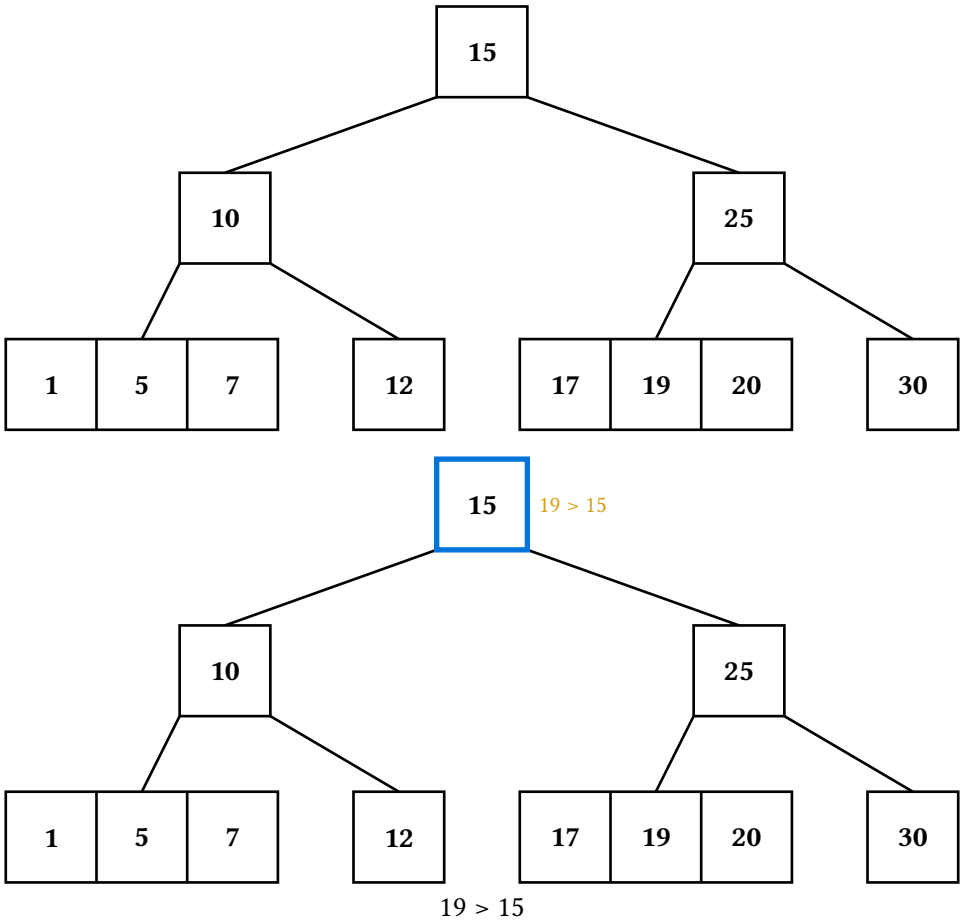
7.1. Static display

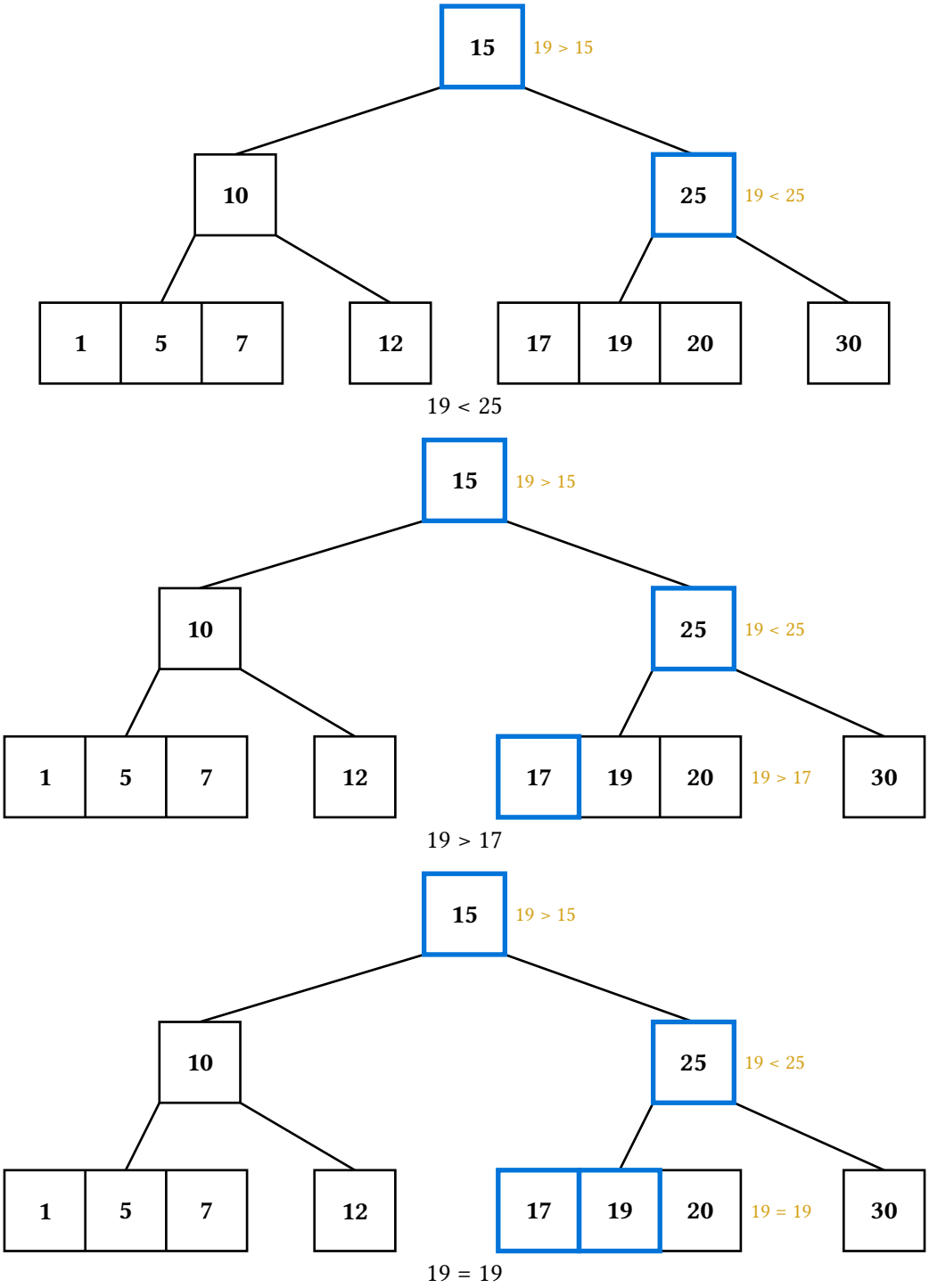
`(t.display)()` returns a one-frame array — the unmodified tree. Compartment widths scale with key count, and edges fan out from the gap anchors (`gap-0` through `gap-k`) on the parent's south face.



7.2. Search

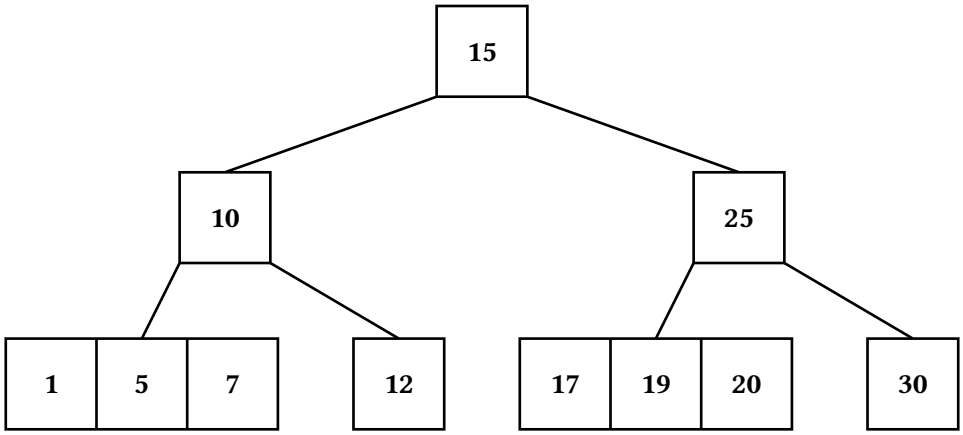
`(t.search-display)(v)` highlights one key compartment per comparison. The descent path stays visible as the search progresses (sticky search strokes); the inline note at each visited node carries the comparison text. Misses end at the leaf without panicking — the final frame reports that the value isn't in the tree.

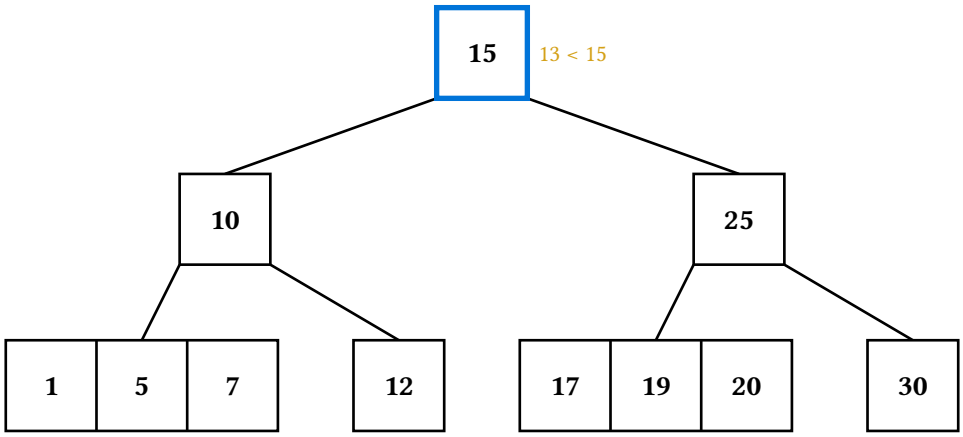




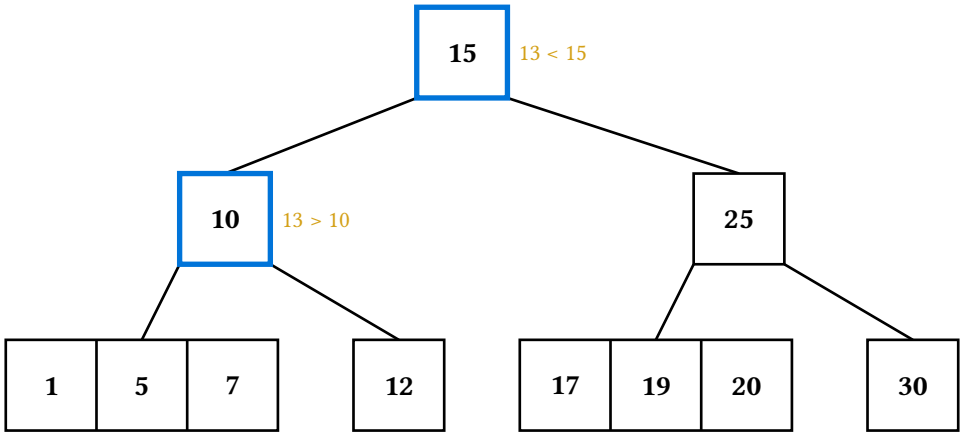
7.3. Insert

`(t.insert-display)(v, strategy: ..)` traces the full insertion algorithm. Top-down splits trigger a `td-pre-split-attention` frame that outlines a full node about to be split, then a `split-done` frame showing the promoted key with its two new child edges. The final `settled` frame highlights the inserted compartment.

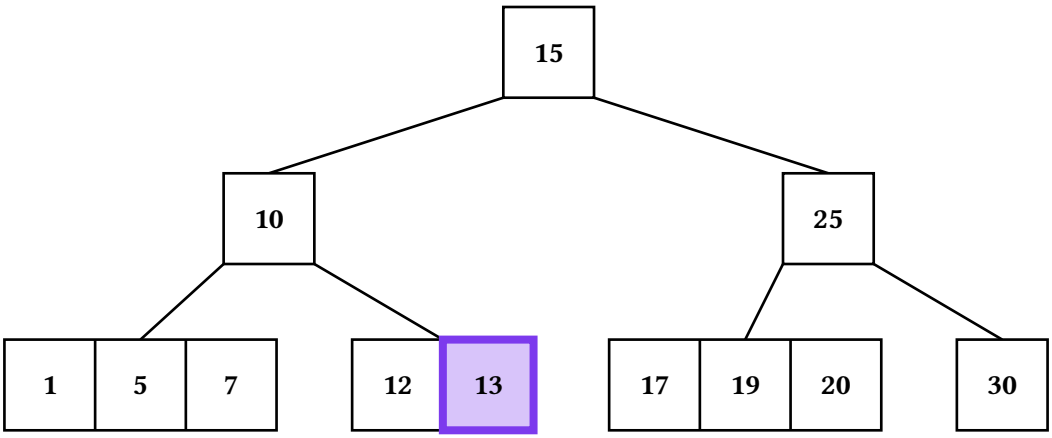

15
10
25
1
5
7
12
17
19
20
30


15
13 < 15
10
25
1
5
7
12
17
19
20
30

13 < 15

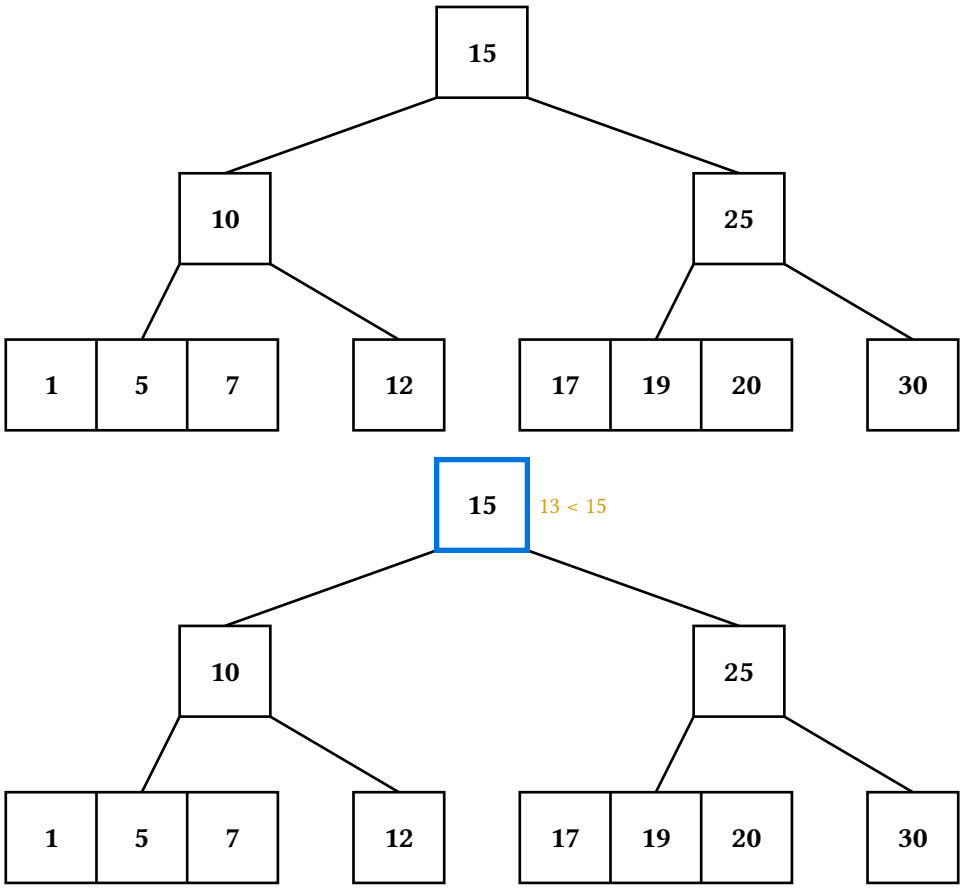

15
13 < 15
10
13 > 10
25
1
5
7
12
17
19
20
30

13 > 10

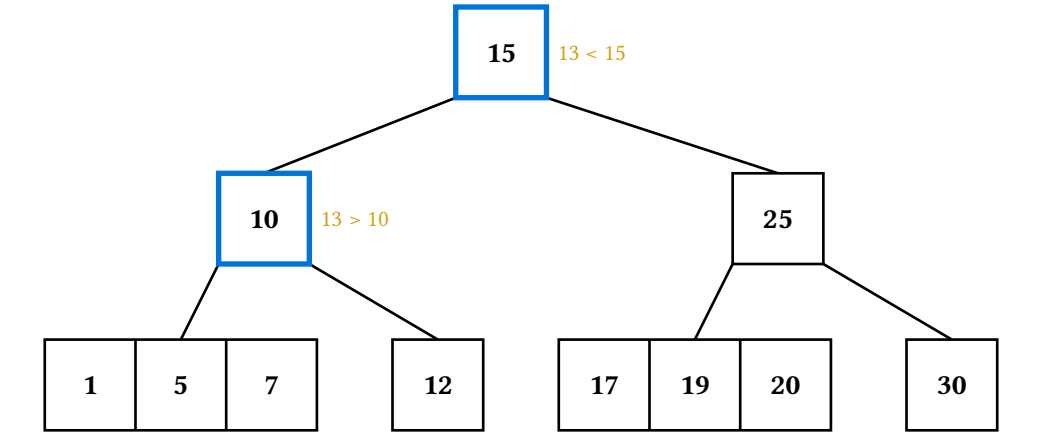


Inserted 13

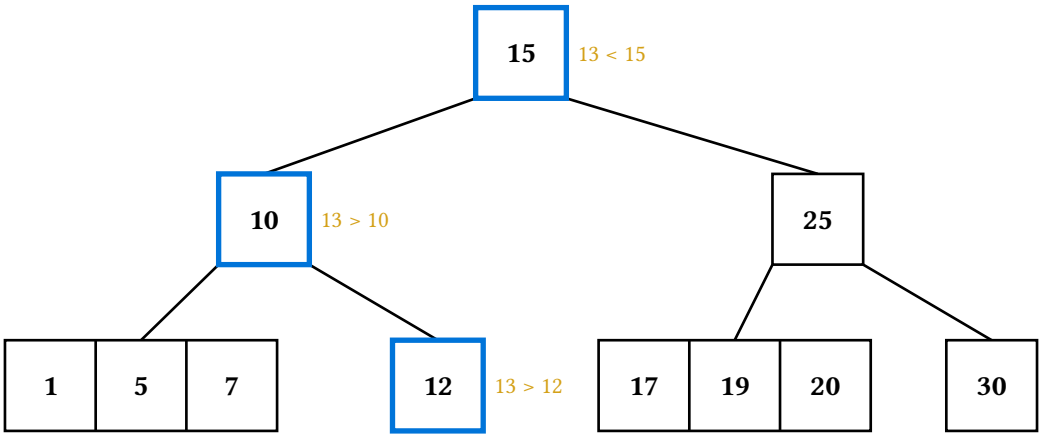
The bottom-up variant runs the descent first, then animates the leaf insertion plus any cascading splits:



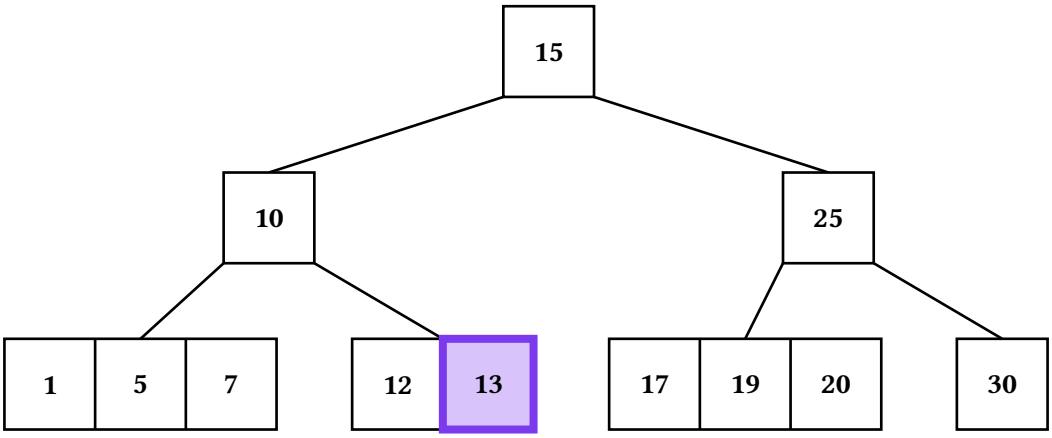
13 < 15



13 > 10



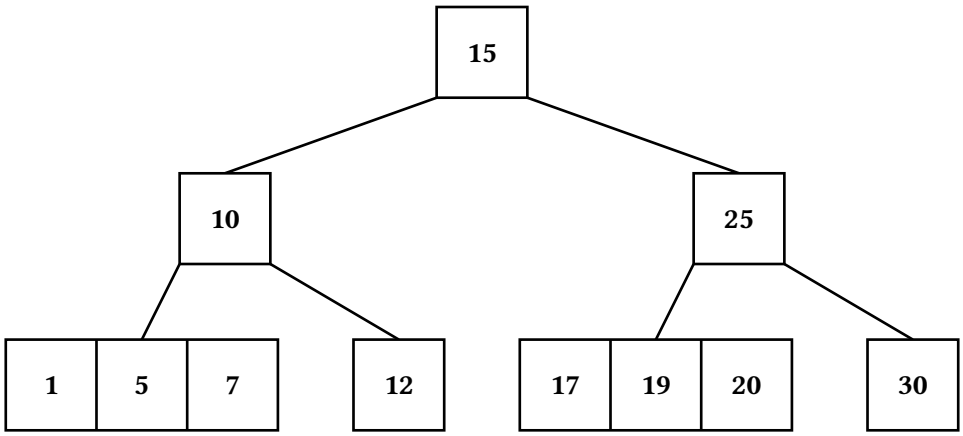
13 > 12

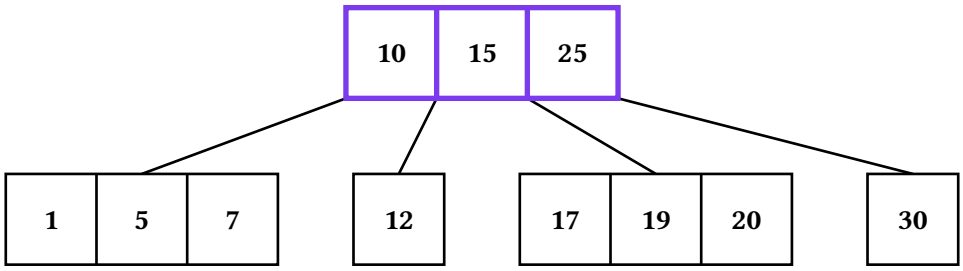


Inserted 13

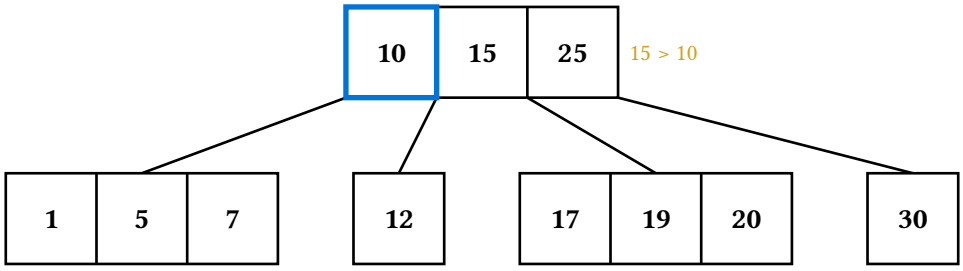
7.4. Delete

`(t.delete-display)(v, strategy: ..)` traces the deletion algorithm. Top-down's preventive `td-pre-fix-attention` frame outlines a 1-key descent target before the fix; subsequent `td-borrow-left` / `td-borrow-right` / `td-merge` frames carry out the rotation or merge. Internal-key deletions visit a `td-target` highlight, swap with the predecessor (`td-pred-swap`), then continue the descent to remove the predecessor's old value from its leaf.

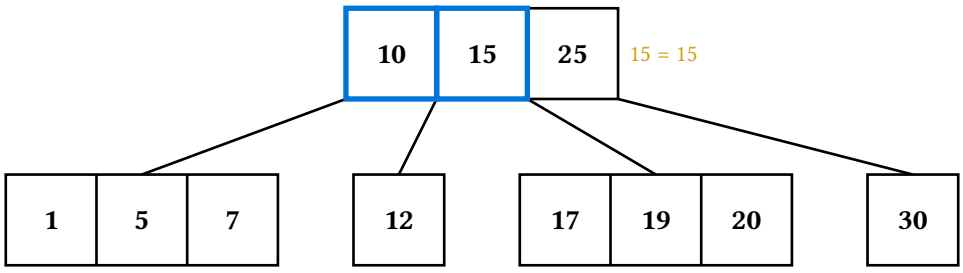




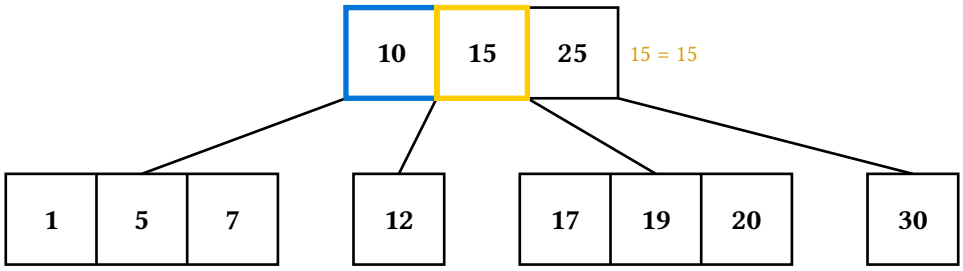
Merge



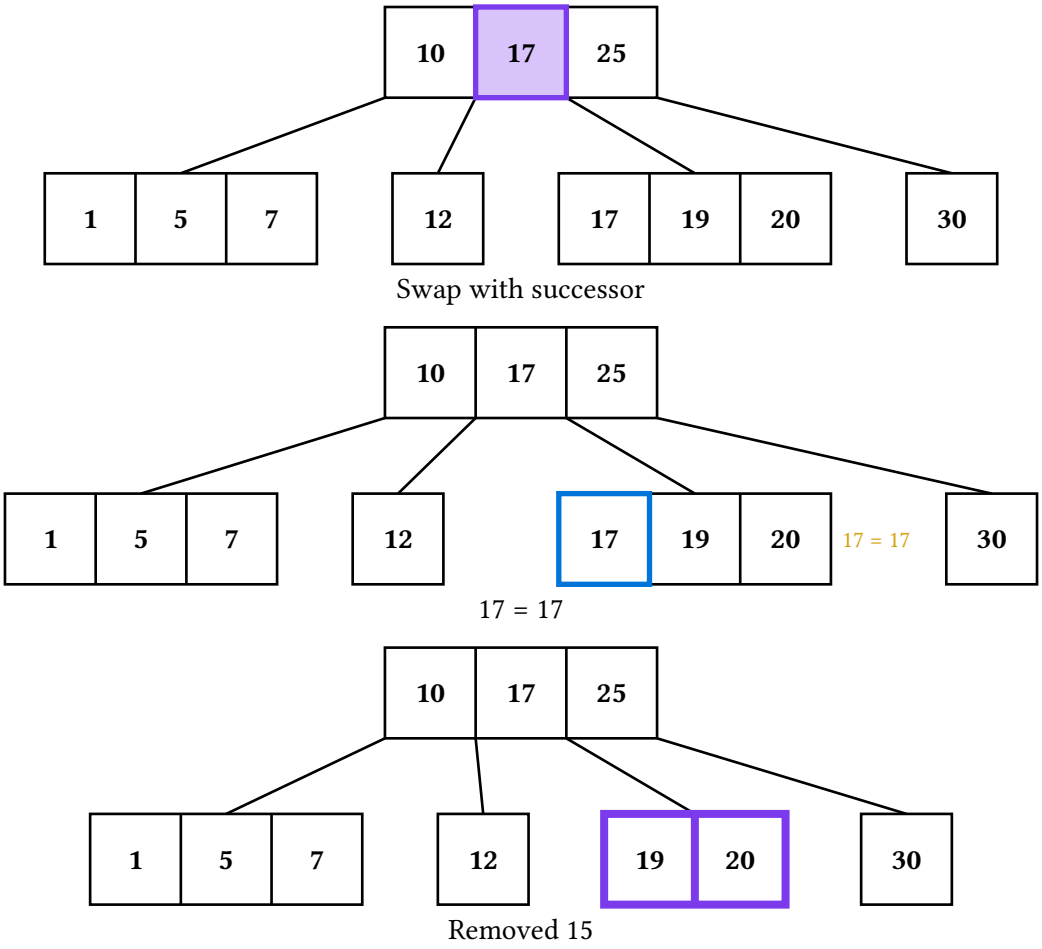
15 > 10



15 = 15

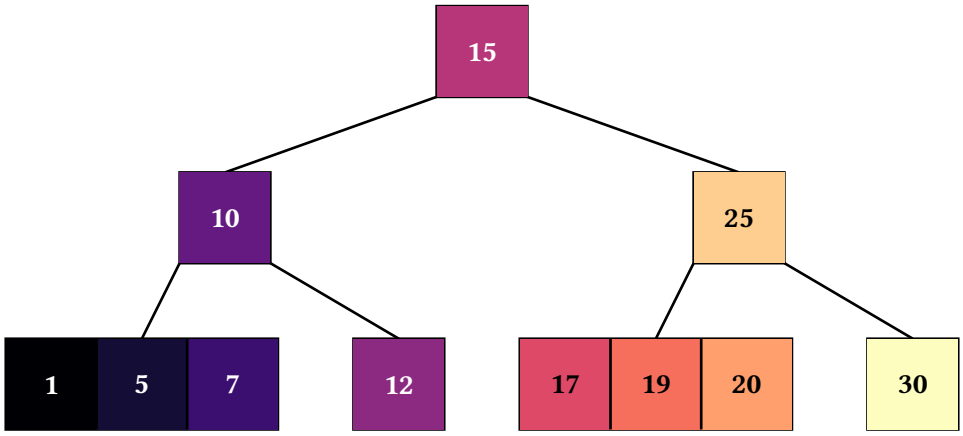


Target found



7.5. Traversals

The four traversal animations sample the op-theme's `traversal-palette` across the per-compartment visit order — each visited compartment gets a fill from the gradient and the caption accumulates the output sequence. In-order on a 2-3-4 tree threads keys with their flanking subtrees (child[0], key[0], child[1], key[1], …, child[k]); pre- and post-order group all of a node's keys at the node-visit point.



8. Graph animations tour

The `Graph` class is the first non-tree structure in starling: an undirected-or-directed weighted graph for teaching minimum spanning trees (Prim and Kruskal), Dijkstra's shortest paths, and breadth- and depth-first traversals. It rides the same `Frame` / op-theme / render-theme stack as the trees, so every render helper, theming knob, and touying composition works unchanged. There is no per-DS theme — animation strokes come from the op-theme, structural defaults from the render theme.

Unlike trees, graphs have no root and no path-from-root, so node identity is a plain string id and edges are keyed by `edge-key(u, v, directed:)` (sorted `"u--v"` when undirected, `"u->v"` when directed). The renderer (`graph-draw.typ`, the `draw-graph` backend) is split from the `Graph` class (`graph.typ`, data plus algorithms) exactly as `draw-tree` is split from the tree classes — both inject their backend into the shared `Renderer` in `anim-core.typ`.

8.1. Construction and layout

Layout is decoupled from rendering: the `Graph` carries node positions (in cetz units) and the renderer simply draws nodes there. The `graph(..)` factory takes placed nodes and weighted edges:

```
#let g = graph(
  (("A", 0, 0), ("B", 3, 0.4), ("C", 1.5, 2.4), ("D", 4.5, 2.2)),
  edges: (("A", "B", 7), ("B", "C", 2), ("A", "C", 4),
          ("C", "D", 5), ("B", "D", 1)),
)
```

Both nodes and edges accept optional custom **labels** — text drawn in place of the id (for a node) or the weight (for an edge). A node spec is a bare id, an `(id, label)` 2-tuple (label, no manual position — pairs with `auto-layout`), an `(id, x, y)` tuple, or an `(id, x, y, label)` tuple. An edge's third slot dispatches by type: a **number** is the weight, a **string or content** is a label drawn in its place; an `(u, v, weight, label)` 4-tuple sets both — the numeric weight still drives Dijkstra and the MSTs while the label is what's shown.

```
#let g = graph(
  ("A", ("B", [Server]), ("C", 1.5, 2.4)),  // B is auto-laid-out
  edges: (
    ("A", "B", 7),          // weight 7
    ("B", "C", [TLS]),       // label "TLS", no weight shown
    ("A", "C", 4, [4 ms]),   // weight 4 for algorithms, shown as "4 ms"
  ),
)
```

Hand-placement is the first-class path — small teaching graphs read best when you control the layout. For larger graphs, the optional `auto-layout` helper (below) computes positions with graphviz.

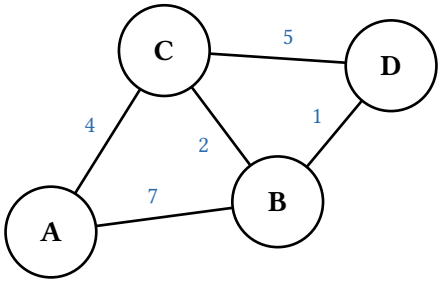
Every display method takes a `scale:` factor (default `1`) that multiplies all node coordinates about the origin, spreading the nodes apart while their drawn size — circles, labels, edge weights — stays fixed. It's the manual-layout analog of `auto-layout`'s `unit:`: reach for it when a hand-placed graph is too cramped (e.g. on a wide slide) without wanting to rescale the whole figure or edit every coordinate.

```
#last((g.display)(scale: 1.6))           // 1.6× the gaps, same node size
#last((g.dijkstra-display)("A", scale: 1.4))
```

To instead resize the **whole** figure — nodes, spacing, and text together — wrap the result in Typst's `scale`: `#scale(140%, reflow: true, last((g.display)()))`.

8.2. Static display

`(g.display)()` returns a one-frame array. Edge weights are drawn at each edge midpoint; a directed graph gets arrowheads.



8.3. Tabular representations

Besides the drawn graph, `(g.adjacency-matrix)()` and `(g.adjacency-list)()` render the structure as plain Typst tables — no cetz, no animation. Unlike the `*-display` methods they return **placeable content directly** (not an `Array(Frame)`), so drop them straight into the document; wrap them in a `figure` yourself if you want a caption or alt text. Both still honor the render theme — header cells take the node fill and a bold node-text-fill, the rules the node stroke.

By default the matrix shows 1/0 **presence** and the list shows bare neighbor ids. Pass `weights: true` to show each edge's display label instead (its label if set, else its numeric weight); the matrix then marks non-edges with `none-marker` (`·`) and the list appends the weight in parentheses:

	A	B	C	D
A	·	7	4	·
B	7	·	2	1
C	4	2	·	5
D	·	1	5	·

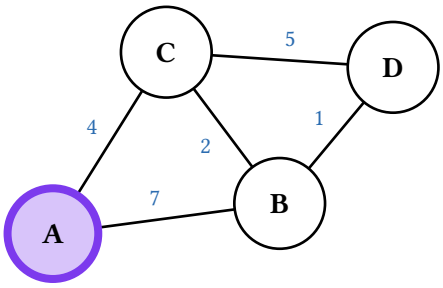
A	B (7), C (4)
B	A (7), C (2), D (1)
C	B (2), A (4), D (5)
D	C (5), B (1)

For a **directed** graph the matrix is asymmetric — row = source, column = target — and the list gives each node's out-neighbors (a sink shows the `empty-marker`, –). An undirected graph gives a symmetric matrix and lists every incident neighbor. A self-loop lands on the matrix diagonal and appears once in the list. Absent matrix cells (the `0`s or `none-marker`) and empty rows are drawn muted so the populated entries read first.

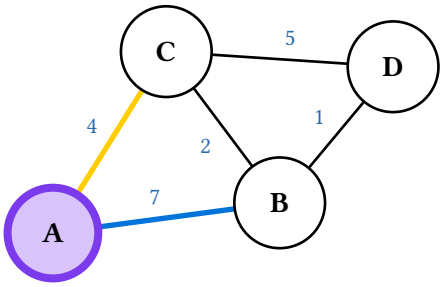
8.4. Prim's minimum spanning tree

`(g.mst-prim-display)(start)` grows the tree from `start`. Each selection shows the frontier (crossing edges in the search stroke) with the lightest edge picked out in the attention stroke, then commits it to the tree (success stroke) and settles the newly reached node. The caption tracks the running tree weight. A

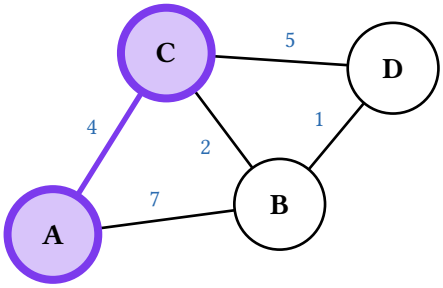
terminal **prune** frame then hides every non-tree edge, so the animation ends on the spanning tree alone (the same close as BFS/DFS `spanning-tree` mode).



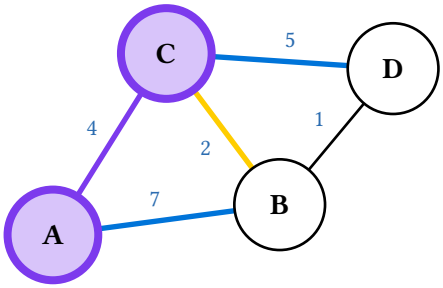
Start at A



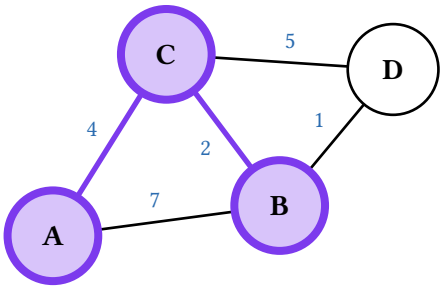
Min crossing edge: A–C (4)



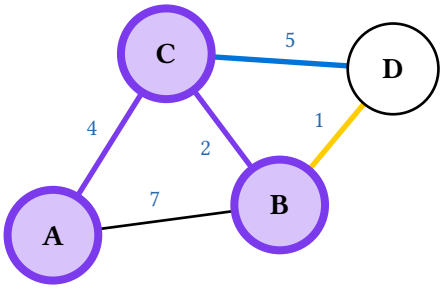
Add A–C tree weight 4



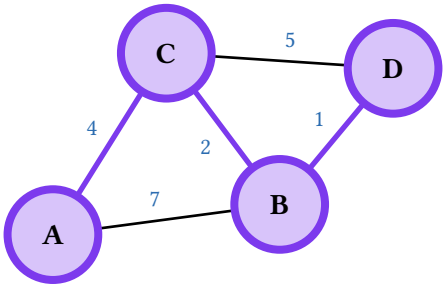
Min crossing edge: B–C (2)



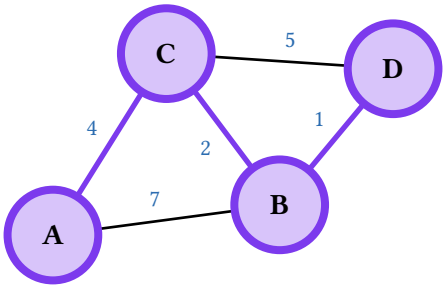
Add B–C tree weight 6



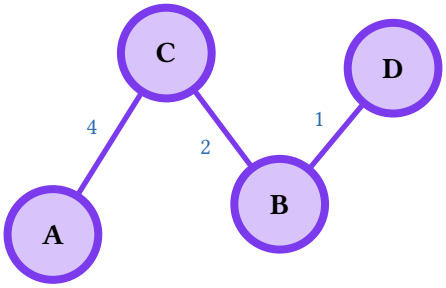
Min crossing edge: B–D (1)



Add B–D tree weight 7

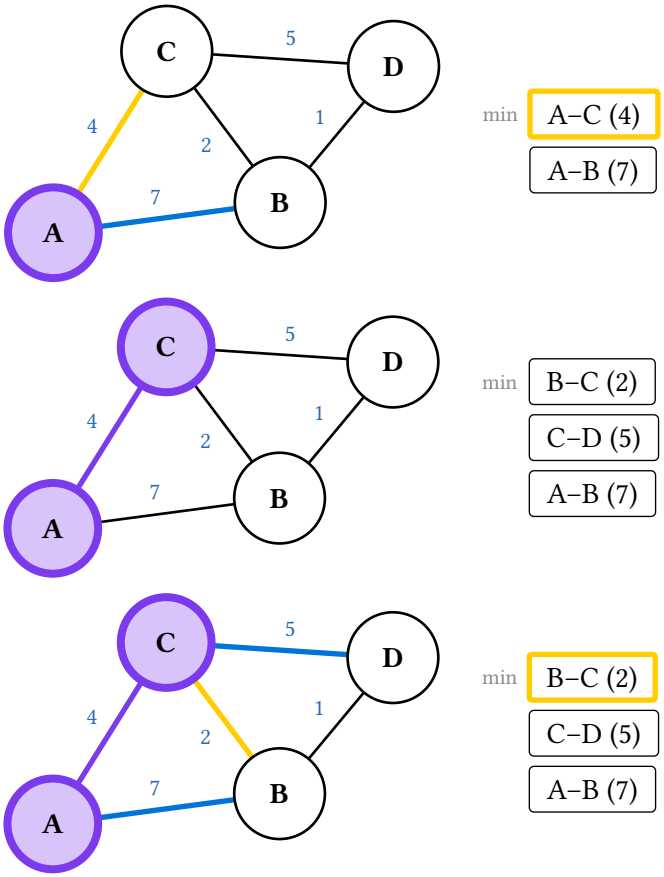


MST weight 7



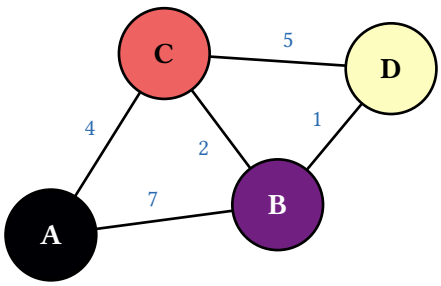
Spanning tree

Prim's bookkeeping is a **priority queue** of the crossing edges, and `aux-strip(frame.step)` renders it beside the canvas — the candidate edges stacked vertically (a queue's natural orientation, and it stays narrow when the frontier is wide), sorted by weight with the `min` on top and the chosen edge ringed in the attention stroke, so the pop is legible step by step. Here are the two selection rounds where the frontier reorders:

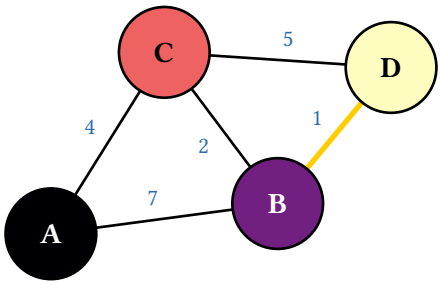


8.5. Kruskal's minimum spanning tree

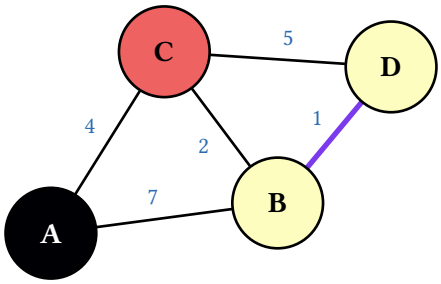
`(g.mst-kruskal-display)()` considers edges in weight order, adding each unless it would join two nodes already in the same component (a cycle, drawn in the danger stroke). The union-find forest is shown by **node color**: same color means same set, and colors merge as the components do — no extra dependency, and robust to any layout. As with Prim, a terminal **prune** frame hides the non-tree edges to finish on the spanning tree.



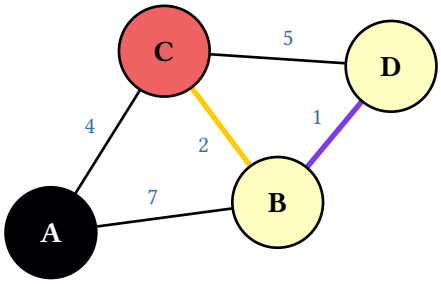
Sort edges by weight



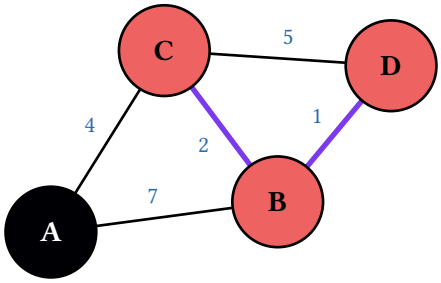
Consider B–D (1)



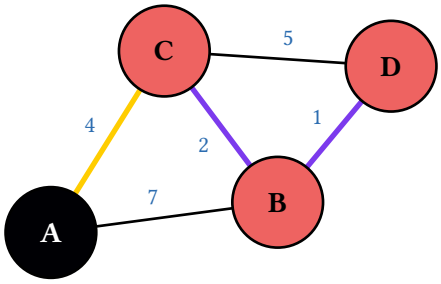
Add B–D weight 1



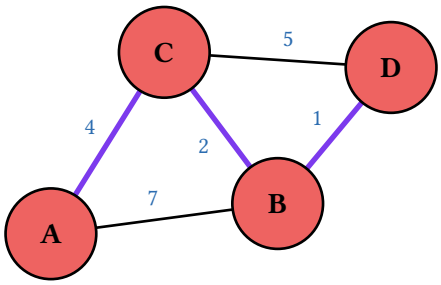
Consider B–C (2)



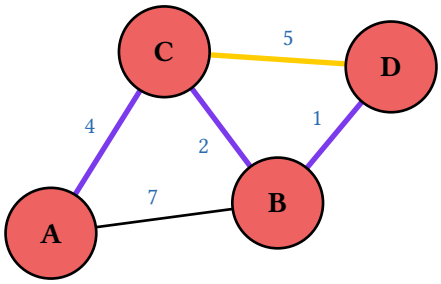
Add B–C weight 3



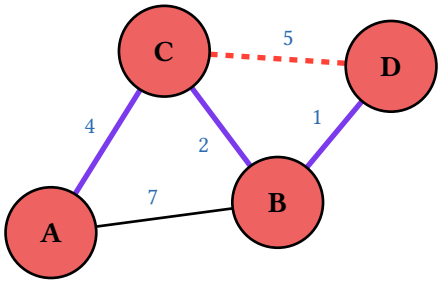
Consider A–C (4)



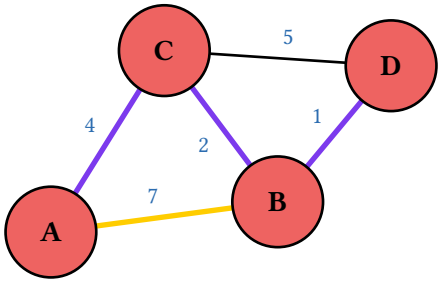
Add A–C weight 7



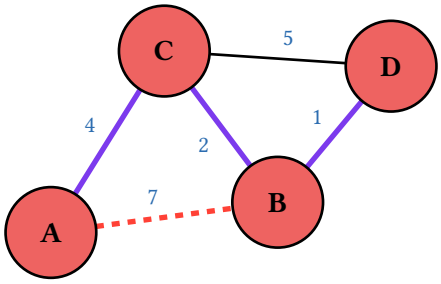
Consider C–D (5)



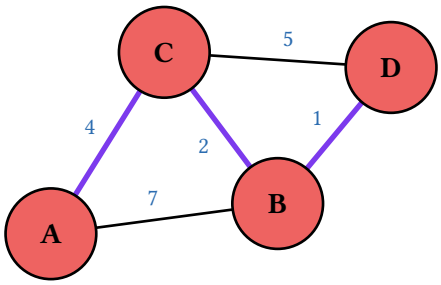
Reject C–D (cycle)



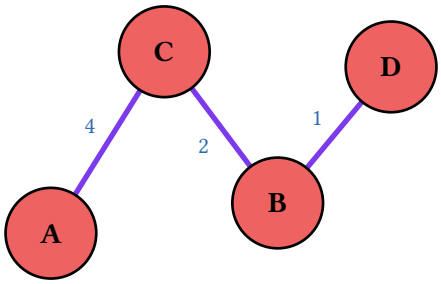
Consider A–B (7)



Reject A–B (cycle)

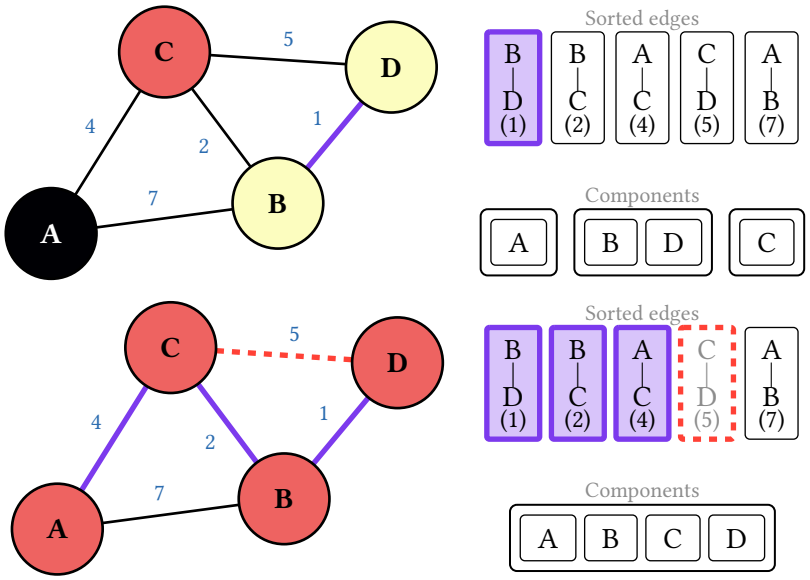


MST weight 7



Spanning tree

Kruskal carries **two** auxiliary structures, and `aux-strip(frame.step)` stacks both: the sorted edge list — every edge in weight order, a cursor (▲) under the one being considered, each tagged added / rejected / pending — and the disjoint-set partition (one bordered group per component, merging as edges are added). The edge labels are stacked vertically (endpoints over the weight) so the list stays compact even with many edges. A committed edge (three components remain) and a later cycle-rejected edge (all merged):

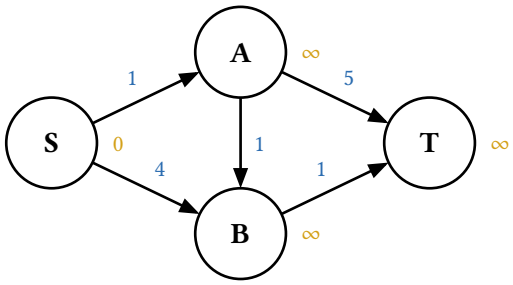


Pass `view:` to render just one of them for separate placement — e.g. `aux-strip(frame.step, view: "partition")` for the components alone.

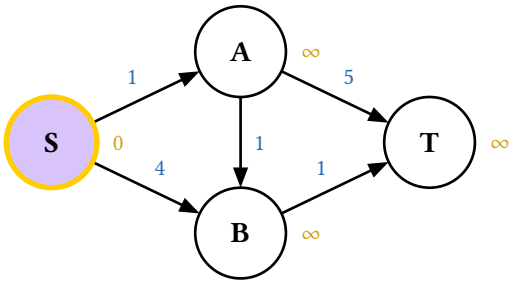
8.6. Dijkstra's shortest paths

`(g.dijkstra-display)(source, target: ..)` carries each node's tentative distance in its note slot (∞ until reached). Each round finalizes the nearest unvisited node (attention stroke, then settled), relaxes its

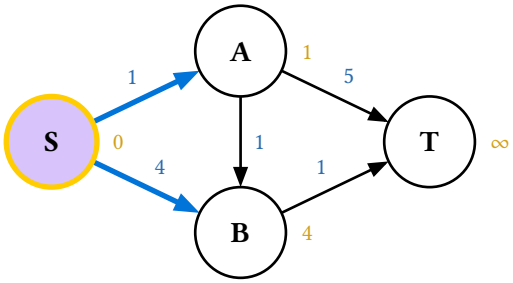
outgoing edges (search stroke, updating distances), and grows the shortest-path tree in the success stroke. With a `target` the search stops early and the path is highlighted. It works on directed graphs:



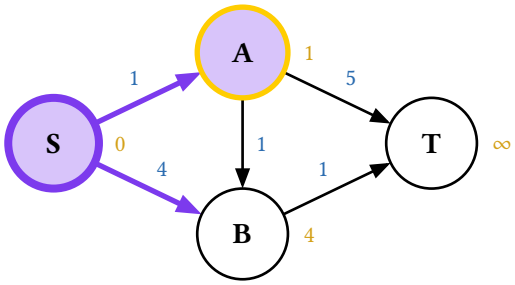
Initialize: S = 0



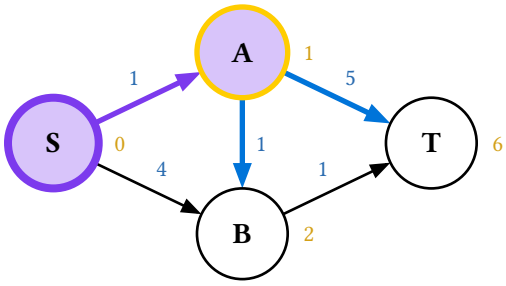
Visit S (0)



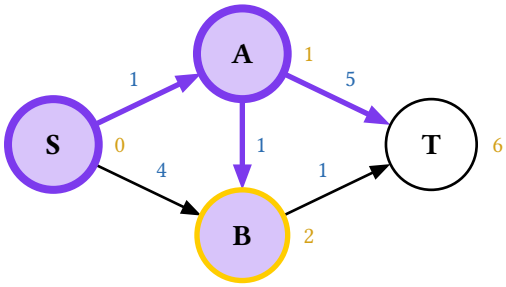
Relax from S



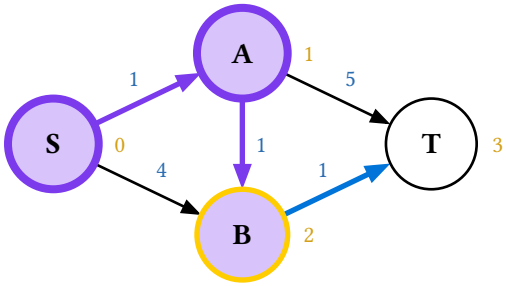
Visit A (1)



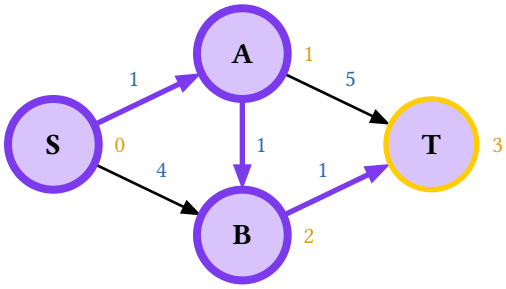
Relax from A



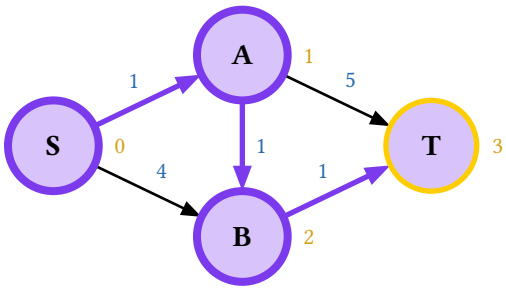
Visit B (2)



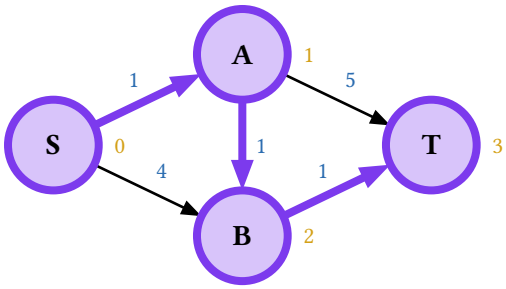
Relax from B



Visit T (3)



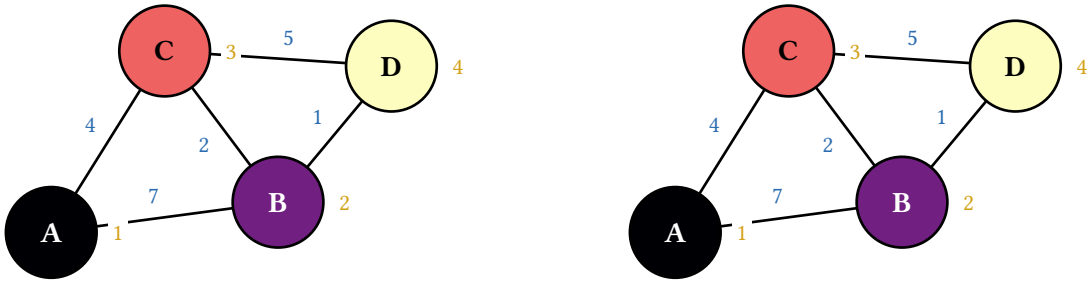
Relax from T



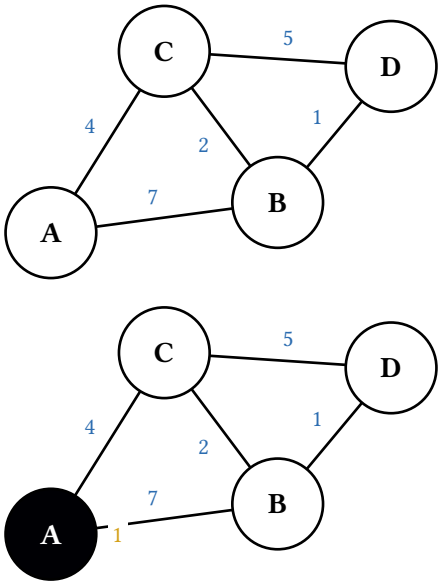
Shortest path to T: 3

8.7. Traversals

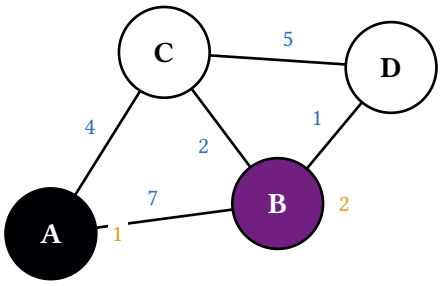
`(g.bfs-display)(start)` and `(g.dfs-display)(start)` sample the op-theme's `traversal-palette` across the visit order — each visited node gets a fill from the gradient and a 1-indexed badge, and the caption accumulates the visit sequence (the same pattern as the tree `*-order-display` methods). The final frame wears the full visit signature:



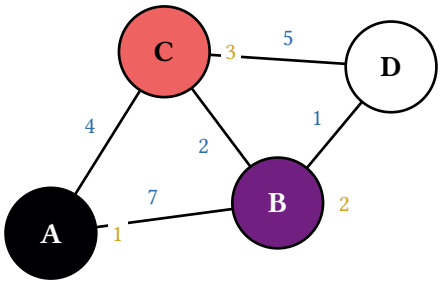
Pass a `target:` node id to either method to turn the traversal into a **search** that stops the moment the target is visited (mirroring `dijkstra-display`'s `target:`). The visit gradient builds up only over the nodes examined before the target, and a terminal frame states the outcome: on success the target gains a `settled-stroke` ring and a `Found <t>` caption; if the target is unreachable the search visits the whole reachable component and ends with a `<t> not found` frame.



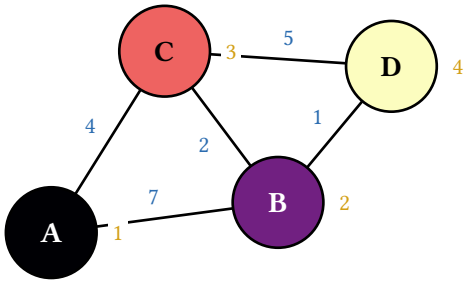
Visited: `[A]`



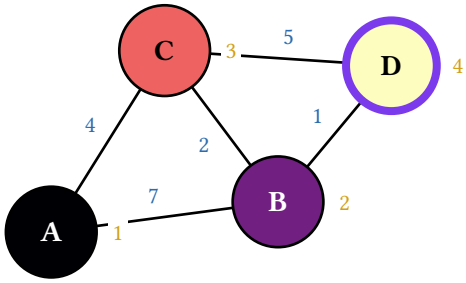
Visited: `[A, B]`



Visited: `[A, B, C]`



Visited: `[A, B, C, D]`



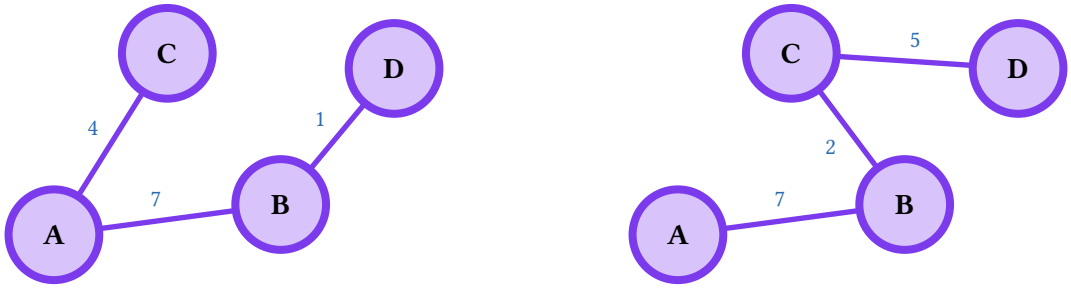
Found D

By default each node's neighbours enter the queue / stack in edge-declaration order. Pass `sort-frontier: true` to instead visit them in ascending node-id order — a lexicographic string sort, so `"A"` before `"B"` and `"0"` before `"1"` — giving a deterministic traversal that doesn't depend on the order edges were added. (Being lexicographic, `"10"` sorts before `"2"`; single-character ids sort as expected.)

8.7.1. Spanning trees

Pass `spanning-tree: true` to either method to render the **traversal tree** rather than the palette walk. Instead of shading nodes across the `traversal-palette`, each node joins the tree with a uniform commit style (`success-fill` + `settled-stroke`) and its **discovery edge** — the edge from the node that first reached it — lights up in `success-stroke`. The highlighted edges accumulate into the BFS / DFS spanning

tree of the reachable component (the same visual vocabulary as `mst-prim-display`, just committing edges in traversal order rather than by weight). Combine with `sort-frontier: true` to pin the tree's shape deterministically. The mode spans the whole reachable component, so it doesn't take a `target:`. A final frame then prunes every non-tree edge — hiding the cross edges and their weights — so the animation ends on the spanning tree by itself (that terminal frame is what `last` shows below).

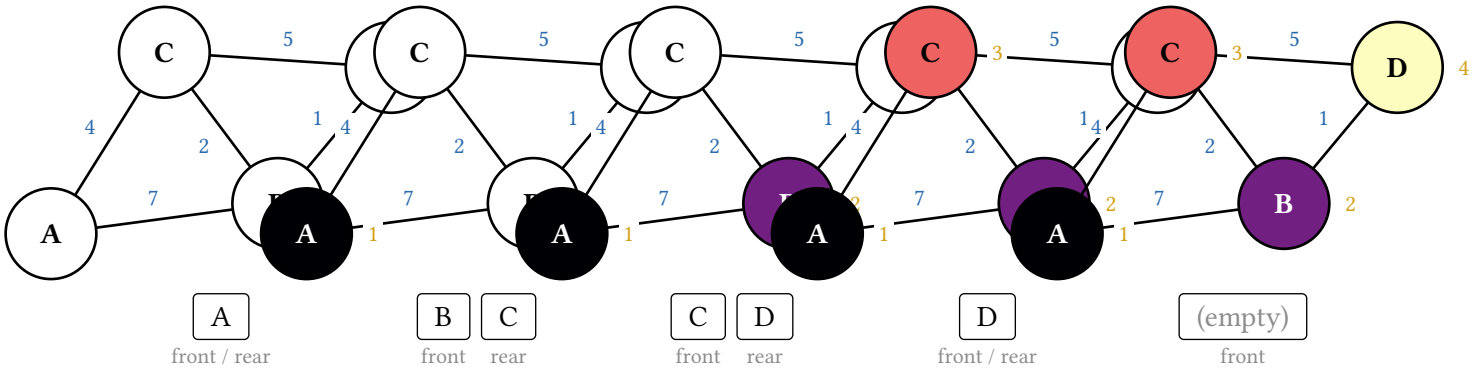


8.7.2. Tracking the queue / stack

BFS and DFS are driven by a helper structure — a FIFO queue and a LIFO stack respectively — and a common sticking point for students is tracking its contents step by step. Every `bfs-display` / `dfs-display` frame records that structure's state in its `step` metadata, and `aux-strip(frame.step)` renders it as a placeable strip of boxes (the front and rear ends marked on a queue, the `top` push/pop end on a stack). The same `aux-strip` helper also serves the MST displays (the Prim frontier and the two Kruskal views, above); it dispatches on the kind of auxiliary state each `step` carries. It returns ordinary content, not a `Frame`, so you lay it out wherever you want — this is deliberately decoupled from the graph canvas so a touying slide can put the graph on one side and the strip on the other:

```
#let frames = (g.bfs-display)("A")
#grid(
  columns: (2fr, 1fr),
  alternatives(..starling.canvases-only(frames)),
  alternatives(..frames.map(f => aux-strip(f.step))),
)
```

Statically, pairing each canvas with its strip shows the queue draining and refilling across the traversal:



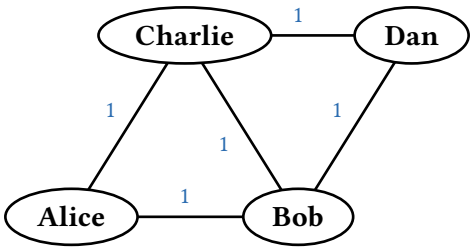
The DFS stack is shown **faithfully**: because the iterative algorithm can push a node before an earlier copy is popped, the same id may appear twice in the strip — exactly the state a hand-trace would produce. Pass a `labels: (id -> content)` map to `aux-strip` when your nodes carry custom display labels.

8.8. Node shapes and sizing

The default circular node is sized for short ids. Longer labels — names, words, multi-character keys — overflow it. Pass a `node-style:` dict to any display to restyle **every** node at once; it sits beneath the per-frame algorithm styling, so shape and size persist while fills and strokes animate on top.

- `shape` accepts `"circle"` (default), `"ellipse"`, or `"rectangle"`/`"square"`.
- `autosize: true` fits the node to its own label (measured at render time), so each node is exactly as wide as it needs to be.
- For a fixed size, set `rx`/`ry` (ellipse / rectangle half-extents) or `r` (circle radius) in cetz units instead, plus `pad-x`/`pad-y` to loosen or tighten an autosize fit.

Edges trim to whatever boundary the shape defines, so arrowheads and edge ends stay flush against a wide ellipse just as they do a circle.



8.9. Optional auto-layout with graphviz

For graphs too large to place by hand, `auto-layout` computes positions with graphviz via the `diagraph-layout` package — a WASM build, so no external binary is needed. It is re-exported from the package entrypoint alongside everything else, so import it the same way and feed its result to any display via the `positions:` argument:

```
#import "@preview/starling:0.3.0": graph, auto-layout, last

// Position-less graph — bare ids, no coordinates.
#let g = graph(("A", "B", "C"), edges: (("A", "B", 7), ("B", "C", 2)))
#last((g.mst-prim-display)("A", positions: auto-layout(g)))
```

Or skip the intermediate map entirely: pass `layout: <engine>` to any display and it runs `auto-layout` for you, internally. Because the display knows the node labels, it feeds graphviz a per-node size estimate so wide labels get spread apart (and sets `overlap=false` so the force-directed engines honor those sizes rather than stacking nodes). This keeps the graph inline at the call site and removes the chance of pairing a positions map with the wrong graph:

```
#last((g.display)(
  node-style: (shape: "ellipse", autosize: true),
  layout: "neato",          // the display calls auto-layout itself
))
```

That size estimate is a cheap label-length heuristic computed **without** `measure`, whereas the drawn node is measured exactly — so the two only approximate each other. Graphviz's node margins absorb most of the slack; if spacing still looks off, spread the graph with the display's `scale:` or tighten/loosen it with

`layout-unit:` (the per-unit point scale, mirroring `auto-layout`'s `unit:`). `layout:` defaults to `none`, in which case the display uses `positions:` and `diagraph-layout` is never touched.

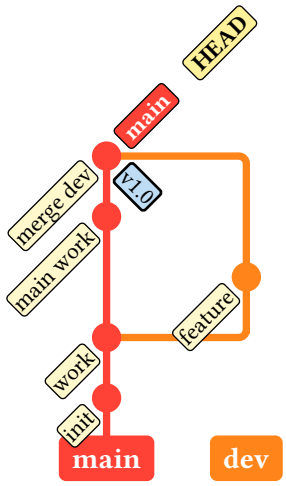
Despite being on the `import starling` path, `diagraph-layout` stays an **optional** dependency: `auto-layout` carries its `diagraph-layout` import inside its own body, which Typst resolves lazily — only when the function is actually called (whether directly or through a display's `layout:` argument). Projects that only hand-place graphs never pull it in. (Typst has no subpath package import, so re-exporting from the entrypoint is the only way to reach `auto-layout`; the older `@preview/starling:<ver>/src/graph-layout.typ` form never worked.)

9. Git graph

The `git-graph` DSL draws git commit graphs — commits, branches, merges, tags, and HEAD/branch pointers. It is the one part of starling that does **not** ride the `Frame` / `Renderer` / `*-display` stack: rather than an immutable structure animated step by step, it is a **stateful, imperative cetz builder**. You call `commit` / `branch` / `merge` inside a `git-graph({ .. })` block and the commands mutate cetz's canvas context as they draw. Consequently the frame helpers (`last`, `stacked`, `figures`) do not apply here — you place a `git-graph` block directly inside a `cetz.canvas(..)`. Animation is touying-native instead: put `pause` / `alternatives(..)` markers inside the canvas body, or redraw a commit dot with `git-highlight`.

Because its verbs use generic names (`commit`, `branch`, `merge`, `tag`, `checkout`), they live behind a `git` namespace rather than being re-exported flat, so `import starling: *` never collides with them:

```
#import "@preview/cetz:0.5.2"
#import "@preview/starling:0.3.0" as starling
#import starling: git

#cetz.canvas(git.git-graph({
  git.branch("main")
  git.commit("init")
  git.commit("work")
  git.branch("dev")
  git.commit("feature")
  git.checkout("main")
  git.commit("main work")
  git.merge("dev", message: "merge dev")
  git.branch-pointer("main")
  git.head-pointer()
  git.tag("v1.0")
}))
```



You must create the initial branch (`branch("main")`) before the first `commit`. `merge(branch)` takes the **branch name** to merge in; `commit`, `branch`, and `merge` accept a `name:` so later annotations survive reordering. `detached-commit(from, msg, name)` draws an orphan dot and `head-pointer(target: name)` hangs a detached HEAD off it.

Pass `direction: "left-to-right"` to `git-graph` to lay commits out rightward (lanes spread downward) instead of the default upward stacking; sensible label angles/anchors switch automatically.

9.1. Theming the git graph

git-graph carries its own per-DS theme, `default-git-theme`, exactly like the RBT palette (see Section 10). Its keys are the branch `colors` palette and the `lane-style` / `graph-style` / `commit-style` / `tag-style` / `pointer-style` sub-dicts. Override document-wide with `set-git-theme(..)` or per-call with `git-graph(theme: (..))`:

```
// Document-wide (state — persists for the rest of the document):
#starling.set-git-theme((graph-style: (stroke: (thickness: 0.4em), radius: 0.15)))

// Per-call (bypasses state entirely — the perf escape hatch):
#cetz.canvas(git.git-graph(
  theme: (colors: (teal, maroon, olive)),
  { git.branch("main"); git.commit("a"); .. },
))
```

Merging is **shallow** at the top level: overriding `commit-style` replaces the whole sub-dict, so pass a complete style dict for the role you change. As elsewhere in starling, the state path costs an extra layout pass (see Section 10.3) — prefer the per-call `theme:` argument on hot paths.

10. Theming

Starling exposes three theme layers so document authors can match starling's palette to their own document style:

- **Render theme** (`default-render-theme`) — the structural defaults the renderer falls back on for the unstyled tree: `node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`.

- **Op theme** (`default-op-theme`) — the operation-semantic colors and strokes that any data structure's `*-display` methods use to communicate operation state: `search-stroke`, `attention-stroke`, `success-stroke`, `settled-stroke`, `success-fill`, `danger-stroke`, `reset-stroke`, `traversal-palette`. Shared across data structures — BST search, RBT search, and a future heap delete all read the same `search-stroke`. Strokes are full stroke dicts (`(paint:, thickness:, dash:)`) so any aspect — color, width, dash — is overridable without adding more keys.
- **Per-DS theme** — styling intrinsic to one data structure. RBT has `default-rbt-theme` with `red-fill`, `red-stroke`, `red-text-fill`, `black-fill`, `black-stroke`, `black-text-fill` for the red/black palette. BST has no per-DS theme of its own because BST nodes carry no intrinsic styling.

10.1. Setting a theme for the whole document

Call any combination of `set-render-theme`, `set-op-theme`, and the per-DS setters (e.g. `set-rbt-theme`) once near the top of your document. Each override is state-based and scoped by Typst's normal layout flow, so it propagates to every subsequent `*-display` call.

```
#import "@preview/starling:0.1.0": BST, set-op-theme, set-render-theme

#set-op-theme((
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  settled-stroke: (paint: olive, thickness: 3.5pt),
  success-fill: olive.lighten(75%),
))
#set-render-theme((node-fill: yellow.lighten(85%)))
```

Pass a partial dictionary — only the keys you list are changed. Unknown keys panic so typos surface immediately rather than silently falling back to defaults.

10.2. Per-call overrides

For one-off variations, every `*-display` method accepts `theme:` and `render-theme:` arguments. A partial dict is merged into the defaults and used in place of the state value for that call only.

```
(t.search-display)(6, theme: (search-stroke: (paint: olive, thickness: 2pt)))
```

10.3. Performance: state-based theming costs a layout pass

Typst's state machinery is what makes `set-op-theme` / `set-render-theme` work: a `state.update` later in the document can affect renders earlier in document order, so Typst evaluates the document, propagates state, and re-evaluates anything that observed it. In practice that means **using either state setter roughly doubles the compile time of the state-observed portion of the document** — a small fixed cost on top of each tree, plus a full extra layout pass through everything that placed a starling render helper or rendered a frame's `render` builder against that state.

Concretely, a 50-tree synthetic benchmark goes from 1.85s to 3.1s when `set-op-theme` is added. The cost scales with document size, not with the number of state reads (one read or a thousand is the same), because the price is "Typst does a second layout pass", not "the read is slow".

This is a structural property of Typst's state model, not something starling can work around without giving up state. If compile speed matters more than the ergonomic win, hoist your theme dict into a plain

`let` and pass it to each `*-display` call via the per-call `theme:` argument — that path skips state entirely and runs at the no-state baseline:

```
#let palette = (
  search-stroke: (paint: teal, thickness: 2.5pt),
  success-stroke: (paint: olive, thickness: 2.5pt),
  // ...
)

#starling.stacked((t.search-display)(6, theme: palette))
#starling.stacked((t.insert-display)(5, theme: palette))
// ...
```

This is the usual perf/ergonomics tradeoff: `set-op-theme` is one declaration at the top of the document and "just works"; per-call `theme:` is more typing but avoids the extra pass.

11. Touying composition examples

Starling is layout-agnostic. In a touying deck, the simplest use splats `figures` into `alternatives`:

```
#import "@preview/touying:0.7.3": *
#import "@preview/starling:0.2.0" as starling
#import starling: BST

== Searching
#alternatives(..starling.figures((t.search-display)(6)))
```

Each frame becomes a subslide; touying handles the rest.

11.1. Captions in a sidebar

When the visual animation should sit alongside synchronised text elsewhere on the slide, pull the captions out separately:

```
== Searching for 6
#let frames = (t.search-display)(6)
#grid(columns: 2, column-gutter: 2em,
  alternatives(..starling.figures(frames, caption: false)),
  alternatives(..frames.map(f => f.caption)),
)
```

Both `alternatives` calls produce the same number of subslides, so they step in lockstep.

11.2. A live queue / stack beside the graph

For BFS and DFS, `aux-strip` turns each frame's helper structure into a strip you can place anywhere on the slide. Drive three `alternatives` off the same frame array — the graph canvas, the queue strip, and the caption — and they share one subslide count, so the whole slide advances together as you step through it:

```
== Breadth-first search
#let frames = (g.bfs-display)("A")
```

```
#grid(columns: (2fr, 1fr), column-gutter: 2em, align: horizon,
  alternatives(..starling.canvases-only(frames)),
  stack(dir: ttb, spacing: 1.2em,
    alternatives(..frames.map(f => starling.aux-strip(f.step))),
    alternatives(..frames.map(f => f.caption)),
  ),
)
```

Each `alternatives` child here is independent content (the strip and the canvas resolve their own theme state), which is exactly what touying wants — the layout-time `alternatives` mark sits **outside** every `context` block, never inside one. Swap `bfs-display` for `dfs-display` to watch the stack instead; the duplicate entries it shows are faithful to the iterative algorithm.

11.3. Driving layout from step metadata

`frame.step` is free-form per-method metadata. For example, a slide that wants to colour-code its narration by step kind:

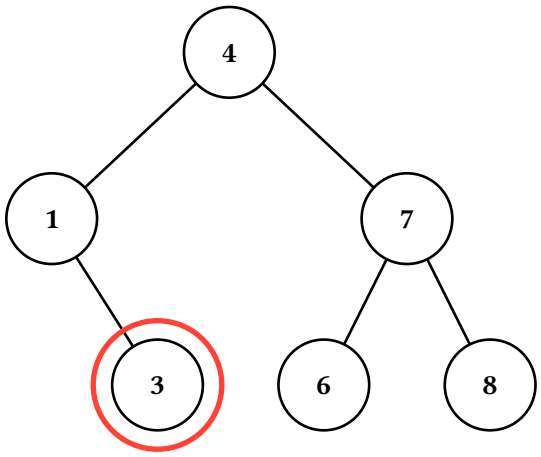
```
#let kind-colors = (compare: blue, inserted: green)

#let captioned-frames = frames.map(f => figure(context {
  let op = starling._op-theme-state.get()
  let rt = starling._render-theme-state.get()
  let color = kind-colors.at(f.step.kind, default: black)
  stack(dir: ttb, spacing: 0.5em,
    (f.render)(op, rt), text(fill: color, f.caption))
}))

#alternatives(..captioned-frames)
```

`frame.render` is a builder `(op, rt) => content` rather than pre-baked content, so themes resolve at layout time. The `context` block above reads the active themes and feeds them in; if you don't need state-driven theming, you can call `(f.render)(starling.default-op-theme, starling.default-render-theme)` without the wrapper.

12. Composing with cetz annotations

For callouts or overlays that need to track specific nodes, drop down to the cetz layer. `draw-tree(tree, snapshot, ...)` emits the tree's cetz drawables without wrapping them in `cetz.canvas`, so you can compose them with your own draw commands in a shared canvas. `path-anchor(path)` translates an L/R path string to the cetz anchor name of the corresponding node.

```
#import "@preview/cetz:0.5.2"

#cetz.canvas({
  starling.draw-tree(t, starling.blank-snapshot())
  import cetz.draw: *
  circle(starling.path-anchor("LR"), radius: 0.85, stroke: red + 2pt)
})
```

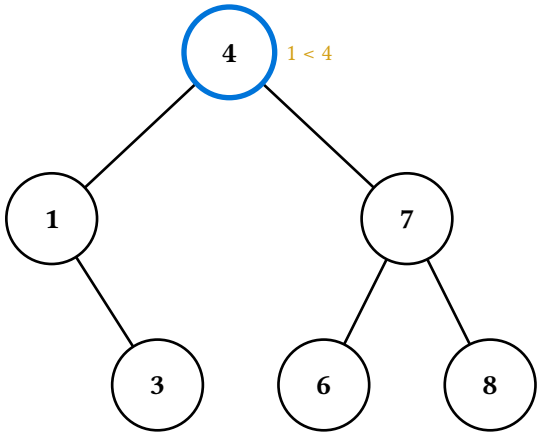


The same trick combines with frame canvases: each frame's `canvas` is itself a `cetz.canvas` block, so for static "annotate the final frame" use you can either render once via `draw-tree` (as above) or extract the cetz block from a frame and add to it externally.

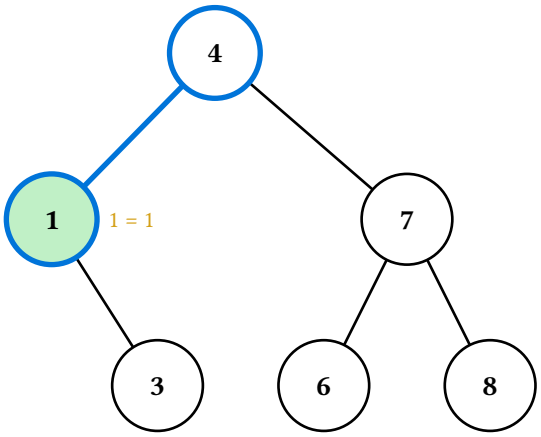
13. The Op command stream

For animations that don't fit the BST's built-in methods — say, a custom hash-table probe sequence or a graph traversal — drop down to the `Op` command stream. Build a sequence of declarative ops and fold them into a renderer with `apply-ops`:

```
#let r = starling.make-renderer(t, sticky: true)
#let r = (r.with-caption)([start at root])
#let r = starling.apply-ops(r, (
  (starling.Op.Highlight.new)(path: "", color: blue),
  (starling.Op.Annotate.new)(path: "", text: [1 < 4]),
  (starling.Op.Commit.new)(
    alt: "Comparing the target 1 against the root 4.",
  ),
))
#let r = (r.with-caption)([descend left])
#let r = starling.apply-ops(r, (
  (starling.Op.ClearNotes.new)(),
  (starling.Op.Highlight.new)(path: "L", color: blue),
  (starling.Op.StyleEdge.new)(path: "L", style: (stroke: blue + 2pt)),
  (starling.Op.StyleNode.new)(path: "L", style: (fill: green.lighten(70%))),
  (starling.Op.Annotate.new)(path: "L", text: [1 = 1]),
  (starling.Op.Alt.new)(
    text: "Descended into the left subtree; 1 matches the left child.",
  ),
))
#starling.stacked((r.render)())
```



start at root



descend left

`Op.Commit` closes the current frame, attaches the supplied `alt` text to it, and opens a fresh blank frame. The `alt` argument is required so every frame the command stream produces carries accessible text. The trailing in-progress frame is kept implicitly — no final `Op.Commit` is needed — but use `Op.Alt(text)` (or `r.with-alt(...)` on the returned renderer) to give that last frame its alt text too.

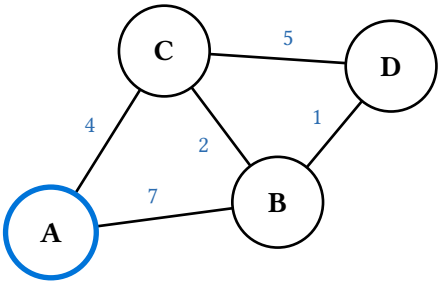
`Op.Caption` does *not* exist; captions and step metadata are renderer-level, set via `r.with-caption(...)` / `r.with-step(...)` directly between Op batches. The split keeps each Op's responsibility narrow (modifying the current snapshot) and matches how the `*-display` methods are written internally.

13.1. On a graph

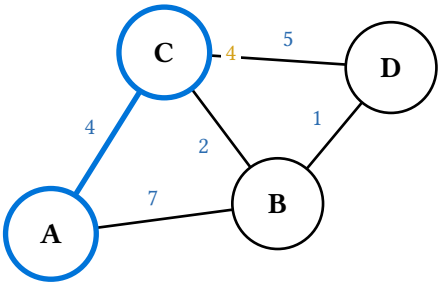
The Op stream is structure-agnostic: an op's `path` is an opaque string, so the same machinery drives a graph. Address nodes by their id and edges by `edge-key(u, v)` (the canonical key the renderer uses — sorted `"u--v"` when undirected, `"u->v"` when directed), and build the renderer with `make-graph-renderer` over a positioned graph instead of `make-renderer`. Everything else — `Op.Highlight`, `Op.StyleEdge`, `Op.Annotate`, `Op.Commit` — is identical.

This is the escape hatch for animations the built-in MST / Dijkstra / traversal displays don't cover. Here we trace a custom walk A ⟶ C ⟶ D, coloring each node and edge and carrying the running cost in the gold note slot (reusing the `g` from the graph tour above):

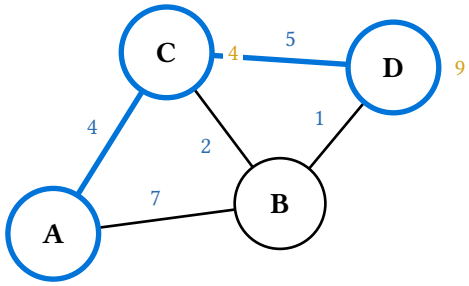
```
#let r = starling.make-graph-renderer((g.positioned)(), sticky: true)
#let r = (r.with-caption)([start at A])
#let r = starling.apply-ops(r, (
  (starling.Op.Highlight.new)(path: "A", color: blue),
  (starling.Op.Commit.new)(alt: "Start the walk at A."),
))
#let r = (r.with-caption)([A → C  (+4)])
#let r = starling.apply-ops(r, (
  (starling.Op.StyleEdge.new)(
    path: starling.edge-key("A", "C"), style: (stroke: blue + 2pt),
  ),
  (starling.Op.Highlight.new)(path: "C", color: blue),
  (starling.Op.Annotate.new)(path: "C", text: [4]),
  (starling.Op.Commit.new)(alt: "Follow edge A–C (weight 4); cost so far 4."),
))
#let r = (r.with-caption)([C → D  (+5)])
#let r = starling.apply-ops(r, (
  (starling.Op.StyleEdge.new)(
    path: starling.edge-key("C", "D"), style: (stroke: blue + 2pt),
  ),
  (starling.Op.Highlight.new)(path: "D", color: blue),
  (starling.Op.Annotate.new)(path: "D", text: [9]),
  (starling.Op.Alt.new)(text: "Follow edge C–D (weight 5); total cost 9."),
))
#starling.stacked((r.render)())
```



start at A



A ⟶ C (+4)



C ⟶ D (+5)

`edge-key` is exported from the package so user code can compute the same key the renderer stores; for a directed graph pass `directed: true` to match. Node and edge ids never collide because they live in separate snapshot dictionaries, so `"A"` (a node) and `"A--C"` (an edge) address different things.

`make-graph-renderer` takes a **positioned** graph, and `(g.positioned)()` defaults to the graph's own stored coordinates. To drive a manual op stream over a graphviz layout instead of hand-placed nodes, compute the positions with `auto-layout` first and thread them through `positioned`'s `positions:` argument before building the renderer — the same seam the displays' `layout:` argument uses internally, done by hand:

```
#let pos = starling.auto-layout(g, engine: "neato", sizes: auto)
#let r = starling.make-graph-renderer(
  (g.positioned)(positions: pos), sticky: true,
)
// ...then apply-ops exactly as above.
```

Pass `sizes: auto` so graphviz reserves room per node (it also sets `overlap=false`) when the nodes are `autosized`; tune absolute spacing with `auto-layout`'s `unit:` or `positioned`'s `scale:`. Keeping this `auto-layout` call explicit is what preserves its laziness — the manual op-stream path never pulls `diagraph-layout` unless you make the call.

13.2. Custom shapes and edge anchors

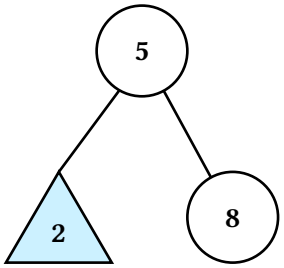
Two node-style and edge-style keys let you swap the default circular node for a triangle or rectangle and steer where the incoming or outgoing edge connects:

- `shape` (node style) accepts `"circle"` (default), `"triangle"` (apex up), or `"rectangle"`. The triangle and rectangle share a 1.4×1.2 bounding box; the circle keeps its 1.2×1.2 footprint.
- `parent-anchor` / `child-anchor` (edge style) accept a cetz anchor name like `"north"` or `"south"` and override the edge endpoint on the parent or child side respectively. The default for both stays the empirical fractional-distance trick that looks clean between two circles.

The canonical use case is using a triangle to stand in for an entire subtree — a common pattern when only the top-level shape matters and the details would clutter the slide. Because the triangle's apex sits at the `north` anchor of its bounding box, an incoming edge with `child-anchor: "north"` lands cleanly on the tip:

```
#let t = (starling.BST.new)(value: 5, label: auto, left: none, right: none)
#let t = (t.insert-many)(2, 8, 1, 3)
```

```
#let ops = (
  (starling.Op.StyleNode.new)(path: "L", style: (shape: "triangle", fill:
aqua.lighten(60%))),
  (starling.Op.StyleEdge.new)(path: "L", style: (child-anchor: "north")),
  (starling.Op.StyleNode.new)(path: "LL", style: (hide: true)),
  (starling.Op.StyleNode.new)(path: "LR", style: (hide: true)),
  (starling.Op.StyleEdge.new)(path: "LL", style: (hide: true)),
  (starling.Op.StyleEdge.new)(path: "LR", style: (hide: true)),
  (starling.Op.Alt.new)(text: "Left subtree summarised as a triangle."),
)
#let r = starling.apply-ops(starling.make-renderer(t, sticky: true), ops)
#starling.last((r.render)())
```



Shape and anchor are deliberately independent — the library doesn't auto-set `child-anchor: "north"` when you switch a node to a triangle, because there are legitimate uses for the default endpoint behavior even on non-circular shapes (e.g. a labelled rectangle whose edges you want pulled toward its center rather than flush against its top). When you want flush meeting points, set the anchors yourself.

14. API reference

The remainder of this manual is an auto-generated reference, produced by the `tidy` package from doc-comments in the source.

14.1. Render helpers

- canvases-only()
- figures()
- last()
- stacked()

14.1.1. canvases-only

Drop captions and step metadata, returning just the array of canvases. Useful when you want to lay out captions yourself — e.g. alongside other slide content in a custom touying layout.

Like `figures()`, this returns one piece of content per frame and therefore can't amortize theme state reads — each canvas resolves state independently. If you're calling this on hot paths under a state-set theme, prefer per-call `theme:` arguments on the `*-display` method.

14.1.1.1. Parameters

`canvases-only(frames: array) -> array`

14.1.1.1.1. frames `array`

The frame array returned by any `*-display` method.

14.1.2. figures

Build an array of `figure` content, one per frame, suitable for splatting into touying's `alternatives(..)` so each frame becomes its own subslide. Starling itself does not depend on touying — the result is just an array of standard Typst figures.

Unlike `last()` / `stacked()`, this helper can't collapse theme state reads to one — each figure is an independent piece of content laid out separately (touying splits them into subslides), so each figure's body resolves theme state in its own `context` block. In many-tree documents that rely on `set-op-theme` / `set-render-theme`, prefer per-call `theme:` arguments on the `*-display` method to skip state altogether when you're using `figures`.

14.1.2.1. Parameters

```
figures(
  frames: array,
  caption: bool,
  alt: auto str,
  caption-spacing: length
) -> array
```

14.1.2.1.1. frames `array`

The frame array returned by any `*-display` method.

14.1.2.1.2. caption `bool`

Whether to stack each frame's caption below its canvas inside the figure body.

Default: `true`

14.1.2.1.3. alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's `alt` field, falling back to the caption (if it's a plain string) and then to a generic `"Animation frame N"` placeholder.

Default: `auto`

14.1.2.1.4. caption-spacing `length`

Spacing between canvas and caption when `caption: true`.

Default: `0.5em`

14.1.3. last

Final frame as a single piece of content. Useful for static renderings in print or for "just show me the end state" usage.

The inline node annotations (drawn next to nodes in the cetz canvas) already label what the final step did, so the textual caption is suppressed by default. Pass `caption: true` to stack the caption below the canvas.

The returned content is wrapped in an alt-tagged figure for accessibility. The alt text comes from the frame's `alt` field; pass an explicit string to `alt` to override.

14.1.3.1. Parameters

```
last(
  frames: array,
  caption: bool,
  spacing: length,
  alt: auto str
) -> content
```

14.1.3.1.1. frames `array`

The frame array returned by any `*-display` method.

14.1.3.1.2. caption `bool`

Whether to render the step's caption below the canvas.

Default: `false`

14.1.3.1.3. spacing `length`

Vertical spacing between canvas and caption when `caption: true`.

Default: `0.5em`

14.1.3.1.4. alt `auto` or `str`

Alt-text override. `auto` uses the frame's `alt` field.

Default: `auto`

14.1.4. stacked

All frames stacked vertically as one block of content. Useful for handouts that want to show the full animation in a single figure. Captions are on by default since the stack is read as a sequence — the caption text annotates which step each tree corresponds to.

Each frame is wrapped individually in an alt-tagged figure so screen-reader users hear the per-step narration. Pass a string to `alt` to set the same override on every frame.

14.1.4.1. Parameters

```
stacked(
  frames: array,
  caption: bool,
  spacing: length,
  caption-spacing: length,
  alt: auto str
) -> content
```

14.1.4.1.1. frames `array`

The frame array returned by any `*-display` method.

14.1.4.1.2. caption `bool`

Whether to include each frame's caption below its canvas.

Default: `true`

14.1.4.1.3. spacing `length`

Vertical spacing between frames.

Default: `1em`

14.1.4.1.4. caption-spacing `length`

Spacing between each canvas and its caption.

Default: `0.5em`

14.1.4.1.5. alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's `alt` field.

Default: `auto`

14.2. Animation core

- apply-ops()
- blank-snapshot()
- concat-frames()
- make-renderer()
- set-render-theme()

Variables

- NodeStyle
- EdgeStyle
- default-render-theme
- RenderTheme
- Snapshot
- Frame
- Renderer
- Op

14.2.1. apply-ops

Fold a sequence of `Op` values into a renderer, returning the updated renderer. The trailing in-progress frame is kept — no explicit `Op.Commit` is needed after the last batch of edits, but attach alt text to that trailing frame via `Op.Alt` (or `r.with-alt(...)` on the returned renderer) so it has accessible text like the committed frames do.

14.2.1.1. Parameters

```
apply-ops(
  renderer: Renderer,
  ops: array
) -> Renderer
```

14.2.1.1.1. renderer `Renderer`

The renderer to apply ops to.

14.2.1.1.2. ops `array`

Sequence of `Op` values to fold left-to-right.

14.2.2. blank-snapshot

Build a fresh `Snapshot` with no node or edge style overrides. Useful when you want to render a structure once with a draw backend and no per-frame state.

14.2.2.1. Parameters

```
blank-snapshot() -> Snapshot
```

14.2.3. concat-frames

Stitch frame arrays from multiple renderers (or pre-rendered arrays) into one flat `Array(Frame)`. Use this when an animation spans a structure-shape change (rotation, deletion, insertion) and one renderer can't hold all the frames — one renderer per shape, joined at the boundary.

Accepts a mix of `Renderer` instances and already-rendered arrays; the former are `render`-ed in place.

14.2.3.1. Parameters

```
concat-frames(..parts) -> array
```

14.2.3.1.1. ..parts

Variadic mix of renderers and frame arrays.

14.2.4. make-renderer

Build a `Renderer` seeded with one blank initial frame, bound to a draw backend.

`draw` is the structure-specific backend wrapped in a singleton dict `(fn: ..)` — see the `Renderer` section comment. Backend-binding wrappers live in each backend module (`make-renderer` in `tree-anim.typ`, `make-graph-renderer` in `graph-draw.typ`).

With `sticky: true` (the default), each new frame pushed via `r.push-frame()` starts from the previous frame's style snapshot — so highlights accumulate over the course of an animation. Set `sticky: false` to start each frame from a clean slate.

With `theme: auto` (the default), each rendered canvas reads the active render theme from state at layout time, so a `set-render-theme(..)` earlier in the document propagates. Pass an explicit dictionary to bake a theme in and skip the state lookup.

14.2.4.1. Parameters

```
make-renderer(
  structure: any,
  draw: dictionary,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  sticky: bool,
  theme: auto dictionary
) -> Renderer
```

14.2.4.1.1. structure `any`

The structure to render — interpreted only by the draw backend.

14.2.4.1.2. draw `dictionary`

Draw backend wrapped in a singleton dict `(fn: ..)`.

14.2.4.1.3. default-node-style `dictionary`

Default node-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.2.4.1.4. default-edge-style `dictionary`

Default edge-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.2.4.1.5. sticky `bool`

When `true`, new frames inherit the previous frame's style. When `false`, each new frame starts blank.

Default: `true`

14.2.4.1.6. theme `auto` or `dictionary`

Render-theme override for this renderer. `auto` reads the active render-theme from state at layout time. A dict is merged into `default-render-theme` once and baked in.

Default: `auto`

14.2.5. set-render-theme

Override one or more render-theme keys for the rest of the document (state-based, scoped by Typst's normal layout flow). Pass a partial dictionary — only the keys you list are changed; the rest stay at their current values. Unknown keys panic.

14.2.5.1. Parameters

```
set-render-theme(theme)
```

14.2.6. NodeStyle

Typsy refinement: a dictionary of node-style overrides. Recognised keys are `fill`, `stroke`, `text-fill`, `note`, `note-fill`, `hide`, `shape`, `tag`, `label`, `materialize`, and `key-styles`. `shape` accepts the string `"circle"` (default), `"triangle"` (apex up; useful as a subtree-summary marker), `"rectangle"`, or `"btree-node"` (B24 subdivided rectangle; reads the node's `keys` array). The triangle and rectangle share a 1.4×1.2 bounding box; the circle keeps its 1.2×1.2 footprint; the `btree-node` width scales as `1.2 × keys.len()`, height stays 1.2. Named cetz anchors on the node's group (`north`, `south`, `east`, `west`) follow each shape's bounding box; `btree-node` additionally exposes `key-<i>` anchors at each compartment center and `gap-<i>` anchors at each child-edge attachment point on the south face. `tag` is a small piece of content drawn just outside the west of the node — useful for compact per-node annotations that don't compete with the label or the operation `note` slot (e.g. the RBT black-height bits enabled by `display(bits: true)`). `label` replaces the node's rendered label (otherwise derived from the structure node's `label` field, or `str(value)` when that's `auto`). For `"btree-node"`, the node has multiple compartments — per-key labels come from the node's `labels` array (entry `auto` falls back to `str(keys.at(i))`); per-compartment styling comes from `key-styles`, an array of dicts (one per compartment) with `fill`/`stroke`/`text-fill` overrides. Use `(:)` in a slot to leave a compartment unstyled. `key-styles` merges index-wise across sticky frames so highlighting compartment 0 in one frame and compartment 1 in the next keeps both annotations. `materialize: true` draws an otherwise-phantom slot as a real node — useful for one-off pedagogical animations that need to expose a nil child (e.g. rendering ∅ at a just-deleted leaf's position). Phantoms only exist at paths the tree either naturally generates as layout-balance siblings or where an edge style sets `force-show: true` materializing without one of those has no node to attach to.

14.2.7. EdgeStyle

Typsy refinement: a dictionary of edge-style overrides. Recognised keys are `stroke`, `note`, `note-fill`, `tag`, `mark`, `hide`, `parent-anchor`, `child-anchor`, `force-show`, and `bend`. The two `*-anchor` keys accept a cetz anchor name (e.g. `"north"`) and override the default fractional-distance endpoint on the parent or child side respectively; useful for connecting edges to non-circular shapes (e.g. `child-anchor: "north"` to land on a triangle's apex). `force-show: true` renders an edge even when one endpoint is a phantom — used by the RBT delete animation to draw a stub edge into a now-nil position when marking double-black. `tag` is a small persistent annotation drawn near the edge midpoint in the render theme's `edge-tag-fill` color — the edge counterpart to a node's `tag`, distinct from the gold operation-level `note` slot. Used by AVL `display(heights: true)` to label each edge with the height of the subtree it points to, and by graphs to label each edge with its weight. `bend` is a graph-only number (cetz units): it curves the edge into an arc whose control point is the straight midpoint pushed perpendicular by this amount (positive = left of the u⟶v direction). Giving a mutual directed pair the same bend fans the two arcs apart so they don't overlap into one line.

14.2.8. default-render-theme

Default render theme. The structural defaults the renderer falls back on when neither a snapshot nor `default-node-style`/ `default-edge-style` specifies otherwise.

14.2.9. RenderTheme

Typsy refinement: a dictionary whose keys are a subset of the render-theme keys (`node-fill`, `node-stroke`, `node-text-fill`, `edge-stroke`, `note-fill`, `edge-tag-fill`, `note-bg`). `note-bg` is the fill drawn behind in-canvas node annotations (the `note` and `tag` slots) so they stay legible over edges — defaults to `white` set it to the page color on a non-white background.

14.2.10. Snapshot

Typsy class representing one sparse style overlay — the per-node and per-edge style overrides for a single frame. Build one via `blank-snapshot()` and chain its `style-node` / `style-edge` / `note-node` / `note-edge` / `clear-notes` methods. Snapshots are rarely constructed directly — they're managed by `make-renderer()` and the data structures' `*-display` methods.

14.2.11. Frame

Typsy class representing one frame in an animation. Fields and methods:

- `render(op-theme, render-theme)` — method that, given resolved theme dicts, produces the cetz canvas for this frame. Theming-aware deferral lives here: by carrying a builder rather than pre-baked content, the lib.typ helpers (`last`, `stacked`, `figures`) can resolve theme state once per call instead of once per frame, which is meaningfully faster in many-structure documents. Per-data-structure themes (e.g. the RBT palette) are read inside each frame's builder rather than passed in — the helpers only carry the two universal layers.
- `caption` — optional textual track for the step (possibly `none`).
- `step` — optional free-form per-method metadata (possibly `none`).
- `alt` — optional accessible text describing the frame (possibly `none`). Carried verbatim into the `alt:` argument on the Typst `figure` wrapping the canvas. The first frame of each operation includes the full structure (via the `describe` method) so screen-reader users have a baseline; subsequent frames describe only the per-step change.

Direct rendering pattern for custom layouts:

```
context {
  let op = _op-theme-state.get()
  let rt = _render-theme-state.get()
  ... (frame.render)(op, rt) ...
}
```

Or simply pass the frame array to one of the lib.typ helpers, which handle the state wrap for you.

The internal `_builder` field carries the actual rendering closure wrapped in a singleton dict so typsy's auto-self-injection doesn't fire on it; treat it as private and call `render` instead.

14.2.12. Renderer

Typsy class accumulating an animation. Holds the structure, a draw backend, parallel arrays of snapshots / captions / steps / alts (one entry per frame), and default node/edge styles. Methods include `push-frame()`, `patch(fn)`, `push-with-node(path, ..style)`, `push-with-edge(path, ..style)`, `push-note-node(path, txt)`, `push-note-edge(path, txt)`, `with-caption(c)`, `with-step(s)`, `with-alt(a)`, and `render()`. Build one via `make-renderer()`.

14.2.13. Op

Typsy enumeration describing a declarative command stream for building animations. Variants:

- `Op.Highlight(path, color)` — set a node's stroke to `color + 2pt`.
- `Op.Annotate(path, text)` — attach an inline note to a node (drawn in-canvas).
- `Op.StyleNode(path, style)` — arbitrary node-style overrides.
- `Op.StyleEdge(path, style)` — arbitrary edge-style overrides.
- `Op.Commit(alt)` — finalise the current frame, attach `alt` text to it, and open a fresh blank frame. `alt` is required so every frame the command stream produces carries accessible text.
- `Op.Alt(text)` — set alt text on the in-progress frame without committing. Primary use is the trailing frame, which has no following `Op.Commit` to carry its alt; can also be used to set alt earlier in a frame's lifecycle if convenient.
- `Op.ClearNotes()` — drop all notes from the current frame's snapshot.

Captions and step metadata are renderer-level rather than snapshot-level: set them between Op batches via `r.with-caption(...)` / `r.with-step(...)`, not through an Op. Fold a sequence of Ops into a renderer with `apply-ops()`.

14.3. Tree backend

- draw-tree()
- make-renderer()
- path-anchor()

Variables

- PathId

14.3.1. draw-tree

Emit the cetz draw commands for one styled snapshot of `tree`, *without* wrapping them in a `cetz.canvas`. Caller is responsible for the canvas wrap. Use this when you want to add your own cetz annotations alongside the tree — for example, callouts anchored at specific nodes.

The whole tree is wrapped in a named cetz group (`name`, default `"tree"`); each non-phantom node has an inner-name of `<node-prefix><cetz-tree-name>` where the cetz-tree name is `"0"` for the root, `"0-0"` for L (or child 0), `"0-1"` for R (or child 1), and so on. Fully qualified anchor names therefore look like `"tree.node-0-0-1"` — use `path-anchor()` to compute them from a path string.

Tree-shape dispatch: if `tree` exposes a `children` field, the n-ary builder is used (no phantom-sibling layout, each child path-segment is a digit indexing into `children`). Otherwise, the binary `.left` / `.right` builder runs.

14.3.1.1. Parameters

```
draw-tree(
  tree: dictionary,
  snapshot: Snapshot,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  render-theme: dictionary,
  name: str,
  node-prefix: str,
  grow: float,
  spread: float
) -> content
```

14.3.1.1.1. tree `dictionary`

The tree to render. Two accepted shapes:

- Binary: any value with `value`, `label`, `left`, and `right` fields (e.g. a `BST` instance). `label` of `auto` falls back to `str(value)` any other value (string, image, content) is rendered as the node's drawn label.
- N-ary: any value with `keys` (array of key values), optional `labels` (parallel array of label overrides), and `children` (array of child subtrees; empty for leaves). Used with `shape: "btree-node"` in the default node-style to render the canonical subdivided rectangle.

14.3.1.1.2. snapshot `Snapshot`

Style overlay for this snapshot. Use `blank-snapshot()` (from the animation core) for an unstyled tree.

14.3.1.1.3. default-node-style `dictionary`

Default node-style overrides applied before per-node overrides.

Default: `(:)`

14.3.1.1.4. default-edge-style `dictionary`

Default edge-style overrides applied before per-edge overrides.

Default: `(:)`

14.3.1.1.5. render-theme `dictionary`

Structural defaults — the lowest layer of the style chain. Defaults to `default-render-theme` pass an already-merged dict (e.g. read from `set-render-theme`'s state inside a `context` block) to apply user overrides.

Default: `default-render-theme`

14.3.1.1.6. name `str`

Outer group name for the whole tree. Must match the `tree-name` argument to `path-anchor()` for anchors to resolve.

Default: `"tree"`

14.3.1.1.7. node-prefix `str`

Prefix attached to each node's cetz-tree internal name. Must match the `prefix` argument to `path-anchor()`.

Default: `"node-"`

14.3.1.1.8. grow `float`

Depth-direction layout factor passed to cetz-tree. Values below `1` shorten edges between levels; values above `1` stretch them. Node sizes are unaffected.

Default: `1`

14.3.1.1.9. spread `float`

Sibling-direction layout factor passed to cetz-tree. Values below `1` pull siblings together; values above `1` push them apart. Node sizes are unaffected.

Default: `1`

14.3.2. make-renderer

Build a tree `Renderer` seeded with one blank initial frame, bound to the `draw-tree()` backend. Thin wrapper over the generic `make-renderer` in `anim-core.typ` that injects the tree draw backend, preserving the historical `make-renderer(tree, ..)` signature.

14.3.2.1. Parameters

```
make-renderer(
  tree: dictionary,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  sticky: bool,
  theme: auto dictionary
) -> Renderer
```

14.3.2.1.1. tree `dictionary`

The tree to render — any value with `value`, `label`, `left`, and `right` fields (binary) or `keys` / `children` (n-ary). See `draw-tree()`.

14.3.2.1.2. default-node-style `dictionary`

Default node-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.3.2.1.3. default-edge-style `dictionary`

Default edge-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.3.2.1.4. sticky `bool`

When `true`, new frames inherit the previous frame's style. When `false`, each new frame starts blank.

Default: `true`

14.3.2.1.5. theme `auto` or `dictionary`

Render-theme override for this renderer. `auto` reads the active render-theme from state at layout time. A dict is merged into `default-render-theme` once and baked in.

Default: `auto`

14.3.3. path-anchor

Translates a starling path-id to the fully-qualified cetz anchor name produced by `draw-tree()`. Path `""` maps to the root anchor; binary paths use `"L"` / `"R"` (mapping to child indices 0 and 1); n-ary paths use digit characters `"0"` to `"9"` that pass through verbatim.

An optional `"#<int>"` suffix on the path (n-ary only) resolves to a per-compartment sub-anchor `key-<int>` on the btree-node — e.g. `path-anchor("01#1")` returns the anchor of the middle key compartment of the leftmost grandchild.

The `tree-name` and `prefix` arguments must match the `name` and `node-prefix` passed to `draw-tree`.

14.3.3.1. Parameters

```
path-anchor(
  path: str,
  tree-name: str,
  prefix: str
) -> str
```

14.3.3.1.1. path `str`

Path-id string identifying a node (optionally followed by `"#<int>"` for a btree-node compartment).

14.3.3.1.2. tree-name `str`

Outer-group name; must match `draw-tree`'s `name`.

Default: `"tree"`

14.3.3.1.3. prefix `str`

Per-node prefix; must match `draw-tree`'s `node-prefix`.

Default: `"node-"`

14.3.4. PathId

Typsy refinement: a path-id string. Binary trees use `"L"` / `"R"` characters (empty string = root, `"L"` = the root's left child, `"LR"` = the root's left child's right child, and so on). N-ary trees use digit characters `"0"` to `"9"` with each digit indexing into the parent's children array. An optional `"#<int>"` suffix addresses an internal key compartment of an n-ary node — used by `path-anchor()`, e.g. `"01#1"` is the middle-key compartment of the leftmost grandchild. The binary and n-ary alphabets are not mixed within a single path.

14.4. Graph

- aux-strip()
- graph()

14.4.1. aux-strip

Render the auxiliary helper structure captured for one animation frame as a placeable strip of boxes — a teaching aid so students can track the algorithm's bookkeeping alongside the graph canvas. Pass a frame's `step` metadata; the strip reads whatever auxiliary state that display stashed:

- `bfs-display` / `dfs-display` — the BFS queue / DFS stack (a single `aux` array + `aux-kind` the DFS stack is shown faithfully, duplicates and all, since a node may be pushed more than once before an earlier copy is popped).
- `mst-prim-display` — the frontier priority queue of crossing edges, min first, the chosen edge ringed.

- `mst-kruskal-display` — two stacked views (an `aux-views` list): the sorted edge list with a cursor and per-edge status, plus the disjoint-set partition (one group per component).

Returns plain Typst content (not a `Frame`), so it drops anywhere — e.g. beside `canvases-only(frames)` in a touying layout. When a step carries several views they stack vertically; pass `view:` to render just one for separate placement.

14.4.1.1. Parameters

```
aux-strip(
  step: dictionary,
  labels: dictionary,
  view: auto str,
  theme: auto dictionary,
  render-theme: auto dictionary
) -> content
```

14.4.1.1.1. step `dictionary`

One frame's `step` metadata dict, as produced by `bfs-display` / `dfs-display` / `mst-prim-display` / `mst-kruskal-display`.

14.4.1.1.2. labels `dictionary`

Optional `id -> content` map giving each node a display label (e.g. to match custom node labels; also used for the endpoints of edge elements). Ids not present fall back to the id string itself.

Default: `(:)`

14.4.1.1.3. view `auto` or `str`

Select a single view by its `aux-kind` when the step carries more than one (e.g. `"partition"` for Kruskal's disjoint sets). `auto` renders every view the step holds, stacked.

Default: `auto`

14.4.1.1.4. theme `auto` or `dictionary`

Op-theme override, used to color MST elements by status (attention / success / danger). `auto` reads the active `set-op-theme` state; a dict bakes it in. Unused by the queue/stack kinds.

Default: `auto`

14.4.1.1.5. render-theme `auto` or `dictionary`

Render-theme override. `auto` reads the active `set-render-theme` state; a dict bakes it in.

Default: `auto`

14.4.2. graph

Factory: build a `Graph` from a positional list of nodes plus a named list of edges. Each node is one of:

- a bare id string (no position — supply positions later, e.g. via `auto-layout` in `graph-layout.typ`, or pass `positions:` to a `*-display` method),
- an `(id, x, y)` tuple (label defaults to the id), or
- an `(id, x, y, label)` tuple.

Each edge is an `(u, v)` tuple (weight defaults to `1`) or `(u, v, weight)`. Positions are in cetz units.

```
// hand-placed
let g = graph(
  (("A", 0, 0), ("B", 3, 1), ("C", 1.5, 2.4)),
  edges: (("A", "B", 7), ("B", "C", 2), ("A", "C", 4)),
)
// position-less (lay out later)
let h = graph(("A", "B", "C"), edges: (("A", "B", 7),))
```

14.4.2.1. Parameters

```
graph(
  nodes: array,
  edges: array,
  directed: bool
) -> Graph
```

14.4.2.1.1. nodes `array`

Positional array of nodes. Each is a bare id, an `(id, label)` 2-tuple (a custom display label, no manual position — handy with `auto-layout`), an `(id, x, y)` tuple (manual position, label defaults to the id), or an `(id, x, y, label)` tuple (both).

14.4.2.1.2. edges `array`

Edges — each an `(u, v)` pair optionally followed by a weight and/or label. The third slot dispatches by type: a number is the edge `weight`, a string/content is a display `label` drawn in place of the weight. An `(u, v, weight, label)` 4-tuple sets both — the numeric weight still drives Dijkstra/MST while the label is what's shown.

Default: `()`

14.4.2.1.3. directed `bool`

Whether the graph is directed (arrowheads + ordered edge keys).

Default: `false`

14.5. Graph renderer

- draw-graph()

- edge-key()
- make-graph-renderer()
- node-anchor()

Variables

- GraphNodeId

14.5.1. draw-graph

Emit the cetz draw commands for one styled snapshot of a positioned graph, *without* wrapping them in a `cetz.canvas`. The caller wraps the canvas — use this to add your own cetz annotations alongside the graph — see `node-anchor()`.

Edges are drawn first (straight lines between node centers) and the filled node glyphs on top, so the line ends are cleanly occluded by the nodes. Each edge's intrinsic weight/label is drawn at its midpoint in the render theme's `edge-tag-fill` a snapshot edge `tag` overrides that text, and a snapshot `note` (gold) draws an operation annotation on the edge. Directed graphs get filled arrowheads (the mark fill follows the edge stroke) unless an edge sets its own `mark` the fill keeps a pair of opposite edges from showing the line through each other's arrowhead. A self-edge (`u == v`) is drawn as a teardrop loop above the node rather than a zero-length line. An edge-style `bend` (cetz units, positive = left of the u⟶v direction) curves the edge into an arc; giving a mutual directed pair the same bend fans the two arcs to opposite sides so they no longer overlap into one line.

14.5.1.1. Parameters

```
draw-graph(
  pg: dictionary,
  snapshot: Snapshot,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  render-theme: dictionary,
  node-prefix: str
) -> content
```

14.5.1.1.1. pg `dictionary`

The positioned graph — see the module header for its shape.

14.5.1.1.2. snapshot `Snapshot`

Style overlay for this snapshot. Use `blank-snapshot()` (from the animation core) for an unstyled graph.

14.5.1.1.3. default-node-style `dictionary`

Default node-style overrides applied before per-node overrides.

Default: `(:)`

14.5.1.1.4. default-edge-style `dictionary`

Default edge-style overrides applied before per-edge overrides.

Default: `(:)`

14.5.1.1.5. render-theme `dictionary`

Structural defaults — the lowest layer of the style chain.

Default: `default-render-theme`

14.5.1.1.6. node-prefix `str`

Per-node cetz element-name prefix. Must match the `prefix` argument to `node-anchor()`.

Default: `"node-"`

14.5.2. edge-key

Canonical snapshot key for the edge between `u` and `v`. Directed graphs key edges by `"u->v"` (order significant); undirected graphs sort the endpoints into `"u--v"` so the same edge has one key regardless of which endpoint is named first.

14.5.2.1. Parameters

```
edge-key(
  u: str,
  v: str,
  directed: bool
) -> str
```

14.5.2.1.1. u `str`

First endpoint id.

14.5.2.1.2. v `str`

Second endpoint id.

14.5.2.1.3. directed `bool`

Whether the graph is directed.

Default: `false`

14.5.3. make-graph-renderer

Build a graph `Renderer` seeded with one blank initial frame, bound to the `draw-graph()` backend. Thin wrapper over the generic `make-renderer` in `anim-core.typ` that injects the graph draw backend.

14.5.3.1. Parameters

```
make-graph-renderer(
  pg: dictionary,
  default-node-style: dictionary,
  default-edge-style: dictionary,
  sticky: bool,
  theme: auto dictionary
) -> Renderer
```

14.5.3.1.1. pg `dictionary`

The positioned graph to render.

14.5.3.1.2. default-node-style `dictionary`

Default node-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.5.3.1.3. default-edge-style `dictionary`

Default edge-style overrides applied before per-snapshot overrides.

Default: `(:)`

14.5.3.1.4. sticky `bool`

When `true`, new frames inherit the previous frame's style.

Default: `true`

14.5.3.1.5. theme `auto` or `dictionary`

Render-theme override; `auto` reads state at layout time.

Default: `auto`

14.5.4. node-anchor

The fully-qualified cetz anchor name for the node `id` drawn by `draw-graph()`. Use it to attach your own callouts to a node when you call `draw-graph` inside your own `cetz.canvas`. The `prefix` must match the `node-prefix` passed to `draw-graph`. Sub-anchors follow (e.g. `node-anchor("A") + ".north"`).

14.5.4.1. Parameters

```
node-anchor(
  id: str,
  prefix: str
) -> str
```

14.5.4.1.1. id `str`

Node id.

14.5.4.1.2. prefix `str`

Per-node name prefix; must match `draw-graph`'s `node-prefix`.

Default: `"node-"`

14.5.5. GraphNodeId

Typsy refinement: a graph node id. Any string.

14.6. Graph auto-layout

- auto-layout()

14.6.1. auto-layout

Compute node positions for `g` with graphviz and return a `(id: (x, y))` map in cetz units, suitable for the `positions:` argument of any `Graph` `*-display` method (or `Graph.positioned`).

diagraph-layout returns node centers in points (y-up, matching cetz); each coordinate is divided by `unit` to convert to cetz units. The default `36pt` (½ inch) gives spacing that suits the renderer's radius-0.6 nodes; lower it to spread the graph out, raise it to pack it tighter. Edge spline points from graphviz are ignored — starling draws straight edges.

`sizes` controls how much room graphviz reserves per node: `none` uses graphviz's default node size (fine for short, circular nodes); `auto` estimates each node's box from its label's character count so wide labels get spread apart; a `(id: (width, height))` dict supplies explicit lengths. The `auto` estimate is intentionally rough — it is computed eagerly (no `measure`, so `auto-layout` stays callable outside a `context`) while `draw-graph` sizes the drawn node **exactly** by measuring the label. The two only approximate each other, but graphviz's node margins absorb the slack; nudge `unit`/the display's `scale:` if spacing looks off. The `*-display` methods pass `sizes: auto` automatically when you use their `layout:` argument.

14.6.1.1. Parameters

```
auto-layout(
  g: Graph,
  engine: str,
  unit: length,
  sizes: none auto dictionary
) -> dictionary
```

14.6.1.1.1. g `Graph`

The `Graph` to lay out. Its positions (if any) are ignored; only nodes, edges, and directedness are used.

14.6.1.1.2. engine `str`

graphviz layout engine — `"dot"` (layered, the default), `"neato"`, `"fdp"`, `"sfdp"`, `"circo"`, or `"twopi"`.

Default: `"dot"`

14.6.1.1.3. unit `length`

Points per cetz unit used to scale graphviz's output.

Default: `36pt`

14.6.1.1.4. sizes `none` or `auto` or `dictionary`

Per-node box sizing fed to graphviz: `none` (graphviz default), `auto` (estimate from label length), or a `(id: (width, height))` dict of explicit lengths.

Default: `none`

14.7. Git graph

- set-git-theme()

Variables

- default-git-theme
- GitTheme

14.7.1. set-git-theme

Override one or more git-theme keys for the rest of the document (state-based, scoped by Typst's normal layout flow). Pass a partial dictionary — only the top-level keys you list are replaced (shallow), the rest stay at their current values. Unknown keys panic.

14.7.1.1. Parameters

```
set-git-theme(theme)
```

14.7.2. default-git-theme

Default styling theme for git graphs. Pass a partial dict to `set-git-theme(..)` (document-wide) or `git-graph(theme: (..))` (per-call) to override individual roles. Each top-level value is a **complete** sub-style dict — merging is shallow.

14.7.3. GitTheme

Typsy refinement: a dictionary whose keys are a subset of the git-theme keys. Used by `set-git-theme` and the per-call `theme:` argument to give early errors on typos.